



西北工业大学
NORTHWESTERN POLYTECHNICAL UNIVERSITY

《软件系统开发综合能力训练课》

实验指导书 (2026 版)

适用专业	软件工程
总学时	48 学时
编写教师	王犇、赵彦发
编写日期	2026 年 1 月

软件学院
实验教学中心

致学生：如何使用本实验指导书

欢迎使用《软件系统开发综合能力训练课》实验指导书。为帮助你最高效地利用本指导书，请在开始前务必阅读并理解以下说明。

一、核心定位

本指导书的根本目的是为你展示企业级软件开发的标准敏捷流程和规范。书中以一个完整的示例项目（如制造执行系统 MES）为贯穿案例，详细说明了从项目启动到收尾的各个环节。

- 你团队的主要任务是：**学习、理解并实践这套流程**，将其应用于你的自选课题中。
- 请勿误解为：逐步模仿、重建一个 MES 系统。案例中的具体功能和技术细节**仅为示例**。

关于迭代次数的说明：

本课程通过 3 次迭代完整展示敏捷项目从启动、改进到成熟的典型演进过程。这一安排旨在帮助你在有限学时内深度体验迭代开发的精髓。请注意，真实项目的迭代次数需根据项目规模、复杂度灵活确定，3 次迭代是教学场景下的优化选择，而非行业固定标准。

二、正确使用

在阅读每个实验时，请按以下方式区分重点：

你应重点关注（通用、必须遵循）	你可参考理解（具体、示例性质）
阶段目标（如：本迭代要交付什么价值）	示例项目中为实现该目标所做的 具体功能决策 （如：MES 的“工单管理”模块）
任务与活动（如：如何召开迭代规划会）	示例项目团队召开会议的具体讨论内容（供参考）
交付物规范（如：《需求规格说明书》应包含哪些要素）	案例《需求规格说明书》中的 具体需求描述
质量标准（如：代码审查的检查清单）	案例代码中的 具体实现方式
协作规范（如：Git 分支管理策略）	案例项目采用的 特定技术栈 （Java/Vue）
方法与思路（如：如何进行技术选型、架构设计）	案例为实现其需求而选择的 具体技术组件 （如 Spring Boot, Vue.js, MySQL）

特别重要：关于技术选型

书中示例项目基于特定技术栈（如 Spring Boot, Vue.js）实现。你应重点学习其**技术选型**的**决策思路、架构设计的原则与方法**，但完全可以根据自身项目的需求特点（如性能要求、团队技术储备、部署环境等），选择更合适的技术方案。

关于你将参考的文档：

为指导你产出符合企业规范的交付物，本指导书附带的 MES 项目文档（如需求规格书、敏捷过程文档等）均是按照**企业级标准**编写的真实文档。这意味着你在参考时，不仅应关注其内容，更应**学习其文档结构、详略程度、表述方式**，并将其作为你为自选课题编写文档的**质量标杆**。

三、实践建议

1. 先通读流程，再动手实践：在开始每个实验前，先完整阅读相应章节，理解本阶段在整体流程中的位置和目标。
2. 建立映射关系：阅读案例时，不断思考：“如果是我的项目，这个环节该怎么做？交付物应该怎么写？”
3. 灵活应用：在遵循流程规范的前提下，鼓励根据项目特点对具体实践进行合理调整和创新。

遵循以上原则，本指导书将成为你的团队顺利完成课程项目、提升软件工程综合能力的强大助力。

目录

致学生：如何使用本实验指导书	i
一、核心定位	i
二、正确使用	i
三、实践建议	ii
引言	1
实验一：敏捷项目启动（团队与需求基线搭建）	11
【实验目标】	11
【实验内容】	11
任务 1：完成自组织团队组建	11
任务 2：完成所选课题需求分析	12
任务 3：项目开发计划编制	13
任务 4：搭建开发环境	13
【实验要求】	14
【实验环境】	15
【实验步骤】	16
阶段 1：自组织团队组建	16
阶段 2：需求分析	17
阶段 3：项目开发计划编制	28
阶段 4：搭建开发环境	32
【实验报告】	39
【实验中的常见问题与解决方案】	39
【实验或拓展思考题】	39
【评分标准】	40
【参考教材与实验拓展】	40
【注意事项】	41
实验二：迭代 1（从规划到回顾的首次价值交付）	42
【实验目标】	42
【实验内容】	42
任务 1：完成第一次迭代规划	42
任务 2：完成第一次迭代冲刺	43

任务 3: 完成第一次冲刺评审	44
任务 4: 完成第一次冲刺演示与交付	44
任务 5: 完成第一次迭代回顾	45
【实验要求】	45
【实验环境】	47
【实验步骤】	47
阶段 1: 迭代规划	47
阶段 2: 迭代冲刺	50
阶段 3: 迭代冲刺评审	53
阶段 4: 冲刺演示与交付	55
阶段 5: 冲刺回顾	58
【实验报告】	60
【实验中的常见问题与解决方案】	60
【实验或拓展思考题】	61
【评分标准】	61
【参考教材与实验拓展】	62
【注意事项】	62
实验三: 迭代 2 (基于回顾的增量交付与流程优化)	63
【实验目标】	63
【实验内容】	63
任务 1: 完成第二次迭代规划	63
任务 2: 完成第二次迭代冲刺	64
任务 3: 完成第二次冲刺评审	65
任务 4: 完成第二次冲刺演示与交付	65
任务 5: 完成第二次迭代回顾	66
【实验要求】	66
【实验环境】	67
【实验步骤】	68
阶段 1: 迭代规划	68
阶段 2: 迭代冲刺	70
阶段 3: 迭代冲刺评审	72
阶段 4: 冲刺演示与交付	73

阶段 5: 冲刺回顾	75
【实验报告】	77
【实验中的常见问题与解决方案】	77
【实验或拓展思考题】	78
【评分标准】	78
【参考教材与实验拓展】	78
【注意事项】	79
实验四: 迭代 3 (成熟实践下的高效价值交付)	80
【实验目标】	80
【实验内容】	80
任务 1: 完成第三次迭代规划	80
任务 2: 完成第三次迭代冲刺	81
任务 3: 完成第三次冲刺评审	81
任务 4: 完成第三次冲刺演示与交付	82
任务 5: 完成第三次迭代回顾	83
【实验要求】	83
【实验环境】	84
【实验步骤】	85
阶段 1: 迭代规划	85
阶段 2: 迭代冲刺	87
阶段 3: 迭代冲刺评审	89
阶段 4: 冲刺演示与交付	92
阶段 5: 冲刺回顾	95
【实验报告】	97
【实验中的常见问题与解决方案】	98
【实验或拓展思考题】	99
【评分标准】	99
【参考教材与实验拓展】	99
【注意事项】	100
实验五: 敏捷项目收尾 (复盘沉淀与团队收尾)	101
【实验目标】	101
【实验内容】	101

任务 1: 完成项目复盘与收尾	101
任务 2: 完成项目演示与答辩	102
任务 3: 项目归档与团队解散	103
【实验要求】	103
【实验环境】	104
【实验步骤】	105
阶段 1: 项目复盘与收尾	105
阶段 2: 项目演示与答辩	108
阶段 3: 项目归档与团队解散	111
【实验报告】	113
【实验中的常见问题与解决方案】	114
【实验或拓展思考题】	114
【评分标准】	115
【参考教材与实验拓展】	115
【注意事项】	115
附录 1 实验报告模版	117

引言

本实验指导书为《软件系统开发综合能力训练课》的配套实践材料，旨在配合课程理论教学，帮助学生通过完成一个完整的软件项目（以制造执行系统 MES 为例的全流程实践操作），深入理解敏捷软件开发的核心概念、原理与方法，系统掌握软件项目从启动、迭代开发到收尾的完整流程。同时，聚焦团队协作、需求分析、技术选型、质量保障、项目复盘等关键环节，着力培养学生的工程实践能力、团队协作能力、问题解决能力与文档规范意识，为学生后续参与企业级软件项目开发奠定坚实的综合能力基础。

本实验指导书共包含 5 个实验项目，严格遵循敏捷项目开发的时间线依次排列：实验一“敏捷项目启动（团队与需求基线搭建）”完成团队组建与需求梳理，搭建项目开发基础；实验二“迭代 1（从规划到回顾的首次价值交付）”实践首次迭代全流程，交付最小可行产品（MVP）；实验三“迭代 2（基于回顾的增量交付与流程优化）”强化架构支撑与质量内建；实验四“迭代 3（成熟实践下的高效价值交付）”实现高价值功能、性能与体验的全方位达标；实验五“敏捷项目收尾（复盘沉淀与团队收尾）”完成项目复盘、答辩与归档的全闭环。

课程总学时为 48 学时，学生需自主组建 7-9 人团队并选定课题，参照本指导书的实验要求、步骤与质量标准开展项目开发。实验所需的过程性材料（文档模板、交付物示例等）已整理至附件，可直接参考使用。建议学生结合课程进度，按实验顺序逐步推进，每完成一个实验后及时总结经验，同时严格遵循指导书的步骤要求完成各环节任务，确保实践过程规范、高效。

为什么要选择 MES 系统作为案例

一、MES 在工业软件中的核心地位

制造执行系统 (Manufacturing Execution System, MES) 是连接企业顶层战略与车间生产执行的“数字化神经中枢”，在现代制造业数字化转型中扮演着不可替代的核心角色。

1.1 工业软件的三层架构

现代工业软件体系通常分为三层架构,每一层承担不同的职责:

层级	系统类型	核心职责	典型产品
决策层	ERP(企业资源计划)	统筹人财物资源,制定宏观生产计划	SAP S/4HANA、用友 U9
执行层	MES(制造执行系统)	监控车间现场,实时调度生产执行	西门子 Opcenter、鼎捷 MES
控制层	SCADA/PLC(监控与控制)	设备自动控制,数据实时采集	西门子 TIA、罗克韦尔 ControlLogix

MES 位于腰部核心位置:向上承接 ERP 的生产计划,向下指挥车间设备和人员执行,是计划与执行之间的关键桥梁。没有 MES,企业的宏观计划无法真正落地;没有 MES,车间的实时状态无法反馈到管理层。

1.2 MES 的战略价值

根据工信部《制造业数字化转型发展报告(2025 年)》数据显示,超过 67%的规上制造企业已启动核心业务系统数字化改造,其中 MES 已成为连接设备层与经营层的“腰部核心”,其渗透率和应用深度直接决定了企业的柔性响应能力与精益管理水平。

MES 的核心价值体现在以下方面:

- 生产过程透明化:实时采集工时、产量、质量、设备状态等数据,打破车间“黑箱”,管理者可随时掌握生产全貌
- 生产效率提升:通过精准排产、动态调度,某新能源电池厂采用 MES 后,工单拆分时间从 8 小时缩短至 1 小时,在制品库存降低 22%
- 质量追溯能力:全流程记录生产批次、人员、设备、工艺参数,某电子元件厂采用后,质量异常响应时间从 2 小时缩短至 15 分钟,不良率下降 35%
- 设备综合效率(OEE)提升:结合预测性维护,某重工企业通过 MES 识别瓶颈工序并优化后,整体生产效率提升 15%

1.3 MES 的市场地位

2026 年 MES 市场呈现“国产崛起、技术分化”的格局:

- 国产化率突破 58%:鼎捷数智、用友网络、中控技术等本土厂商在自主可控、行业适配、服务响应等方面展现出强劲竞争力
- 技术差异化:AI 大模型、云原生、数字孪生成为核心竞争力,用户体验正式成为选型关键指标
- 行业聚焦:鼎捷数智在离散制造 MES 市场份额稳居行业第一;中控技术在石化行业 MES 市

场份额达 14%, 连续五年稳居第一

二、MES 与其他工业软件的关联关系

理解 MES 的定位, 必须将其放在“研产供销”全价值链中来看。MES 并非孤立存在, 而是与 PLM、ERP 等系统形成紧密的协同网络。

2.1 MES 与 PLM 的关系

PLM(Product Lifecycle Management, 产品生命周期管理)是产品的“全生命周期管家”, 覆盖从创意、设计、量产到退市的完整流程。

核心关系：设计与执行的桥梁

PLM(设计侧)	MES(执行侧)
管理 EBOM(工程 BOM)——设计视角的物料清单	接收 MBOM(制造 BOM)——生产装配顺序
输出工艺文件、作业指导书(BOP)	按工艺文件执行生产,记录实际参数
设计变更(ECN)管理	现场立即同步设计变更,避免用错图纸

2.2 MES 与 ERP 的关系

ERP 是企业“大管家”, 统筹人财物资源;MES 是车间“小班长”, 监控生产细节。二者的协同是“计划与执行”的闭环。

核心关系：计划与执行的互动

ERP(计划层)	MES(执行层)
根据销售订单制定主生产计划(MPS)	接收 ERP 下发的生产计划,拆解为具体工单
执行 MRP 物料需求计划	实时反馈物料消耗、在制品库存
进行成本核算、财务结算	上报工时、产量、不良品数据,为成本核算提供依据

2.3 三者协同的价值

当 PLM、ERP、MES 真正打通时, 企业将获得以下核心价值:

价值维度	描述	典型指标提升
业务流程自动化	消除人工录入、转接,数据自动流转	数据录入效率提升 70%,错误率降低 95%
手返工消除	设计变更实时同步到生产,避免用错图纸	手返工率降低 85%,废料成本降低 40%
数据驱动决策	实时数据支撑,基于事实而非经验决策	决策响应速度提升 60%,计划准确率提升 30%
供应链敏捷响应	销售-生产-采购端到端透明	牛鞭效应降低 50%,库存周转天数减少 25%

三、MES 系统在工业软件领域的重要性总结

选择 MES 作为本课程的贯穿案例,具有以下教育意义:

- 1. 技术复杂度适中: MES 既涉及前端交互(工单管理、报表展示),又涵盖后端业务逻辑(排产调度、库存管理),还与外部系统(ERP、设备接口)集成,是锻炼全栈开发能力的理想载体。
- 2. 业务场景真实: MES 系统需求来自真实制造业场景,业务规则清晰(如先投料后报工、不良品隔离),学生能理解技术如何解决实际业务问题。
- 3. 工程实践全面: MES 系统需要考虑高并发(多工位同时报工)、数据一致性(库存扣减、工单状态同步)、可靠性(生产中断不能丢数据),是检验工程实践能力的试金石。
- 4. 职业发展导向: 2026 年 MES 市场规模持续扩大,国产化替代加速,掌握 MES 开发能力将为学生在智能制造、工业互联网领域的职业发展提供有力支撑。

技术选型参考清单

在软件开发项目中,技术选型是决定项目成败的关键决策之一。本节提供各技术层面的主流技术选项及其适用场景,帮助团队根据项目需求做出合理选择。

一、前端技术选型

1.1 主流框架对比

技术栈	核心定位	学习曲线	生态成熟度	适用场景
Vue 3	渐进式框架,开箱即用	低(模板语法接近 HTML)	官方主导,统一完善	中小型项目、快速开发、后台管理系统
React	视图层库,灵活可扩展	高(需掌握 JSX、Hooks)	社区庞大,生态碎片化	大型复杂应用、跨端开发 (React Native)
Angular	全功能框架,强规范	高(概念多,TypeScript 为主)	官方完善,适合企业级	大型企业应用、金融系统

选型建议:

- 初创团队/中小项目:Vue 3(学习成本低,开发效率高)
- 大型复杂应用/国际化团队:React + Next.js(生态最丰富,SSR 能力强)
- 金融/银行等强规范企业:Angular(架构稳定,适合长期维护)

1.2 前端技术栈组合

类型	Vue 生态	React 生态	备注
框架核心	Vue 3	React 18+	均支持 TypeScript
状态管理	Pinia	Zustand / Redux Toolkit	Pinia 更简洁,Zustand 更轻量
路由	Vue Router	React Router v6	功能基本对等
UI 组件库	Element Plus / Naive UI	Ant Design / Material-UI	Element Plus 适合后台系统
构建工具	Vite	Vite / Turbopack	Vite 均为官方推荐
服务端渲染	Nuxt.js	Next.js	Next.js 生态更强大

二、后端技术选型**2.1 主流语言与框架对比**

技术栈	核心框架	适用场景
Java	Spring Boot	大型企业应用、金融系统
Node.js	Express / NestJS	高并发 API、实时应用
Python	Django / FastAPI	快速原型、数据密集应用
Go	Gin / Echo	云原生应用、微服务
.NET	ASP.NET Core	企业级应用、微软技术栈团队

选型建议:

- 团队熟悉 Java/需要企业级支持:Spring Boot(本案例采用)
- 需要高并发 I/O 密集型应用:Node.js + NestJS
- 快速开发/数据科学结合:Django / FastAPI

- 追求极致性能/微服务:Go + Gin

2.2 后端技术栈组合

层级	Java 技术栈	Node.js 技术栈	Go 技术栈
Web 框架	Spring Boot 3.x	NestJS / Express	Gin / Echo
ORM	MyBatis-Plus / JPA	TypeORM / Prisma	GORM
认证授权	Spring Security / JWT	Passport.js	go-jwt
数据验证	Hibernate Validator	class-validator	go-playground/validator
日志	Logback / SLF4J	Winston / Pino	Zap
监控	Micrometer + Prometheus	Prometheus Client	Prometheus Client

三、中间件技术选型

3.1 消息队列对比

MQ 类型	代表产品	吞吐量	延迟	适用场景	特点
Kafka	Apache Kafka	百万级 TPS	中等 (10-100ms)	日志收集、流计算、大数据管道	高吞吐、持久化、分区顺序
RabbitMQ	RabbitMQ	万级 TPS	低(毫秒级)	任务队列、复杂路由、企业应用	灵活路由、AMQP 协议、高可用
RocketMQ	Apache RocketMQ	十万级 TPS	低(毫秒级)	电商、金融、事务消息	事务消息、顺序消息、阿里背书
Redis Stream	Redis	十万级 TPS	极低(<1ms)	轻量级消息、实时通知	轻量级、与 Redis 集成

3.2 缓存技术对比

缓存类型	产品	特点	适用场景
内存缓存	Redis	数据结构丰富	会话存储、热点数

		(String/List/Hash/Set/ZSet)、持久化 (RDB/AOF)、集群模式	据缓存、排行榜
内存缓存	Memcached	纯 KV 存储、多线程、简单易用	简单缓存、多核 CPU 高并发
本地缓存	Caffeine / Guava	JVM 内存缓存、高性能、淘汰策略丰富	本地热点数据、减轻 Redis 压力
分布式缓存	Hazelcast / Apache Ignite	内存网格、支持 SQL 查询、分布式计算	分布式会话、实时计算

选型建议:

- 通用缓存:Redis 6.0+(功能最全面)
- 纯性能缓存:Memcached(简单场景)
- 本地缓存:Caffeine(Java 项目,高性能)
- 复杂分布式场景:Hazelcast

四、数据库技术选型

4.1 关系型数据库对比

数据库	特点	优势	劣势	适用场景
MySQL	Web 应用主流	生态庞大、云服务成熟、运维简单	复杂查询性能弱、JSON 支持有限	电商、CMS、传统后台系统
PostgreSQL	开源全能选手	支持 JSONB/GIS/数组、高级 SQL 特性、扩展性强	配置复杂、学习曲线陡峭	复杂数据模型、地理信息系统、金融风控
Oracle	企业级王者	高可用、强 ACID、企业功能完善	授权费用高、运维复杂	核心银行系统、电信计费

选型建议:

- 大多数 Web 应用:MySQL 8.0+(稳定、生态好)
- 需要复杂查询/JSON 支持/GIS:PostgreSQL 14+
- 金融/银行核心系统:Oracle 23c

4.2 NoSQL 数据库对比

数据库类型	代表产品	数据模型	适用场景
文档型	MongoDB	BSON 文档	灵活 Schema、内容管理、

			商品目录
键值型	Redis	Key-Value	缓存、会话、排行榜
列式存储	Cassandra	宽表	时序数据、日志、大规模写入
搜索引擎	Elasticsearch	文档+倒排索引	全文搜索、日志分析、监控告警
图数据库	Neo4j	节点+边	社交关系、推荐系统、风控图谱

4.3 多数据库组合方案

典型场景	数据库组合	说明
电商平台	MySQL(交易) + Elasticsearch(搜索) + Redis(缓存)	交易强一致性,搜索全文检索,缓存提升性能
IoT 平台	MongoDB(设备数据) + InfluxDB(时序指标) + Kafka(消息队列)	设备状态灵活存储,指标高效写入,消息解耦
内容平台	MongoDB(文章) + PostgreSQL(用户) + Redis(热点)	内容灵活结构,用户关系复杂,热点数据缓存
社交平台	Neo4j(关系) + MySQL(用户) + Redis(Feed 流)	社交关系图查询,用户数据存储,Feed 流缓存

五、DevOps 与运维工具选型

5.1 容器与编排

工具	定位	适用场景
Docker	容器引擎	应用容器化、环境一致性
Kubernetes	容器编排	大规模微服务部署、自动扩缩容
Docker Compose	本地开发编排	开发环境一键启动多服务

5.2 CI/CD 工具

工具	特点	适用场景
Jenkins	插件生态庞大、高度可定制	复杂构建流程、企业级 CI/CD
GitHub Actions	与 GitHub 集成、免费友好	开源项目、GitHub 托管项目

GitLab CI	与 GitLab 集成、YAML 配置简单	GitLab 托管项目
CodeArts 流水线	华为云集成、国内访问快	华为云项目、本课程推荐

5.3 监控与日志

工具类型	代表产品	说明
监控指标	Prometheus + Grafana	指标采集与可视化告警
链路追踪	Jaeger / SkyWalking	分布式追踪、性能分析
日志聚合	ELK Stack(Elasticsearch + Logstash + Kibana)	日志收集、搜索、分析

六、技术选型决策框架

6.1 评估维度

在做出技术选型决策时,应综合考虑以下维度:

维度	评估要点	权重建议
业务需求匹配度	技术能否高效满足核心业务需求?	30%
团队能力	团队对技术的熟悉程度、学习成本	25%
生态成熟度	社区活跃度、第三方库丰富度、文档质量	20%
性能与可扩展性	吞吐量、延迟、水平扩展能力	15%
运维成本	部署复杂度、监控工具、故障排查难度	10%

6.2 选型决策流程

- 明确需求
 - 业务需求:功能、性能、并发量
 - 约束条件:团队技能、预算、部署环境
- 候选技术筛选
 - 根据需求列出 3-5 个候选方案
 - 排除明显不匹配的选项
- 深度评估
 - 原型验证:搭建 Demo 验证关键技术点
 - 风险评估:识别潜在技术风险

4. 决策与记录

团队讨论并投票

形成技术选型说明文档, 记录决策理由

5. 持续审视

定期评估技术选型是否仍然适配

必要时进行技术迁移

6.3 本案例技术栈说明

本指导书示例项目采用以下技术栈, 供学生参考学习:

层级	技术选型	选型理由
前端	Vue 3 + Element Plus + Vite	学习曲线平缓, 适合教学演示
后端	Spring Boot 3.x + MyBatis-Plus	企业级主流技术, 生态成熟
数据库	MySQL 8.0	应用最广泛的开源数据库
缓存	Redis 6.0	主流缓存方案, 数据结构丰富
消息队列	RabbitMQ	路由灵活, 适合入门学习
版本控制	Git + 华为 CodeArts Repo	企业级代码托管平台
项目管理	华为 CodeArts(需求+看板+流水线)	国产平台, 集成度高

重要提示: 本技术栈为教学示例, 学生在实际项目中应根据自身需求、团队能力和项目特点进行技术选型, 不要求与本示例完全一致。

实验一：敏捷项目启动（团队与需求基线搭建）

【实验目标】

（一）知识目标

掌握敏捷开发的核心原理与价值观，理解自组织团队的运作机制。

熟练掌握需求分析的方法，包括用户故事编写、用户故事地图绘制及产品待办列表梳理。

了解项目开发计划编制的关键要素，包括技术选型、风险识别与管控的核心逻辑。

熟悉远程代码仓库的搭建与团队协作流程，掌握主流开发环境的配置方法。

（二）能力目标

能够独立完成自组织团队的组建与分工，制定规范的团队章程。

能够针对所选课题开展全面的需求分析，输出完整的《需求规格说明书》、用户故事及用户故事地图。

能够结合项目需求与团队实际，编制合理的技术选型文档、风险管控文档及开发计划。

能够独立搭建本地开发环境与远程代码仓库，实现团队协作开发的基础配置。

（三）素质目标

培养团队协作意识与沟通能力，树立规范的工程协作理念。

养成严谨的文档编写习惯，遵循企业级项目管理标准与规范。

建立风险防控思维，提升项目前期规划的科学性与前瞻性。

强化学术诚信意识，规范引用 AI 或他人代码的行为。

【实验内容】

任务 1：完成自组织团队组建

任务描述

自主组建 7-9 人项目团队，明确团队成员分工，制定团队协作规则，形成正式的团队相关文档，为项目开发奠定组织基础。

任务知识

自组织团队运作模式、团队分工原则、团队章程核心要素。

任务成果

自组织团队名单、团队分工表、《团队章程》

思路简述

基于敏捷开发自组织团队理念，通过成员互识明确各自优势，合理分配角色（如产品负责人、开发工程师、测试工程师、UI 工程师等），制定包含沟通机制、会议规则、代码规范等内容的团队章程。

💡 **教学提示：**本环节核心是建立高效的协作基础。团队章程中的角色、沟通规则、代码规范是确保后续开发顺畅的关键，需全体成员共同确认并遵守。案例中的角色（如产品负责人、敏捷教练）是敏捷标准角色，团队可根据规模灵活调整，但需确保核心职责有人承担。

任务 2：完成所选课题需求分析

任务描述

明确你团队项目相关的需求目标与范围，通过用户故事梳理核心需求，绘制用户故事地图，形成产品待办列表，完成需求评审与规划。

任务知识

需求分析方法、用户故事编写规范、用户故事地图绘制技巧、需求评审流程。

任务成果

《需求规格说明书》、用户故事、用户故事地图、产品待办列表（基线）、产品待办列表（动态更新）、需求评审记录、思维导图规划、甘特规划。

思路简述

基于你团队项目的业务场景，明确需求边界；（可参考示例项目中如销售管理、采购管理等模块的划分方式）；采用“作为[角色]，我希望[功能]，以便[价值]”的格式编写用户故事；按优先级排序用户故事并绘制地图；区分静态基线与动态跟踪列表记录需求；通过内部评审验证需求可行性；在项目管理平台（本案例选择 CodeArts 作为管理工具）完成需求规划。

💡 **教学提示：**本环节重点在于掌握从业务场景到用户故事地图的产品梳理方法。案

例中“销售管理、采购管理”等模块划分仅为示例，你团队应聚焦于自身项目的核心业务流程，进行需求拆解和优先级排序，确保用户故事地图真实反映产品全貌。

任务 3：项目开发计划编制

任务描述

结合需求分析结果，完成项目技术选型、风险识别与管控，制定详细的开发计划与甘特规划，为项目执行提供指导依据。

任务知识

技术选型原则、风险识别与应对方法、项目进度规划技巧、甘特图制作规范。

任务成果

技术选型说明文档、风险识别与管控文档、开发计划表。

思路简述

基于你团队项目的技术需求(如数据存储、并发处理、界面交互等)，结合团队技术栈选择合适的开发语言、框架、数据库及 AI 辅助编程工具；识别技术风险、进度风险、需求变更风险等，制定规避或应对策略；按迭代周期拆开发任务，明确各阶段时间节点与交付物；在 CodeArts 平台制定粗略甘特规划，确定迭代周期与关键里程碑。

 **教学提示：**技术选型的依据是项目需求、团队技术储备和社区生态，而非案例所用技术。本环节重在实践选型决策的思考过程（如为何选 Spring Boot 而非其他框架），并学会识别和管控技术、进度、需求变更等早期风险。

任务 4：搭建开发环境

任务描述

完成远程代码仓库的创建与团队成员授权，搭建统一的本地开发环境，约定分支管理规范，保障团队协作开发的顺畅进行。

任务知识

远程代码仓库使用流程、本地开发环境配置方法、分支管理策略。

任务成果

远程代码仓库地址、本地开发环境、分支管理规范。

思路简述

基于华为 CodeArts 平台（其他同类型平台亦可）创建远程代码仓库，初始化项目框架并邀请团队成员加入；统一团队开发工具（如 IDEA）、JDK 版本、数据库（MySQL）、缓存（Redis）等环境配置；制定主分支、开发分支、功能分支的创建与合并规范。

💡 教学提示：“分支管理策略”是本环节的工程核心。案例中 feature/用户故事 ID-功能名称的命名规范、主分支/开发分支/功能分支的用法，是行业通用实践，请严格遵守。此举旨在从第一天起杜绝代码合并混乱问题。

【实验要求】

交付物

任务一：自组织团队名单、团队分工表、《团队章程》（必要）。

任务二：《需求规格说明书》（必要）、用户故事地图、产品待办列表（必要，基线为 DOCX 格式，动态更新为 Excel 格式）、需求评审记录、CodeArts 思维导图规划（截图）、CodeArts 甘特规划（截图）。

任务三：技术选型说明文档、风险识别与管控文档（含风险类型、风险描述、应对措施、责任人）、开发计划表（必要，含迭代周期、阶段任务、时间节点、责任人、交付物）。

任务四：远程代码仓库地址（必要，含访问权限说明）、本地开发环境（截图，团队互检）、分支管理规范。

-  01.制造执行系统 (MES) _自组织团队名单.xlsx
-  02.制造执行系统 (MES) _团队分工表.xlsx
-  03.制造执行系统 (MES) _敏捷开发团队章程V1.1.0.docx
-  04.制造执行系统 (MES) _用户故事V1.1.0.docx
-  05.制造执行系统 (MES) _用户故事地图V1.1.0.pdf
-  06-1.制造执行系统 (MES) _产品待办列表V1.1.0 (用于静态需求阶段) .docx
-  06-2.制造执行系统 (MES) _产品待办列表V1.1.0 (用于动态迭代阶段) .xlsx
-  07.制造执行系统 (MES) _1.0.0_需求规格说明书V1.0.0.docx
-  08.制造执行系统 (MES) _需求评审记录V1.1.0.docx
-  09.制造执行系统 (MES) _开发计划表V1.1.0.docx
-  11.制造执行系统 (MES) _技术选型说明文档V1.1.0.docx
-  12.制造执行系统 (MES) _风险识别与管控文档V1.1.0.docx

质量标准

文档结构完整，内容详实，逻辑清晰，无遗漏关键要素。

需求分析方法科学，数据来源可靠，结论符合系统业务实际。

技术选型合理，风险识别全面，开发计划安排可行，进度估算准确，资源配置适当。

格式规范统一，语言表达准确，无错别字和语法错误。

符合企业级项目管理标准和敏捷开发规范，可直接用于后续项目开发指导。

【实验环境】

（一）硬件环境

处理器：Intel Core i5 及以上。

内存：8GB 及以上。

硬盘：剩余空间 50GB 及以上。

网络：稳定的互联网连接（用于访问远程代码仓库、项目管理平台）。

（二）软件环境

操作系统：Windows 10/11 64 位或 macOS 12 及以上。

开发工具：IntelliJ IDEA 2022 及以上、Visual Studio Code（可选）。

编程语言及运行环境：JDK17。

数据库：MySQL 8.0、Navicat 17 社区版。

缓存：Redis 6.0.0 及以上。

项目管理平台：华为 CodeArts（推荐）、GitHub Projects（可选）。

代码仓库：华为 CodeArts Repo、GitHub（可选）、Gitee（可选）。

文档工具：Microsoft Office 2019 及以上、WPS 2022 及以上。

原型图工具：Figma、Axure RP、墨刀（均推荐使用最新版）。

浏览器：火狐浏览器、谷歌浏览器（推荐）。

【实验步骤】

阶段 1：自组织团队组建

本阶段目标是建立高效协作的团队基础。核心产出是得到全体成员认可并愿意共同遵守的《团队章程》。

1. 团队成员互识：

操作指南：

1. 召集所有成员，组织一次正式的启动会议（30 分钟）。
2. 每位成员进行 2 分钟的自我介绍，必须包含三要素：姓名/昵称、技术栈或兴趣方向（如前端、Java 后端、数据库、UI 设计）、在本次项目中最希望承担的角色或学习的目标。
3. 指定一名记录员，使用在线文档实时记录每位成员的介绍要点。

验收标准：

会议结束后，团队拥有一份包含所有成员核心信息的共享笔记。

2. 确定自组织团队名单：

操作指南：

5. 基于互识环节的共享笔记，使用提供的 01. 自组织团队名单.xlsx 模板。
6. 团队名称：应简洁、有辨识度，与项目主题相关。
7. 拟承担角色：参考敏捷标准角色（产品负责人、敏捷教练、开发工程师、测试工程师等），结合成员意愿和项目需求确定。允许一人承担多个角色，但每个角色职责必须明确。

决策框架（角色职责澄清）：

- 产品负责人：定义需求优先级，确保团队做“正确的事”。是需求的最终决策者。
- 敏捷教练：维护流程高效，扫除协作障碍，引导团队改进。不一定是导师，可以是团队成员兼任。
- 开发团队：负责交付“可工作的软件”，集体对成果负责。包括开发、测试等所有执行人员。
- UI/UX 负责人：保证产品界面与用户体验的一致性。非必要角色，可根据项目需要设置。

验收清单（填写名单后核对）：

- 团队成员姓名、学号、联系方式准确无误。
- 每个成员至少承担一个明确的角色。

- 团队名称已最终确定。

3. 明确团队分工与制定团队章程：

操作指南：

1. 召开团队章程制定会议（45-60 分钟）。由敏捷教练引导，确保每位成员发言。
2. 使用提供的 03. 团队章程 V1.1.0.docx 模板，围绕以下核心议题进行讨论并达成共识：
 - 沟通机制：每日站会的固定时间（如上午 9:15）、所用工具（如腾讯会议）、时长（严格≤15 分钟）。
 - 会议规则：站会每人发言时间盒（如 1.5 分钟），决策机制（“少数服从多数”或“负责人决策”）。
 - 代码规范：约定代码注释、命名的基本规则，明确在首次提交代码前由技术负责人提供一份更详细的规范文档。
 - 工具与文档：统一 Git 分支命名规范（如 feature/US001-login）、文档命名格式（如 04-需求规格说明书 V1.0.docx）。
 - 冲突解决：约定出现分歧时的解决流程。
3. 将讨论结果实时记录到章程文档中。

避坑指南：

- 避免空泛：章程条款必须具体。例如，不应写“及时沟通”，而应写“工作时间内，企业微信消息需在 1 小时内回复”。
- 技巧：章程是“活”的文档。可在章程中约定“每轮迭代回顾会议时复核本章程，必要时可经团队同意后修订”。
- 关键决策：必须明确 Git 仓库的提交规范（如[feat]：完成用户登录功能）和分支管理策略（如功能开发必须从 dev 分支拉取 feature 分支，完成后通过 Pull Request 合并），这是后续协作的基石。

验收清单（章程定稿后核对）：

- 章程涵盖了沟通、会议、代码、文档、冲突解决等核心协作维度。
- 所有条款都是具体、可执行的。
- 全体团队成员阅读并理解章程内容，并在文档末尾签署姓名和日期。

阶段 2：需求分析

本阶段目标是将你团队选定的项目课题，转化为清晰、可执行的产品需求，为后续迭代规划提供可靠依据。核心产出是获得团队内部一致认可的《需求规格说明书》、用户故事地图和产品待办列表。

1. 明确需求目标和范围：

操作指南：

1. 召开需求启动会议（约 60 分钟），全体团队成员参加。会议主持人（建议由产品负责人担任）需提前准备项目背景资料。
2. 围绕以下核心问题展开讨论，并将结论实时记录在共享文档中：
 - **项目要解决的核心问题是什么？**（例如：解决线下手工排产效率低下、数据不透明的问题）
 - **主要用户是谁？**（至少列出 2-3 个核心用户角色，如：生产计划员、车间主任）
 - **这些用户最需要系统帮助他们完成什么任务？**（列出 3-5 个最高优先级的任务）
 - **项目的边界在哪里？**（明确声明本次项目**不做哪些功能**，例如：不涉及与 ERP 系统的深度集成、不开发移动 APP）
3. 根据讨论结果，开始撰写《需求规格说明书》的“项目背景与目标”、“用户角色”、“核心功能概述”和“范围与约束”等章节。

避坑指南：

- **避免范围蔓延：**初期明确“不做”的内容至关重要，可有效防止后期需求无限扩大。
- **技巧：**用户角色不要虚构，应基于真实或合理的业务场景。可以简要描述每个角色的工作职责和痛点。

验收标准：

团队对“我们要做一个什么系统，为谁做，先做哪些核心功能”达成清晰共识，并体现在《需求规格说明书》的初稿中。

2. 编写用户故事：**操作指南：**

1. 针对已明确的核心功能点，采用标准格式编写用户故事。格式为：“作为一名 [用户角色]，我希望 [完成某个任务]，以便于 [实现某种价值/解决某个问题]。”
2. 为每个用户故事定义验收标准。这是判断故事是否完成的依据，应简单、可测试。通常以“给定…当…那么…”的格式描述。
 - 示例（用户故事：车间主任查看生产进度报表）：
 - 验收标准 1：给定车间主任已登录系统，当他进入报表页面时，那么应能看到当日所有产线的计划产量和实际产量。
 - 验收标准 2：给定报表数据，当实际产量低于计划产量时，那么该数据行应以红色高亮显示。
3. 为每个用户故事分配一个初步优先级（高/中/低）和粗略的故事点（S/M/L）。故事点代表相对工作量，团队内部达成一致即可，无需精确计算。

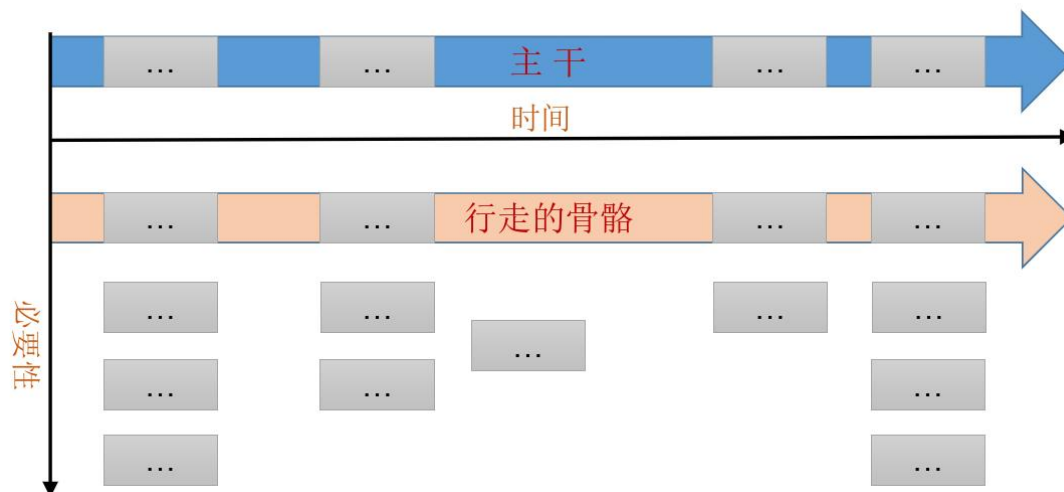
验收清单（用户故事编写完成后核对）：

- 每个故事都符合“角色-任务-价值”的标准格式。
- 每个故事都包含 1-3 条清晰的验收标准。
- 每个故事都有初步的优先级和故事点标识。

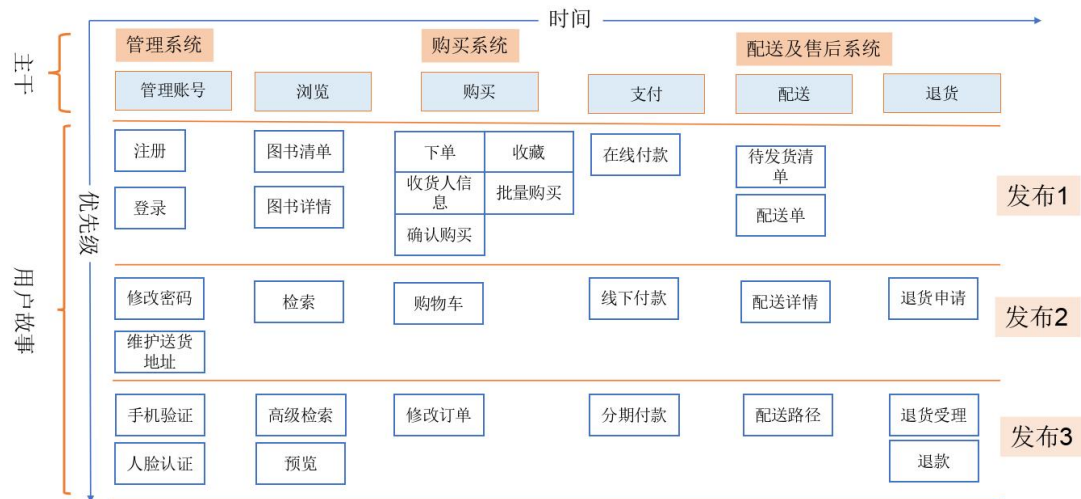
3. 绘制用户故事地图**操作指南：**

1. 使用白板工具（如华为 CodeArts 的白板、WPS 的设计或线下实体白板）进行协作绘制。
2. 第一步：梳理用户活动流（横向骨架）。在顶部横向排列用户完成一个目标所经历的核心步骤。这构成了系统的骨干流程。

示例（生产管理核心流程）：制定生产计划→ 下发工单→ 接收并执行工单→ 报工与数据采集→ 查看生产报表。



3. 第二步：填充用户故事（纵向分解）。在每个用户活动下方，纵向排列步骤 2 中编写的相关用户故事。位置越高，优先级越高。最上面一行故事应能组成一个可运行的“最小可行产品（MVP）”。



4. 第三步：划定发布边界。在地图上画线，标识出第1次迭代（MVP）、第2次迭代等计划要完成的故事范围。

避坑指南：

- 避免过度复杂：第一个版本的地图应聚焦核心流程，无需一次性列出所有可能的故事。地图是活的，后续可以补充。
- 技巧：地图绘制的关键是团队协作和讨论的过程，而非最终生成的图片。确保每位成员都理解地图所讲述的“用户旅程”。

验收标准：

生成一张用户故事地图，能够清晰展示系统的核心业务流程、故事的优先级关系以及迭代发布计划。

4. 生成与管理产品待办列表

操作指南：

1. 创建基线待办列表：将用户故事地图中最上面两行的故事（即高优先级故事）整理到文档中，形成初始的产品待办列表基线。此文档版本相对静态，作为需求范围的基准。
2. 创建动态待办列表：在华为 CodeArts 或在线表格中创建动态跟踪列表。包含以下核心字段：故事 ID、故事标题、用户角色、优先级、故事点、迭代分配（待分配/迭代 1/迭代 2...）、状态（待办/设计中/开发中/待测试/完成）。
3. 需求评审：组织内部评审会议。团队成员模拟不同用户角色，逐条评审用户故事和验收标准，检查其是否独立、可讨论、可估算、可测试。根据评审意见修改故事。

验收清单（待办列表整理完成后核对）：

- 基线待办列表已固化，并作为需求范围的约定。
- 动态待办列表已在协作平台创建完毕，所有高优先级故事已录入。

- 动态列表的字段设置合理，能够满足后续迭代跟踪的需求。
- 已完成需求评审，故事描述清晰，无歧义。

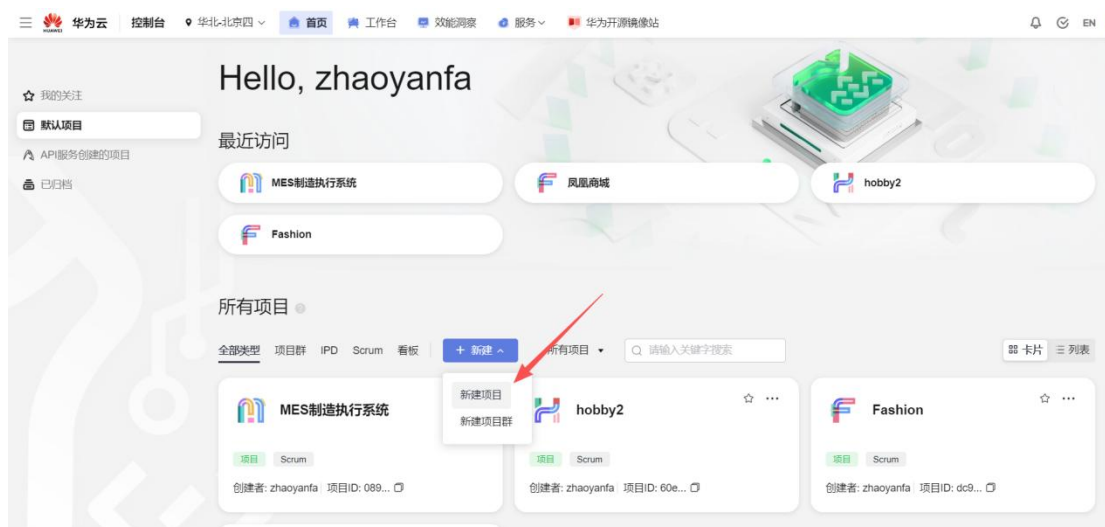
5. 在 CodeArts 中进行需求规划

操作指南：

登录华为 CodeArts 平台(地址:<https://devcloud.cn-north-4.huaweicloud.com/home>)

首次登录需要自行注册账号，创建项目，将需求放入 CodeArts 平台进行管理，步骤如下：

1. 登录后点击“新建”按钮，选择新建项目。



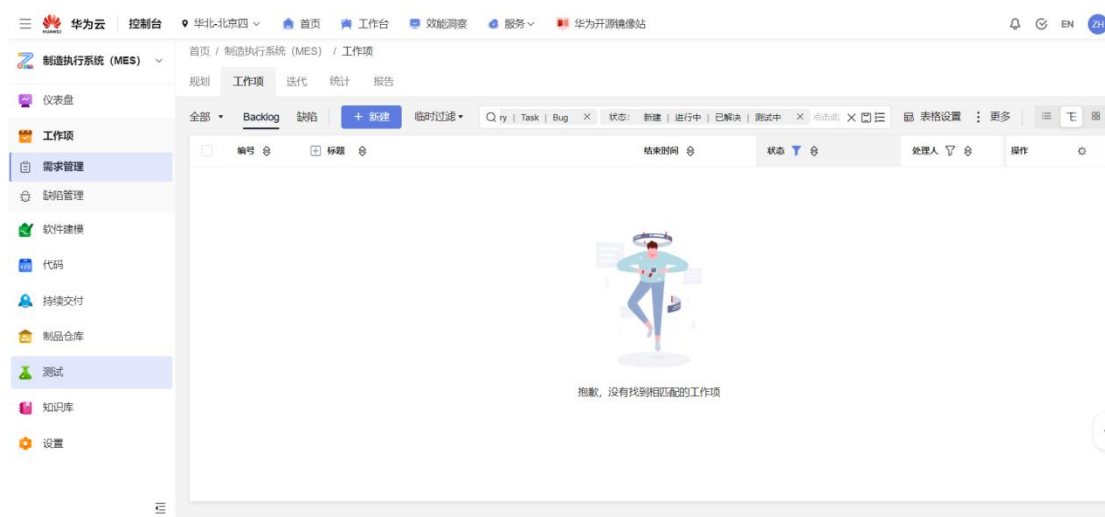
2. CodeArts 中内置了常见开发场景的模版，这里我们选择 Scrum（最长选用）模版。



3. 填写项目名称、项目代号（不重复即可）、项目描述，点击“确认”按钮。完成项目创建。



4. 创建后默认进入需求管理界面，在这里开始做需求规划。



5. 首先点击需求管理下的全部，当前的工作项为空，工作项分为 Epic、Feature、Story、Task、Bug，其中 Task 和 Bug 属于 Story 下两个最小单元，分别用来记录任务和测试发现的缺陷。

用户故事依据其颗粒度（即范围与详细程度），通常分为四个层级，从宏观到微观依次为：

史诗（Epic）

定义：代表一个宏大的战略目标或一个完整的产品方向，通常需要多个团队或较长时间（如数个季度）才能完成。

示例：“构建一套全新的智能仓储管理系统”。

核心：回答“我们需要做一个什么系统或实现什么宏大目标？”。

特性（Feature）

定义：是史诗下的大型功能模块或能力集合，代表一组能为用户提供显著价值的完整功能。

示例：“实现仓储管理的入库管理模块”。

核心：回答了“为了达成史诗，我们需要提供哪些核心能力或模块？”。

故事 (Story)

定义：是特性的具体组成部分，一个独立的、可交付用户价值的功能点。它是敏捷开发中计划与交付的基本单元。

示例：“作为仓库管理员，我需要通过订单号查询入库单的详细信息，以便快速处理异常”。

核心：描述“一个具体的、可测试的功能需求”。

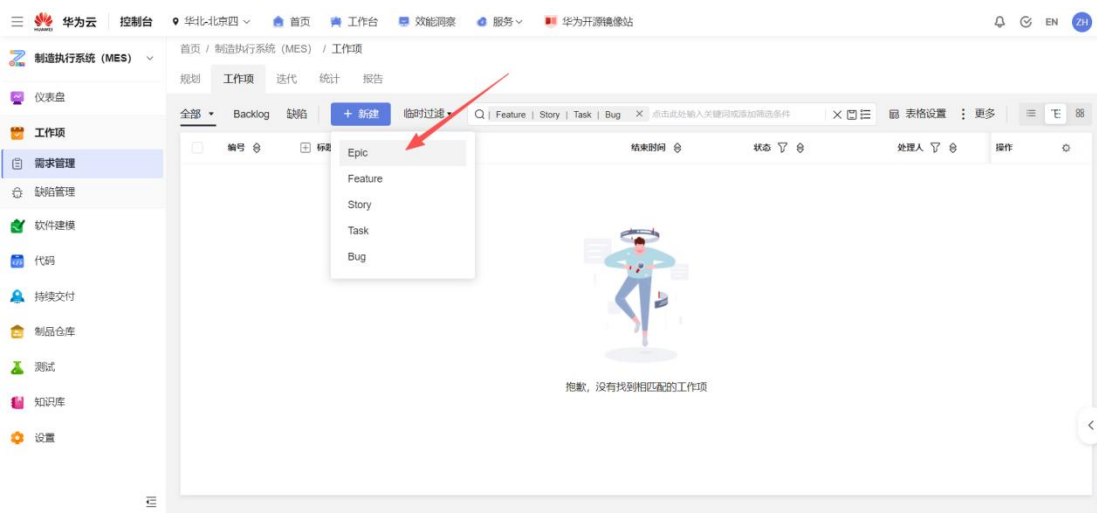
任务 (Task)

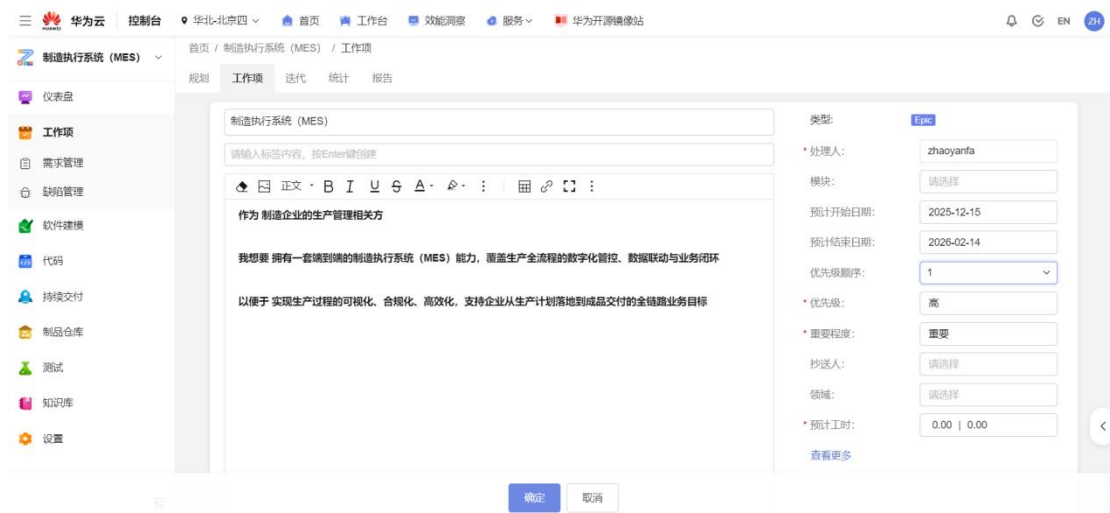
定义：是完成一个故事所需拆解出的具体、可执行的技术或工作步骤。这是开发人员日常工作的最小单元。

示例：“开发订单查询前端页面”、“实现订单查询后端 API 接口”。

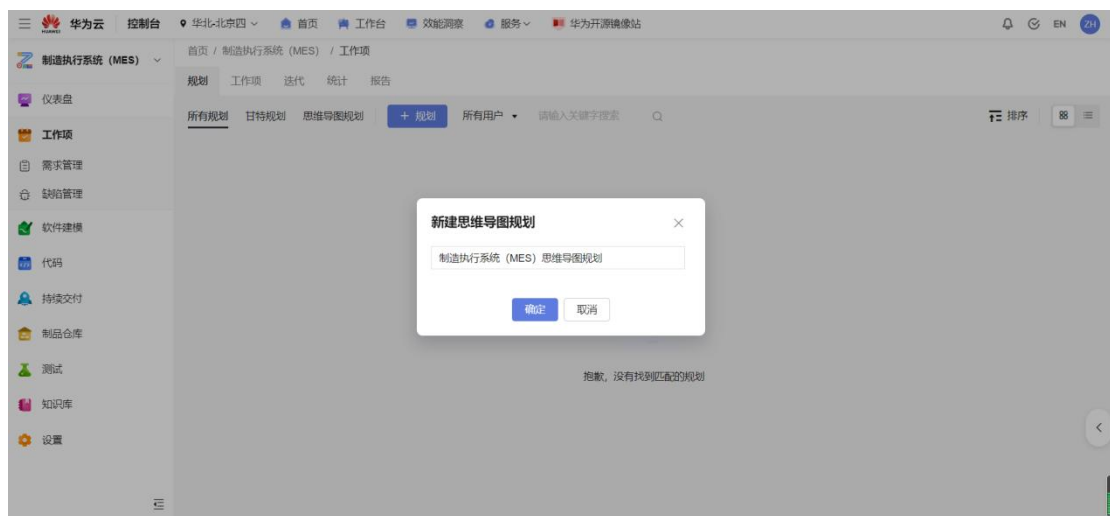
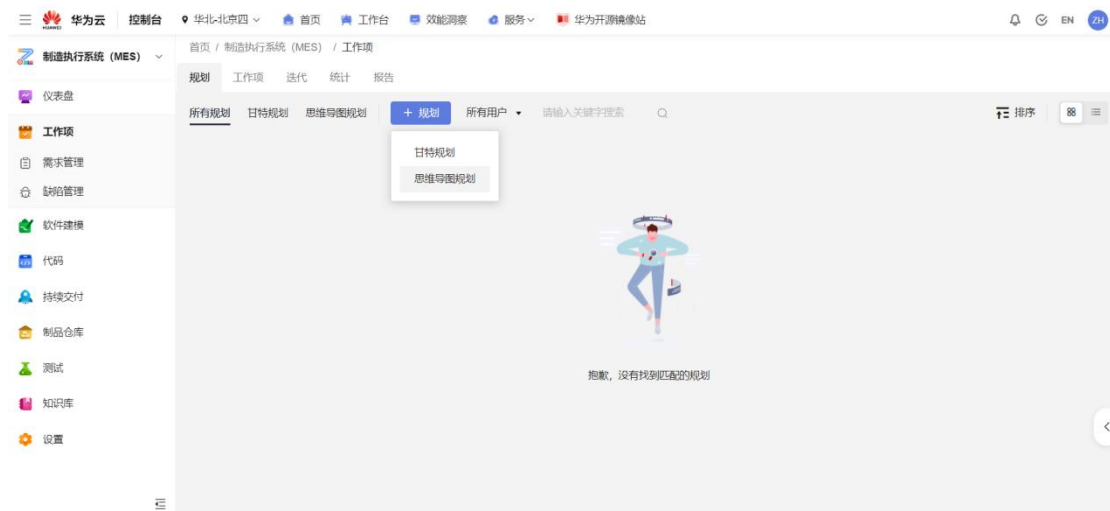
核心：定义“如何实现故事的具体工作项”。

首先需要在工作项中创建一个史诗级用户故事 (Epic)。

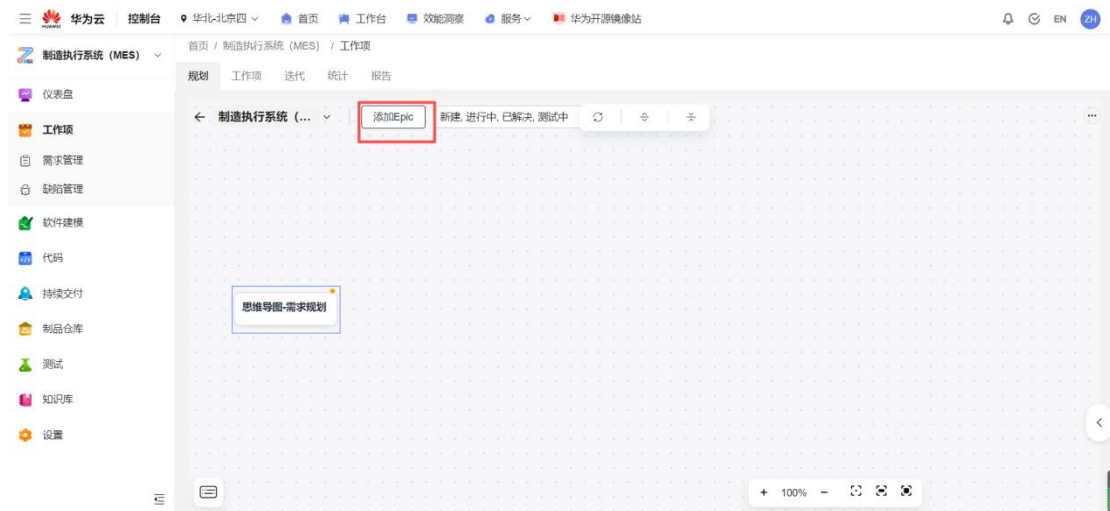




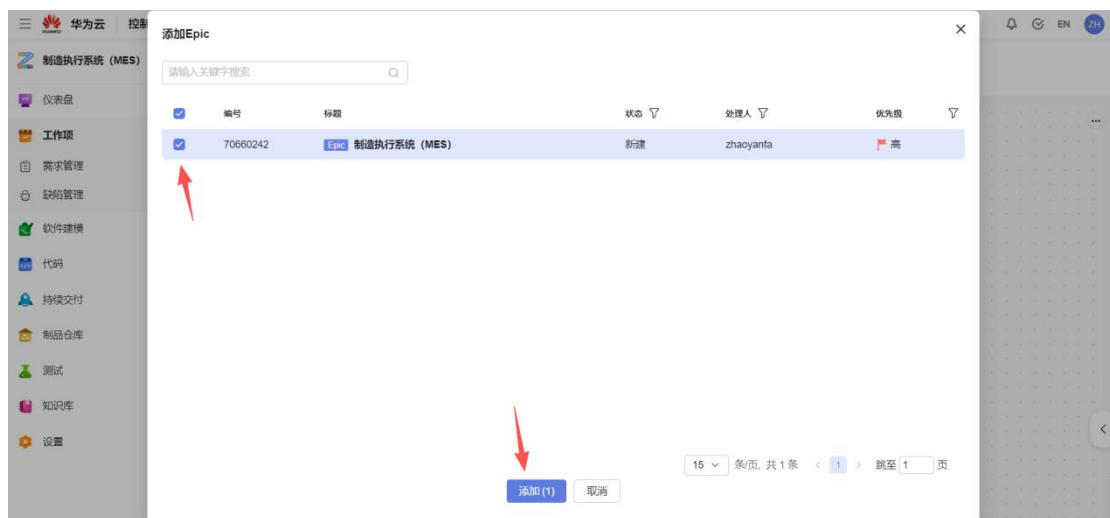
6. 创建好史诗级用户故事后，切换至需求管理下的规划 Tab 页，选择“规划”按钮下的“思维导图规划”，填写思维导图规划名称，点击确认保存。



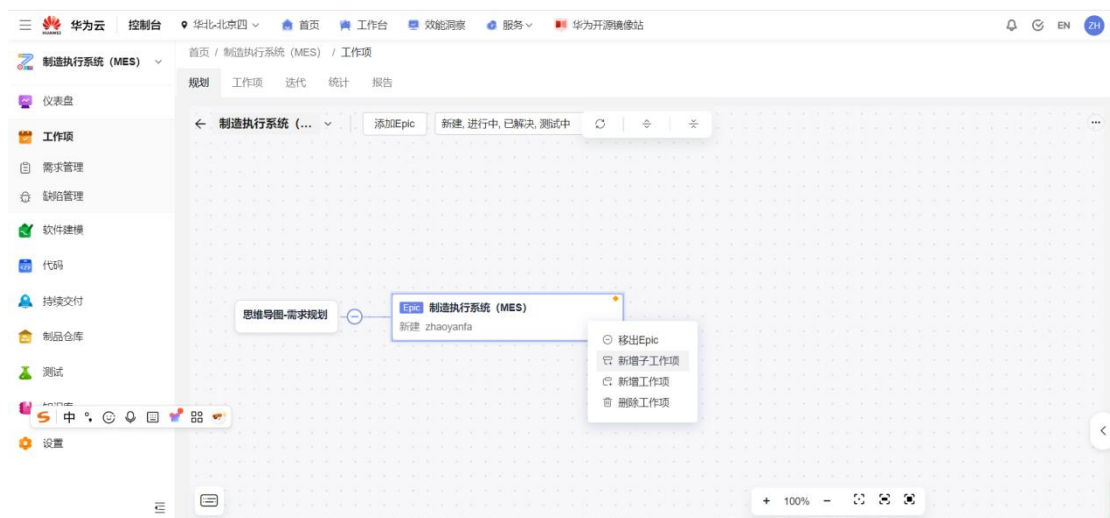
7. 点击刚刚创建好的思维导图规划，进去规划页面，首先选中思维导图-需求规划，点击上边的“添加 Epic”按钮



8. 选中刚刚创建好的史诗级用户故事，点击“添加”



9. 选中刚刚添加的 Epic 用户故事，右键选择“新增子工作项”，为其添加 Feature 用户故事

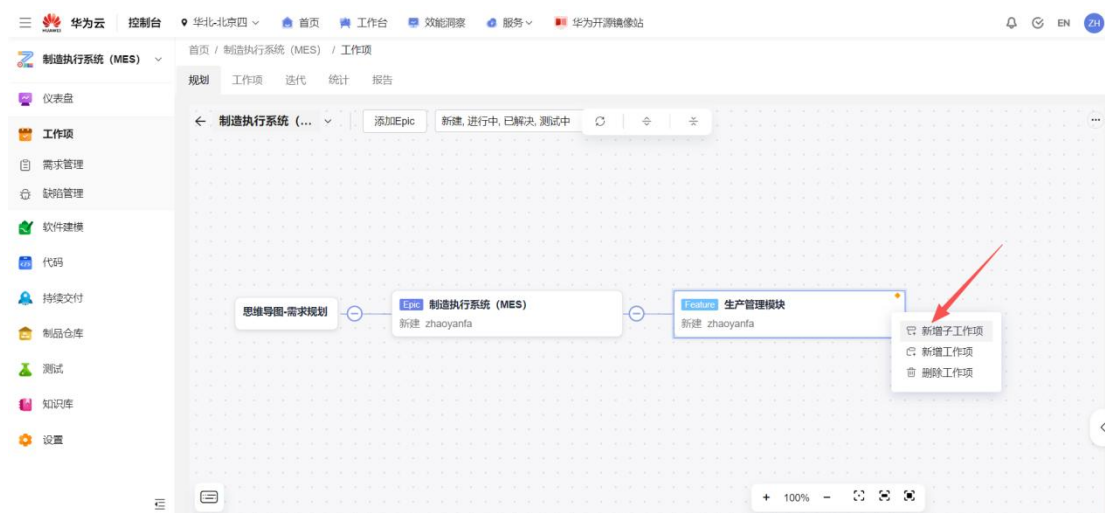


10. 填入 Feature 级用户故事，设定一个粗略的计划日期，设置优先级，点“保存”完成添

加



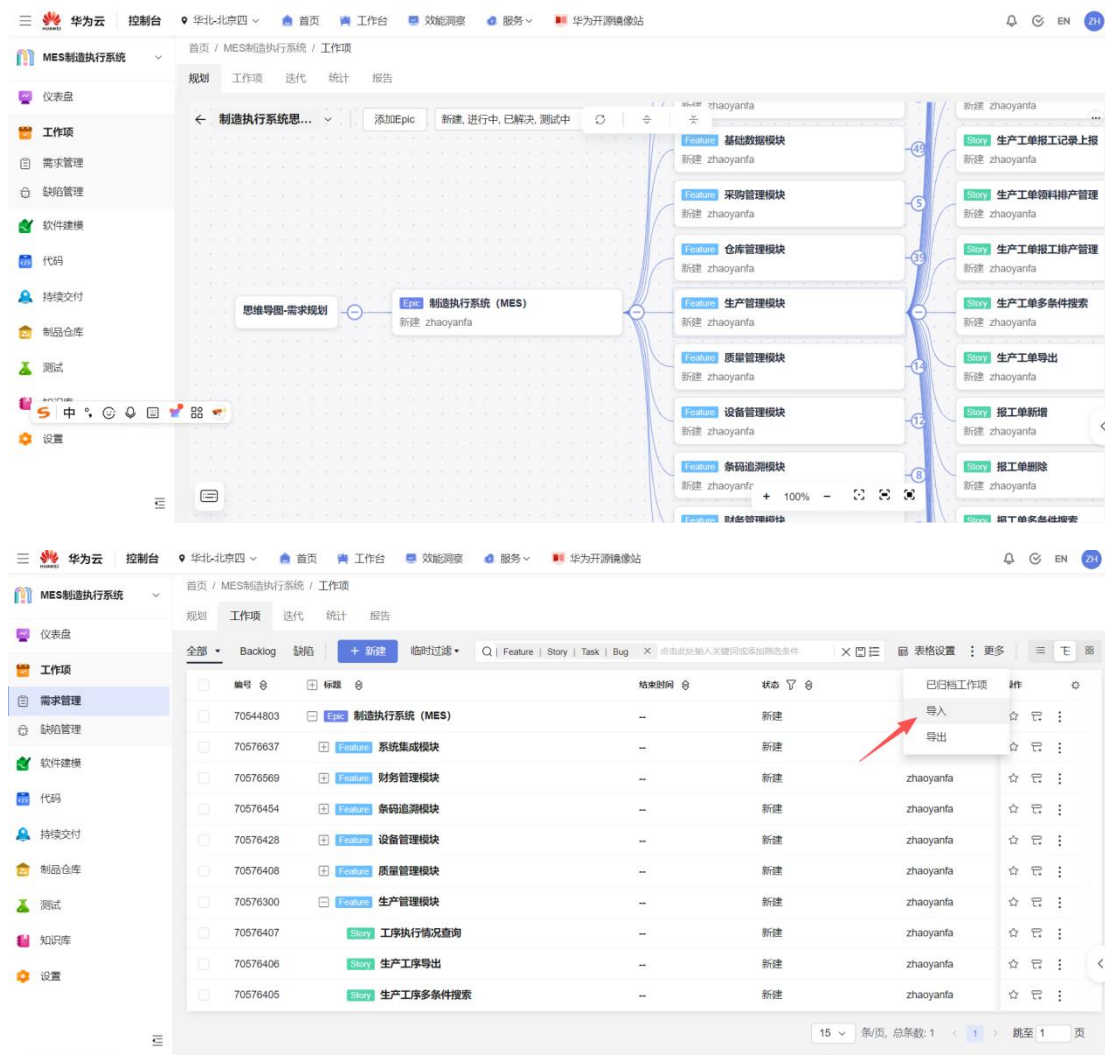
11. 选中刚刚新增的 Feature 级用户故事，右键“新增子工作项”，为其添加一个 Story 级用户故事



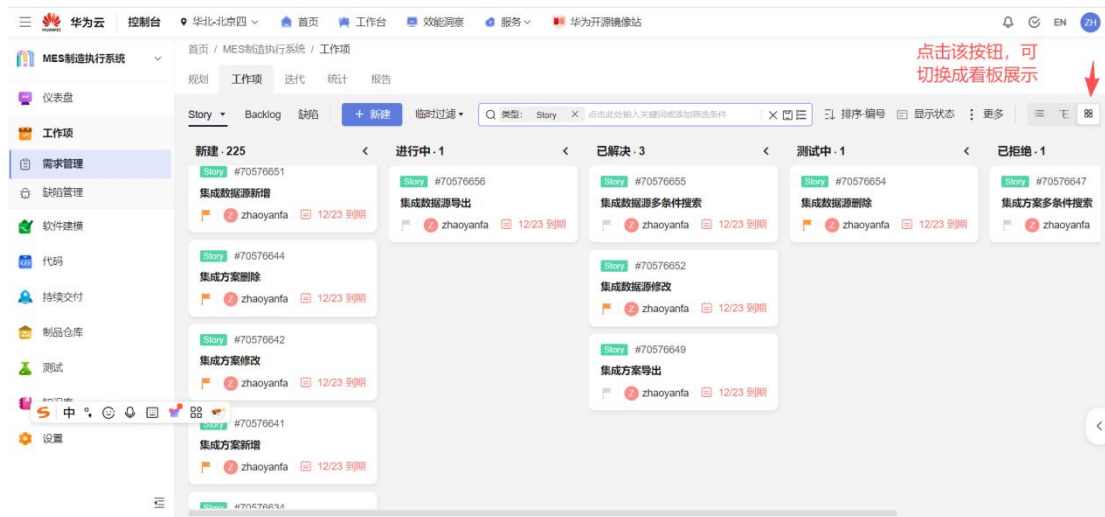
12. 填入 Story 级用户故事，设定日期、优先级后点“保存”完成添加



13. 按照以上步骤，根据《需求规格说明书》以及用户故事列表，将所有用户故事录入平台中，也可以根据平台的模版，直接在工作项中导入，完成批量新增。



14. 将用户故事全部录入 CodeArts 平台中，就形成了思维导图规划，后期可根据该规划生成甘特图规划，也可以直接生成看板视图（如下图）。这个阶段将需求规划到故事（Story）级别即可，任务（Task）在之后的迭代中逐步拆分。



验收标准:

所有高优先级的用户故事已作为“Story”工作项录入 CodeArts，并完成了初步的迭代规划。

阶段 3：项目开发计划编制

本阶段目标是为项目制定一份可行的技术实施蓝图和进度路线图。核心产出是指导后续开发的技术选型说明、风险管控策略和详细的开发计划。

1. 技术选型

操作指南:

1. 召开技术研讨会（约 60 分钟）。技术负责人需提前梳理项目的主要技术需求点（如：数据持久化、用户界面、并发处理、外部接口等）。
2. 针对每个技术需求点，团队共同讨论并决策。讨论时应围绕以下核心问题：
 - **项目需求：**哪种技术栈最能高效、稳定地满足我们的核心业务需求？
 - **团队能力：**团队成员对候选技术的熟悉程度如何？学习成本是否可接受？
 - **社区生态：**该技术是否主流？社区是否活跃？遇到问题时能否快速找到解决方案？
 - **部署与集成：**该技术是否易于部署和与其它组件（如数据库、缓存）集成？是否符合学校实验环境的要求？
3. 将讨论决策结果整理到《技术选型说明文档》中。文档需说明每个选型的决策理由、具体版本号和主要应用场景。
 - **示例（后端框架选型）：**选用 Spring Boot。理由：团队熟悉 Java，Spring Boot 生态丰富、开发效率高，能快速构建 RESTful API。版本：3.2.0。应用场景：所有后端业务逻辑开发。

避坑指南:

- **避免盲目追新：**优先选择团队熟悉且社区稳定的技术，避免因追求最新技术而引入不可控的风险。
- **技巧：**对于不确定的技术点，可以花少量时间（如 2 小时）进行“技术原型验证”，编写一个简单的 Demo 来测试其可行性。

验收标准：

团队对项目的主要技术栈（前端、后端、数据库、缓存等）达成一致，并形成了有理有据的《技术选型说明文档》。

2. 风险识别与管控

操作指南：

1. 召开风险识别头脑风暴会议（约 30 分钟）。采用“假设-后果”法，引导团队成员思考：“如果…发生，会导致什么后果？”
2. 从以下几个维度系统性地识别潜在风险：
 - **技术风险：**如团队对新技术不熟悉、第三方 API 不稳定、性能不达标。
 - **管理风险：**如任务估算不准确、成员时间冲突、沟通不畅。
 - **需求风险：**如需求理解偏差、客户中途提出重大变更。
 - **外部风险：**如实验环境故障、关键成员退出。
3. 将识别出的所有风险记录到《风险识别与管控表》中。对每个风险进行评级（高/中/低），并制定具体的应对措施、明确责任人和监控方式。
 - **示例（技术风险）：**风险描述：团队成员不熟悉 Vue 3 的组合式 API。评级：中。应对措施：安排一次内部培训，并指定一位有经验的成员作为答疑人。责任人：前端负责人。监控方式：在首次开发任务中检查代码质量。

验收标准：

形成一份包含至少 5-8 个关键风险点的《风险识别与管控文档》，且每个风险都有对应的、可执行的应对策略。

3. 制定开发计划与甘特图

操作指南：

1. 任务分解：基于阶段 2 产出的用户故事（Story），将其进一步拆分为更小的、可分配和可跟踪的开发任务（Task）。每个任务应能在 1-3 天内完成。

示例（Story：用户登录）：可拆分为：设计登录界面 → 开发登录后端 API → 实现前端登录逻辑 → 前后端联调 → 编写登录功能测试用例。

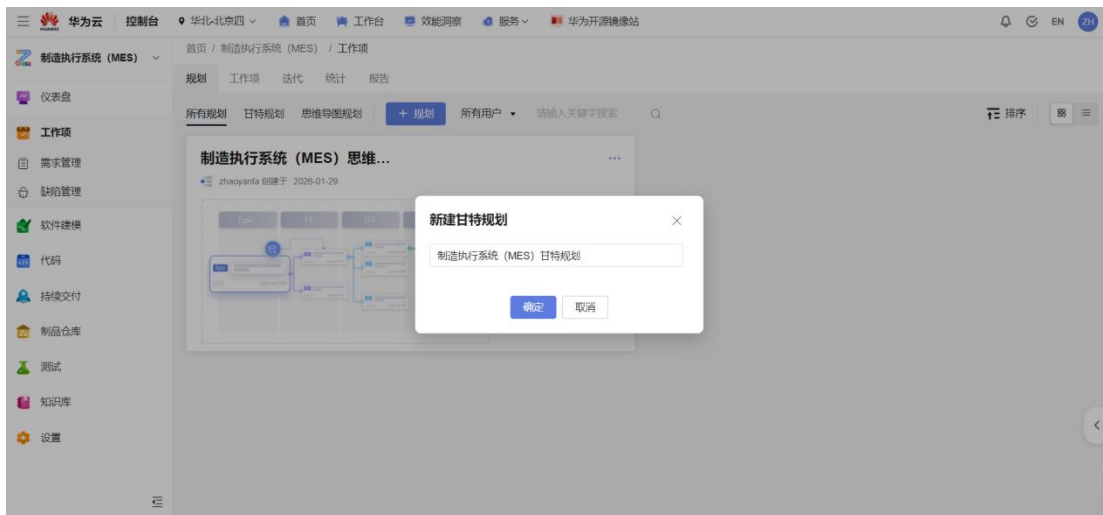
2. 估算与分配：为每个开发任务估算工作量（以“人天”为单位），并初步指定负责人。

3. 制定迭代计划：将任务按优先级和依赖关系分配到三个迭代周期中，明确每个迭代的起止日期、核心目标和主要交付物。
4. 生成甘特图：在 CodeArts 平台关联需求思维导图，根据《开发计划表》设置各迭代周期、关键任务的开始时间和结束时间，形成粗略甘特图，时间颗粒度到“天”，后续迭代中逐步细化。生成甘特图步骤如下：

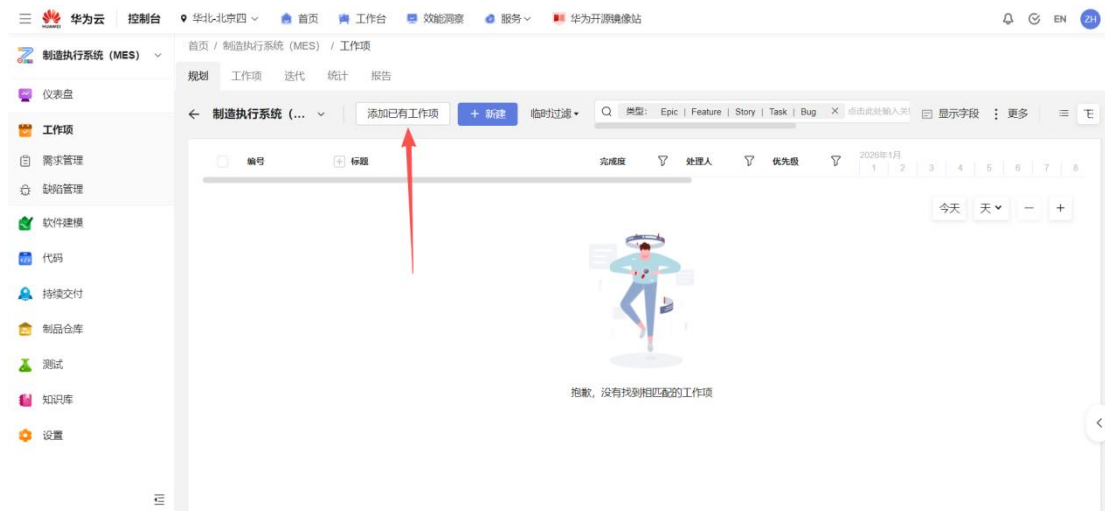
(1) 选择需求管理下的规划，点击规划按钮中的甘特规划



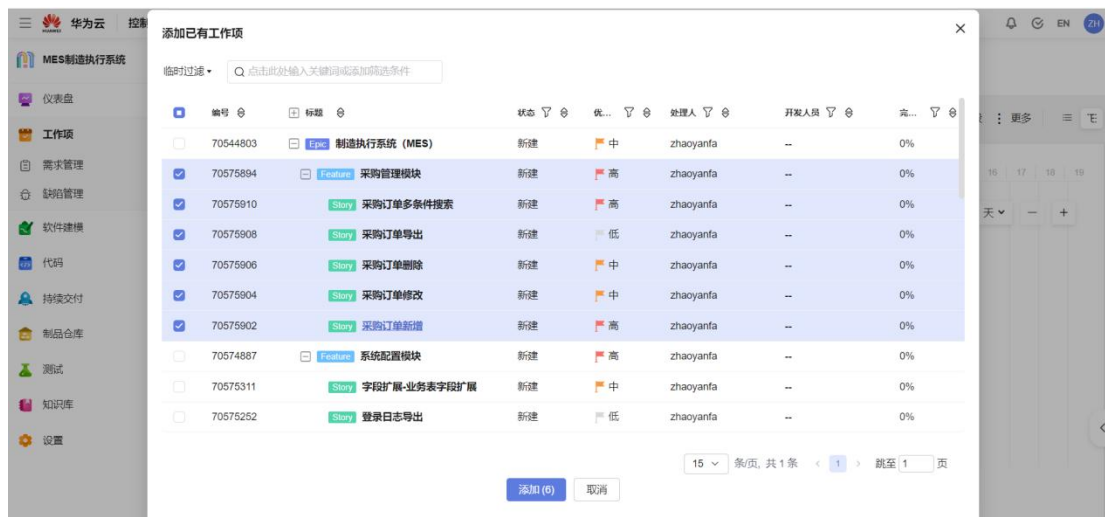
(2) 填入甘特规划名称（自定义）



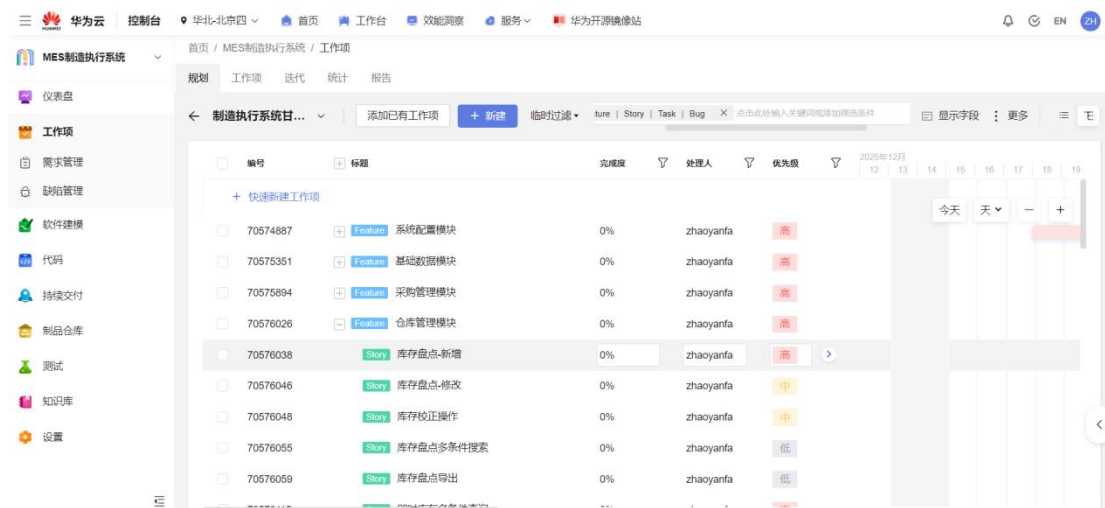
(3) 在刚创建的规划中选择“添加已有工作项”按钮



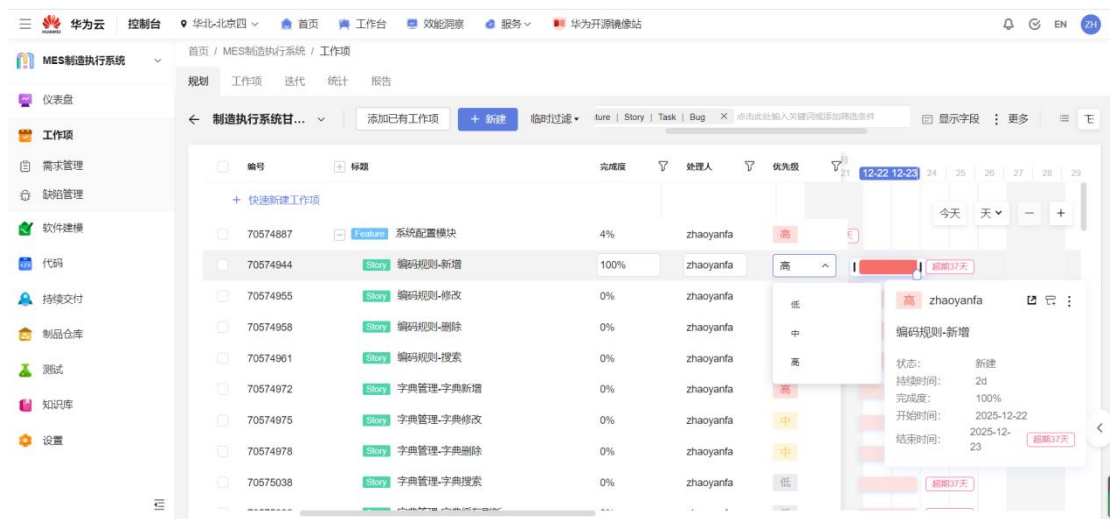
(4) 这里可以看到思维导图中的所有工作项，选择 Story 级用户故事，点击“添加”按钮，即可将想要管理的用户故事放入甘特规划中，形成甘特图。



(5) 形成的甘特图如下，可更新用户故事状态、处理人、优先级等，便于团队共享任务进度。



(6) 可以直接在甘特图中查看延期的任务。



避坑指南:

- **避免计划过满:** 计划中需预留约 20% 的缓冲时间, 用于应对突发问题、会议和不可预见的任务。
- **技巧:** 甘特图在规划阶段是“计划图”, 在迭代过程中是“跟踪图”。应定期(如每日站会后)更新任务的实际进度, 使其真实反映项目状态。

验收标准:

形成一份详细的《开发计划表》, 并成功在 CodeArts 上生成了直观的项目甘特图, 团队对后续的工作安排和时间节点有清晰的共识。

阶段 4: 搭建开发环境

本阶段目标是为团队建立统一、可用的协作开发环境, 这是所有后续开发工作的基础。核心产出是配置好的远程代码仓库和全体成员可正常运行的本地开发环境。

1. 初始化远程代码仓库

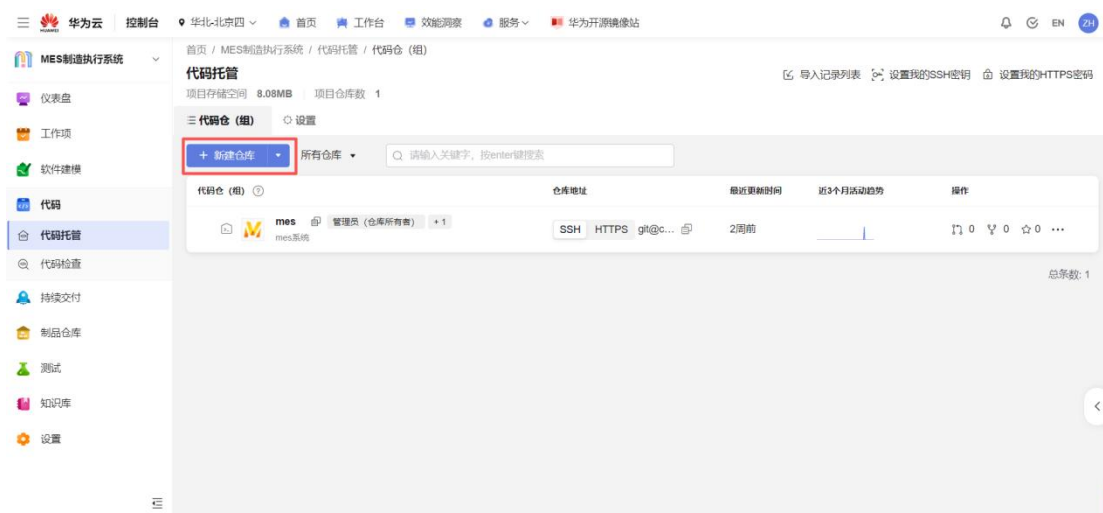
操作指南:

1. 创建仓库: 团队负责人登录华为 CodeArts 平台, 进入在需求规划阶段创建的项目。
2. 导航到“代码托管”模块, 点击“新建仓库”。
3. 选择“普通创建”方式, 填写仓库基本信息:
 - **仓库名称:** 通常与项目名称一致, 如 manufacturing-execution-system。
 - **仓库描述:** 简要说明仓库用途。
 - **初始化仓库:** 务必勾选“允许生产 README 文件”选项, 这将自动创建 README.md 文件。
4. 点击确认完成仓库创建。系统将生成一个远程仓库地址(通常是 HTTPS 或 SSH 格式)。

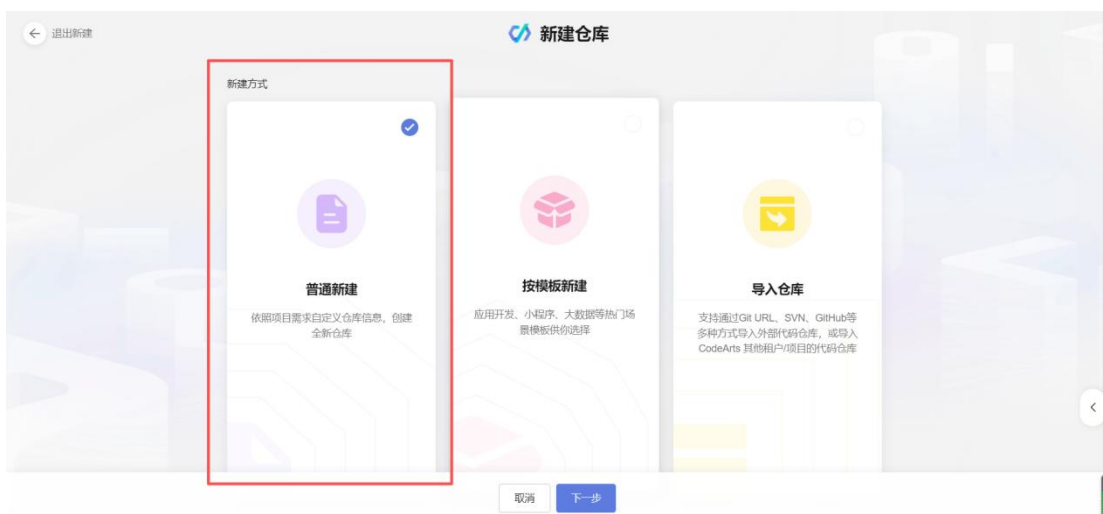
5. 配置权限：进入仓库的“成员管理”设置，邀请所有团队成员，并为他们分配“开发者”权限，确保他们拥有推送代码的权限。

CodeArts 创建远程代码仓库步骤如下：

- (1) 点击代码-代码托管-新建仓库按钮，开始为团队建立一个空的仓库



- (2) 选择普通创建，其他创建方式请自行查阅华为官方文档



- (3) 填入仓库基本信息，初始仓库需勾选“允许生产 README 文件”，然后点击确认。

退出新建

普通新建

选择新建方式

填写基本信息

基本信息

代码组路径

代码仓库名称

描述

选择gitignore

初始化设置

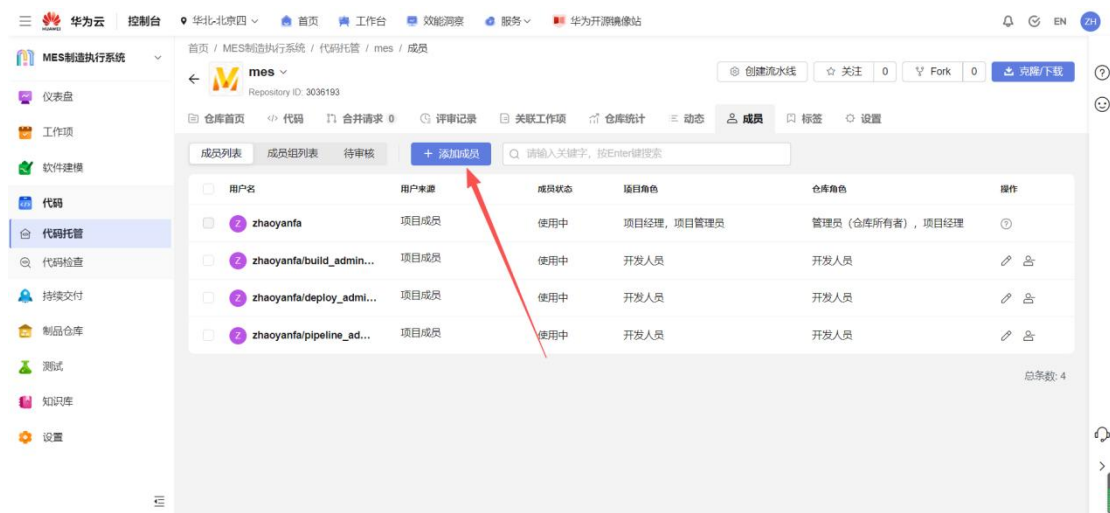
可见范围

上一步 取消 确定

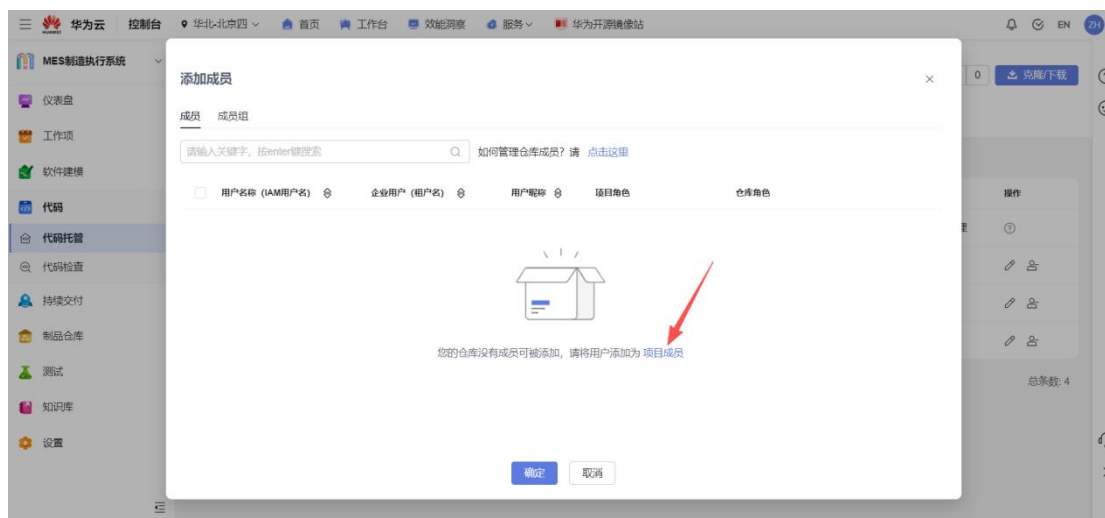
(4) 点击刚刚创建好的仓库“查看更多”，选择成员管理。



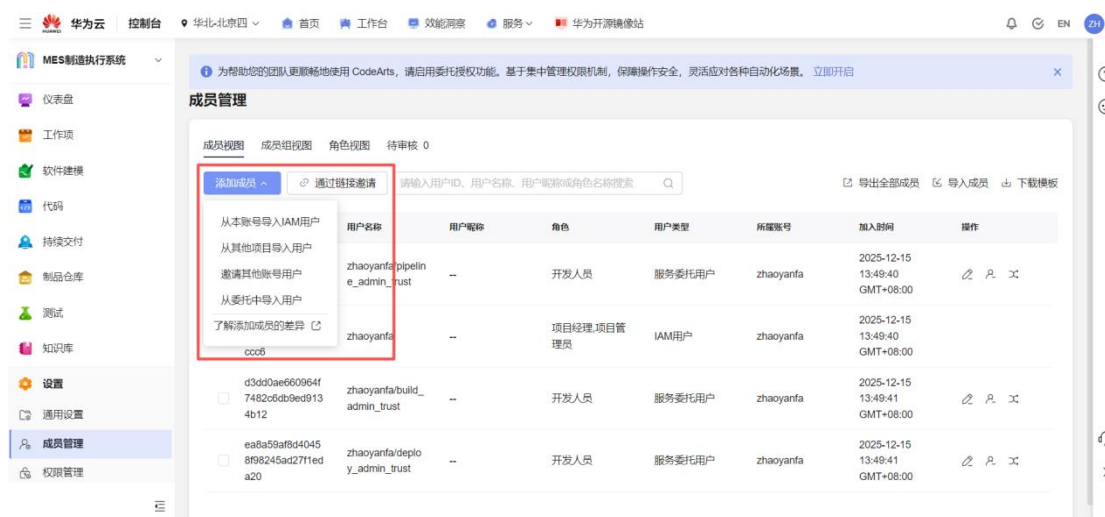
(5) 点击“添加成员”，将其他团队成员加入到仓库中



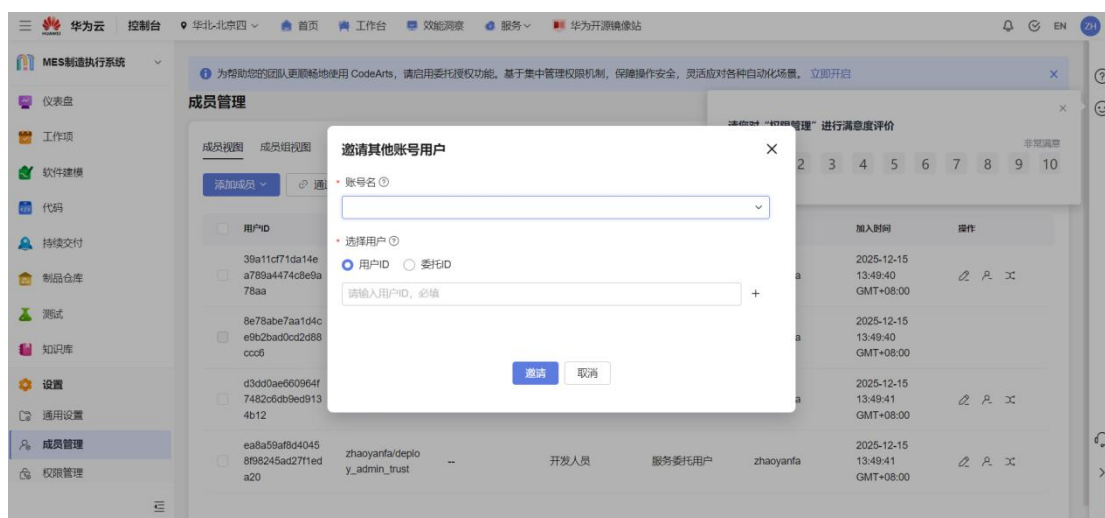
(6) 第一次新建的账号没有项目成员，需要点击下方的按钮，先将其他成员加入到该项目中后再进行添加



(7) 点击添加成员-邀请其他账号用户进行添加，也可以通过链接邀请。

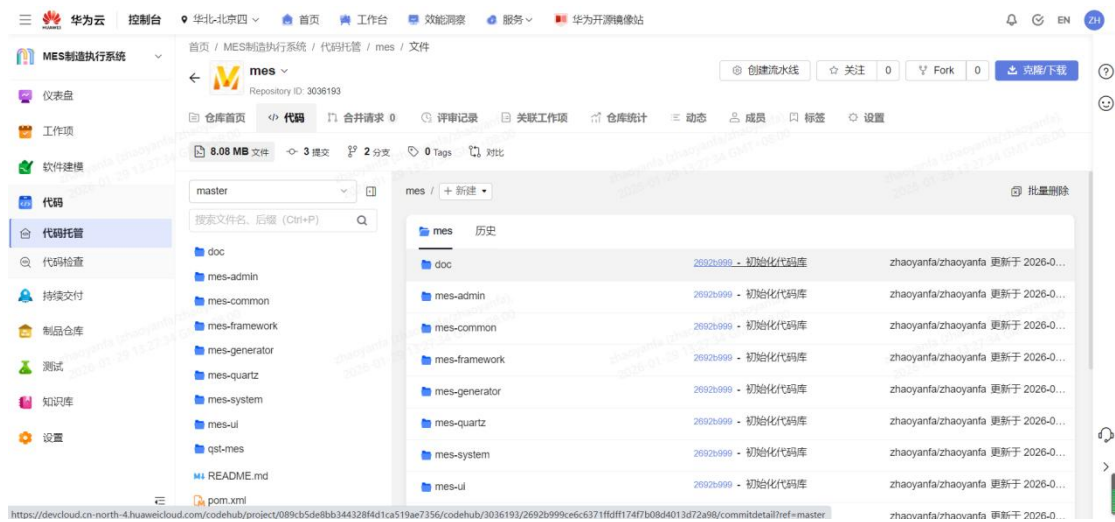


(8) 填写其他成员的账号名和用户 ID，邀请进入该项目组，被邀请的成员需要同意确认。

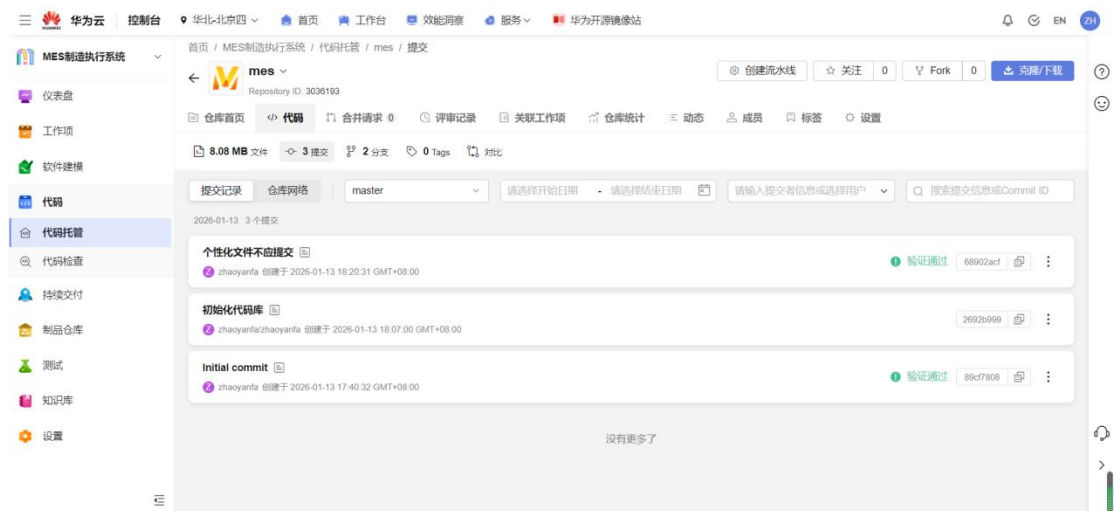


(9) 完成邀请后，所有成员都可以看到仓库，点击进入刚才的，可以查看仓库的代码文

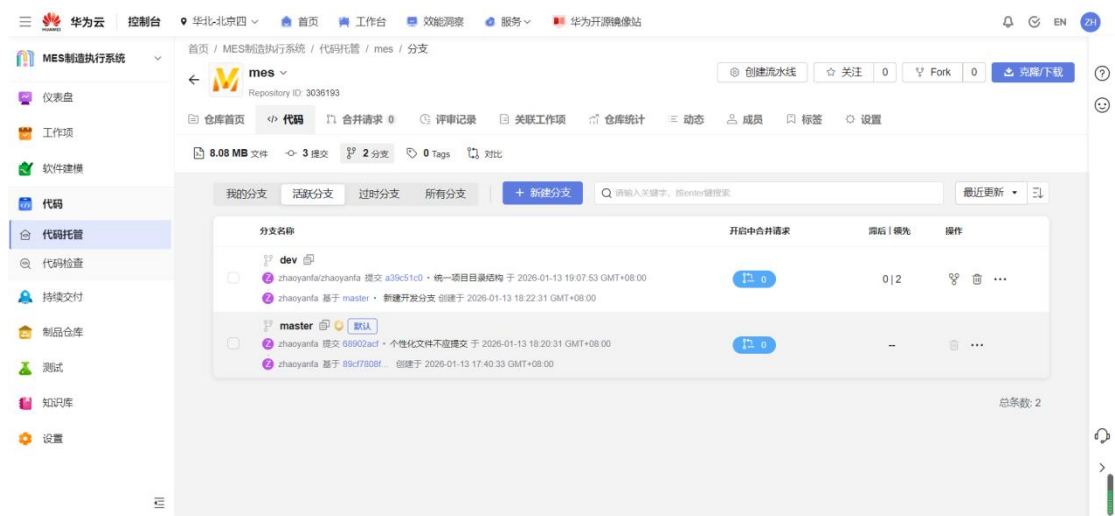
件、提交记录、分支等信息。



(10) 查看提交记录



(11) 管理代码分支



避坑指南：

- **权限管理：**谨慎分配“管理员”权限，通常只有项目负责人需要。避免权限过高导致误操作。
- **仓库类型：**在课程实验中，通常选择“私有”仓库，以保护代码。

验收标准：

远程代码仓库创建成功，所有团队成员均已成功加入该仓库并拥有适当的权限。仓库根目录包含一个初始的 README.md 文件。

2. 制定分支管理规范**操作指南：**

1. 团队共同讨论并确定一套分支管理策略，并写入《团队章程》或单独的规范文档。这是团队协作的“法律”。
2. 推荐策略：采用基于功能分支的工作流（GitFlow 的简化版）。
 - **main/master 分支：**保护分支，仅用于存放可发布的稳定代码。禁止直接推送，只能通过合并请求（Merge Request/Pull Request）更新。
 - **dev 分支：**集成分支，所有开发完成的功能最终合并到此分支。代表下一个版本的最新开发成果。
 - **feature/xxx 分支：**功能分支，从 dev 分支创建，用于开发新功能。命名格式为 feature/用户故事 ID-功能简述，例如 feature/US001-user-login。
 - **hotfix/xxx 分支：**热修复分支，从 main 分支创建，用于线上版本的紧急修复。
3. 提交信息规范：约定提交代码时的注释格式。例如：[任务类型] 简要描述。任务类型可以是 feat（新功能）、fix（修复 bug）、docs（文档更新）、style（代码格式调整）等。

验收标准：

团队拥有一份成文的、所有成员理解并同意的《分支管理与提交规范》文档。

3. 搭建统一的本地开发环境**操作指南：**

1. 克隆仓库：每位团队成员在本地选择一个合适的工作目录，使用 Git 命令或图形化工具（如 GitKraken, SourceTree）将远程仓库克隆到本地。
 - 命令示例：git clone <你的远程仓库地址>
2. 安装与配置基础软件：团队需统一以下关键软件的版本，确保环境一致性。
 - **JDK：**安装 JDK 17，并配置 JAVA_HOME 环境变量。在终端输入 java -version 和 javac

-version 验证安装是否成功。

- 开发工具 (IDE): 推荐使用 IntelliJ IDEA。团队可共享相同的代码风格模板和基础插件配置。
- 数据库: 安装 MySQL 8.0, 并创建一个与项目同名的数据库。使用 Navicat 或 IDEA 自带的数据库工具测试连接。
- 缓存: 若技术选型包含 Redis, 则安装 Redis 6.0 或以上版本, 并启动服务。

3. 导入并配置项目:

- 在 IDEA 中打开克隆下来的项目。
- 配置 Maven, 加载 pom.xml 文件以下载所有依赖项。
- 检查项目 JDK 版本, 确保设置为 JDK 17。
- 根据项目中的 README.md 或相关文档, 配置数据库连接参数 (如 URL、用户名、密码), 通常是在 application.properties 或 application.yml 文件中。

避坑指南:

- 环境一致性: JDK 和 MySQL 的版本必须严格统一, 这是避免“在我电脑上是好的”这类问题的最重要前提。
- 依赖管理: 确保 pom.xml 中的依赖版本号明确, 避免使用不稳定的 SNAPSHOT 版本, 除非有必要。

验收标准:

团队每位成员都能在本地成功启动项目, 无编译错误, 并能成功连接到本地数据库。可以通过运行一个简单的启动类或单元测试来验证。

4. 环境验证与团队互检

操作指南:

1. 基础验证: 每位成员独立完成以下检查:

- 项目能否成功编译 (mvn clean compile)。
 - 应用能否成功启动 (运行 Spring Boot 主类)。
 - 能否通过浏览器或 API 测试工具 (如 Postman) 访问项目的默认端口 (如 http://localhost:8080), 并看到预期响应 (如欢迎页面或检查接口的返回信息)。
2. 数据库验证: 运行一个简单的数据库查询脚本, 验证数据库连接和基础表结构是否正确。
3. 团队互检: 团队成员两两配对, 互相检查对方的本地环境配置是否正确, 特别是容易出错的环节, 如环境变量、配置文件中的密码等。

验收标准:

所有团队成员提交本地环境验证成功的截图(如 IDEA 成功运行界面、浏览器访问成功页面),并由项目经理汇总确认。团队内部互检记录归档。

【实验报告】

按附录 1《实验报告参考模版》要求编写,需包含以下核心内容:

实验环境:详细记录硬件配置、软件版本。

实验过程:按“团队组建-需求分析-计划编制-环境搭建”四个阶段,详细描述操作步骤,附关键过程截图。

实验结果及分析:列出所有用户故事清单,说明各用户故事的依赖关系和验收标准;分析实验过程中需求分析的完整性、技术选型的合理性、环境配置的稳定性。

问题与解决:记录实验过程中遇到的具体问题、解决方法、解决效果。

实验总结:总结实验的收获与体会,反思实验过程中存在的不足及改进方向。

【实验中的常见问题与解决方案】

1. 问题:需求范围模糊,导致后续频繁变更

- 解决方案:采用用户故事地图可视化全貌,明确区分 MVP(最小可行产品)与后续迭代范围。将 MVP 作为不可动摇的基线,所有新需求必须进入产品待办列表进行优先级排序。

2. 问题:技术选型争论不休,拖延项目启动

- 解决方案:制定决策矩阵。明确评估维度(如团队熟悉度、社区支持、项目匹配度),为各选项打分。设置决策截止时间,由技术负责人综合意见后拍板,并记录决策理由。。

3. 问题:团队章程空泛,缺乏实际约束力

- 解决方案:章程条款必须具体化、可执行。例如,将“及时沟通”具体为“工作消息 30 分钟内回复”;将“代码规范”具体为“提交前必须通过 ESLint 检测”。章程应在首次迭代回顾中复核修订。

【实验或拓展思考题】

1. “最小”的边界在哪里?

你们是如何界定 MVP(最小可行产品)的“最小”边界的?如果一个功能被一半用户认为是“必须有”,而被另一半用户认为是“锦上添花”,你如何决策?请尝试提出一种超越“少数服从多数”的决策框架。

2. “活”的章程 vs “死”的规范

《团队章程》被设计为一份“活”的文档。请设想，在什么情况下，你们会主动推翻或大幅修改章程中的既定规则？这种“灵活性”和规则本身的“权威性”之间，应如何平衡？

3. 技术选型的“后悔药”

技术选型本质上是基于当前信息的预测。如果项目进行到一半，发现当初的一个核心技术选型（如数据库、前端框架）存在严重缺陷或有了更优选择，你们会如何应对？请设计一个“技术选型纠偏”的轻量级流程。

【评分标准】

评分项	权重	评分标准
功能完整性	30%	所有交付物齐全，无遗漏；需求分析覆盖系统核心功能，用户故事地图逻辑清晰；开发计划可行，环境配置稳定可用。
代码规范性	20%	远程代码仓库分支管理规范，README.md 文档完整；本地项目代码命名、注释符合开发规范；文档命名格式统一，无格式错误。
实验报告质量	25%	实验过程描述详细，截图清晰；结果分析深入，结合项目实际；问题与解决记录具体，总结反思有深度。
创新性	15%	在需求分析中提出贴合业务的创新功能点；技术选型有优化；团队协作流程有改进。
团队协作	10%	Git 提交记录完整，反映团队成员有效分工；会议纪要详细，体现协作过程；交付物汇总及时，无推诿现象。

【参考教材与实验拓展】

参考教材：

《敏捷软件开发：原则、模式与实践》（Robert C. Martin 著）、

《用户故事与敏捷方法》（Mike Cohn 著）、

《用户故事地图》（Jeff Patton 著）。

在线资源：

华为 CodeArts 官方文档（<https://support.huaweicloud.com/codearts/>）、

MySQL 官方文档（<https://dev.mysql.com/doc/>）、

Spring Boot 官方指南（<https://spring.io/guides/gs/spring-boot/>）。

实验拓展：调研主流 MES 系统（如西门子 SIMATIC IT、SAP ME）的核心功能与架构设计，对比自身项目需求，优化产品待办列表；学习敏捷估算方法（如计划扑克），为后续迭代规划提供更精准的时间估算。

【注意事项】

实验室安全：禁止私自拔插实验室设备，避免因操作不当导致设备故障；遇到网络中断、软件崩溃等异常情况，立即报告指导教师，不得自行拆机维修。

学术诚信：引用 AI 生成的代码或他人开源代码时，需在代码注释中注明来源；所有文档和代码需独立完成，禁止抄袭。

团队协作：严格遵守团队章程约定的沟通机制和会议规则，每日同步工作进度；项目经理需及时跟踪交付物完成情况，协调解决团队冲突，确保实验按时完成。

数据安全：远程代码仓库需设置合理权限，避免项目代码泄露；重要文档需定期备份。

实验二：迭代 1（从规划到回顾的首次价值交付）

【实验目标】

（一）知识目标

掌握 MVP（最小可行产品）的核心概念与筛选方法，理解敏捷迭代“规划-冲刺-评审-回顾”的基础闭环逻辑。

熟悉故事点估算的基本方法，掌握 Git 分支管理、每日站会等基础协作工具与仪式的规范。

了解冲刺评审与迭代回顾的核心目的，掌握基础的反馈收集与问题梳理方法。

（二）能力目标

能够基于需求基线筛选核心用户故事，完成 MVP 定义与首次迭代规划，输出规范的冲刺计划文档。

能够跑通敏捷基础协作流程，有效组织每日站会、跟踪任务进度，解决首次协作中的基础障碍。

能够完成 MVP 核心功能开发与基础测试，实现首次价值交付与评审反馈收集。

能够组织基础的迭代回顾会议，梳理首次协作中的核心问题，形成初步改进方向。

（三）素质目标

强化团队初建阶段的协作意识，提升快速磨合与高效沟通的能力，适应敏捷开发的基础节奏。

培养“流程先行、规范落地”的工程理念，重视基础代码规范与版本管理习惯的养成。

建立“快速试错、及时调整”的思维，从首次迭代实践中积累协作与开发的基础经验。

【实验内容】

任务 1：完成第一次迭代规划

任务描述

基于实验一确立的需求基线与产品待办列表，确定迭代策略，通过故事点估算筛选构成 MVP 的最高优先级用户故事，明确冲刺目标、待办事项与交付标准，完成首次迭代规划。

任务知识

MVP 核心定义与筛选原则、故事点估算方法（计划扑克）、迭代规划流程、冲刺目标制定规范。

任务成果

冲刺计划会议纪要、冲刺待办列表（待办基线）、冲刺待办列表跟踪表（在线文档）、冲刺目标文档。

思路简述

以“快速交付可演示核心价值”为核心，优先选择覆盖你团队项目核心业务流程的用户故事构成 MVP；采用简化的故事点估算方法建立估算意识，无需追求精确值；明确冲刺目标，拆分冲刺待办事项并制定跟踪表，确保进度可追溯。

💡教学提示：MVP 的核心是“最小”与“可行”。本环节重在掌握价值优先的筛选逻辑。故事点估算（S/M/L）的目的是建立相对工作量概念，促进团队沟通，无需追求绝对精确。案例的业务流程仅供参考，你团队应定义属于自己的核心价值闭环。

任务 2：完成第一次迭代冲刺

任务描述

执行冲刺计划，严格遵循每日站会规范，使用 Git 进行代码开发与提交，实现初步的团队协作，使用 CodeArts 搭建任务实现代码检查，同步开展轻量测试与缺陷跟踪，确保冲刺任务按计划推进。

任务知识

每日站会规范、Git 操作、代码检查任务搭建、轻量测试用例设计、缺陷跟踪方法。

任务成果

每日站会记录、障碍清单、燃尽图/看板进度视图、轻量测试用例、测试报告、缺陷跟踪记录表、Git 提交记录、代码检查截图。

思路简述

以“团队磨合与流程建立”为核心，允许开发过程中的小范围试错；代码开发遵循“分支创建-开发-提交-合并”的规范流程；代码审查应聚焦于核心规范，初期无需过度复杂配置；测试与开发同步进行，重点覆盖核心功能的正向流程，及时跟踪并修复缺陷。

💡 **教学提示：**本次冲刺的首要目标是“跑通流程”，其次才是“完成功能”。允许在代码、协作上试错，但必须严格执行站会、代码提交、缺陷跟踪等仪式，旨在将敏捷协作规范内化为团队习惯。

任务 3：完成第一次冲刺评审

任务描述

组织内部冲刺评审会议，演示 MVP 核心功能，收集评审反馈，验证技术路径与协作流程的可行性，记录需求变更与缺陷问题，形成评审结论。

任务知识

冲刺评审会议流程、反馈收集与分析方法、需求变更管理基础、缺陷分级标准。

任务成果

冲刺评审会议纪要、用户变更记录表、评审反馈表（含修改意见）。

思路简述

评审重点聚焦“技术路径是否可行、协作流程是否顺畅、MVP 是否满足核心价值”，而非功能完美度；每个团队成员扮演“评审代表”，从不同视角（用户、开发、测试）提出反馈；对收集的反馈分类处理（缺陷修复、需求优化、后续迭代需求），明确是否需要调整后续迭代计划。

💡 **教学提示：**首次评审的重点是验证“技术路径是否可行”与“协作流程是否顺畅”，而非功能完美度。每个成员需从不同视角（用户、开发、测试）模拟真实评审，旨在培养从反馈中识别关键问题（Bug/优化/新需求）的能力。

任务 4：完成第一次冲刺演示与交付

任务描述

录制冲刺成果演示录屏，编写轻量化用户手册，完成系统部署发布，签订交付确认单，实现首次价值交付。

任务知识

产品演示技巧、轻量化用户手册编写规范。

任务成果

冲刺成果演示记录、冲刺成果演示录屏（3-5 分钟）、程序源码（含 README 说明）、轻量化用户手册、交付确认单。

思路简述

演示录屏聚焦 MVP 核心流程，时长控制在 3-5 分钟，清晰展示功能操作与实现效果；轻量化用户手册以“操作步骤+截图”为主，满足基础使用需求；交付确认单明确本次迭代的交付范围与验收标准，形成正式交付记录。

💡 **教学提示：**演示录屏是项目价值的集中体现。需用简洁的语言和操作，在短时间内清晰展示 MVP 解决的核心问题。案例中的操作步骤是形式，你团队应提炼自身项目最能打动人的价值点进行演示。

任务 5：完成第一次迭代回顾

任务描述

组织迭代回顾会议，聚焦流程本身（站会效率、沟通协作、工具使用等），梳理成功经验与待改进问题，制定 1-2 项核心改进措施，为后续迭代优化奠定基础。

任务知识

迭代回顾会议召开技巧、问题分析与改进措施制定方法。

任务成果

冲刺回顾会议纪要、改进计划表、回顾改进行动项（非必要）。

思路简述

回顾会议聚焦“过程改进”而非“结果追责”，引导团队成员坦诚交流；采用结构化模板收集意见，确保问题梳理全面；改进措施需具体、可落地，避免空泛表述；必要时更新《团队章程》，将改进措施固化为团队规则。

💡 **教学提示：**回顾会议的原则是“对事不对人”，聚焦改进流程而非追究责任。采用结构化模板（如“做得好的/待改进的/可尝试的”）引导讨论，旨在建立团队心理安全感，确保问题被公开、理性地解决。

【实验要求】

（一）交付物

任务一：冲刺计划会议纪要（含会议时间、参与人、迭代策略、冲刺目标、待办清单等）、冲刺待办列表（待办基线，DOCX 格式）、冲刺待办列表跟踪表（必要，在线文档，含计划完成时间、实际完成时间、任务状态、负责人等字段）、冲刺目标文档（明确冲刺核心价值、交付范围、验收标准）。

任务二：每日站会记录（必要，按日期整理，含每位成员发言要点、障碍记录）、障碍清单（含障碍描述、责任人、解决措施、解决状态）、燃尽图/看板进度视图（截图，需体现冲刺全周期进度变化）、轻量测试用例（含用例 ID、模块、功能点、输入数据、预期结果、实际结果、测试人员、测试时间）、测试报告（含测试范围、用例执行情况、缺陷统计、测试结论）、缺陷跟踪记录表（含缺陷 ID、模块、描述、严重程度、处理状态、修复人、修复时间）。

任务三：冲刺评审会议纪要（含会议时间、参与人、演示内容、评审意见、评审结论）、用户变更记录表（含变更 ID、需求描述、变更原因、处理方式、影响范围）、评审反馈表（含反馈人、反馈内容、类型（缺陷/优化/新需求）、处理意见）。

任务四：冲刺成果演示记录（含演示大纲、关键操作步骤、演示结论）、冲刺成果演示录屏（必要，MP4 格式，3-5 分钟，画面清晰、操作流畅、配音简洁）、程序源码（必要，压缩包格式，含完整项目代码）、轻量化用户手册（含系统简介、操作流程、常见问题）、交付确认单（含交付物清单、验收标准、确认签字）。

任务五：冲刺回顾会议纪要（含会议时间、参与人、讨论要点、成功经验、待改进问题）、改进计划表（含改进项、现状描述、改进措施、责任人、计划完成时间）、回顾改进行动项（非必要，针对关键改进项制定详细执行步骤）。

（二）质量标准

迭代规划合理：迭代策略制定合理，覆盖核心价值；故事点估算逻辑一致；冲刺目标明确，待办清单可执行。

协作流程规范：每日站会记录完整，障碍跟踪及时；Git 提交记录规范（分支命名、提交信息），体现有效分工；有效使用代码静态检查。

成果交付达标：核心功能可正常运行，无致命缺陷；演示录屏符合时长要求，内容清晰；文档格式统一、逻辑清晰，无错别字。

反馈与改进有效：评审反馈收集全面，分类合理；迭代回顾问题分析深入，改进措施具体可落地。

符合敏捷开发规范：全程遵循“规划-冲刺-评审-回顾”的迭代闭环，体现快速交付、持续反馈的核心思想。

【实验环境】

（一）硬件环境

处理器：Intel Core i5 及以上。

内存：8GB 及以上。

硬盘：剩余空间 50GB 及以上。

网络：稳定的互联网连接（用于访问远程代码仓库、项目管理平台、CI 工具）。

（二）软件环境

操作系统：Windows 10/11 64 位或 macOS 12 及以上。

开发工具：IntelliJ IDEA 2022 及以上、Visual Studio Code（可选）。

版本控制工具：Git、Git GUI（如 Tortoise，可选）。

项目管理平台：华为 CodeArts（含需求管理、甘特图、看板）。

持续集成工具：华为 CodeArts CI。

数据库：MySQL 8.0、Navicat 15 及以上。

录屏：使用腾讯会议或者其他会议软件进行录制。

文档工具：Microsoft Office 2019 及以上、WPS 2022 及以上。

【实验步骤】

阶段 1：迭代规划

本阶段目标是为第一次迭代冲刺制定清晰、可行的计划。核心任务是基于需求基线，筛选出构成最小可行产品（MVP）的核心用户故事，明确冲刺目标，并将任务拆解到可执行的程度。

1. 召开冲刺计划会议

操作指南：

1. 会议准备：会前，产品负责人（或项目经理）需准备好实验一产出的核心材料：产品待办列表基线、用户故事地图和用户故事清单。
2. 确定迭代策略：会议开始时，团队首先明确本次迭代的核心策略。首次迭代强烈建议采用“价值优先”策略，即优先交付那些能最直观体现项目核心价值、形成可演示闭环的

功能。

3. 筛选 MVP 用户故事：团队共同审视用户故事地图，筛选出位于最上层、覆盖核心业务流程的用户故事，构成 MVP。MVP 的故事数量不宜过多，建议 3-5 个，确保团队能在规定时间内完成。
- 示例：对于一个生产管理系统，MVP 可能是“用户登录” -> “查看生产任务列表” -> “上报任务工时” -> “查看个人工时统计”。这形成了一个最小闭环。
4. 进行故事点估算：采用简化的计划扑克法对筛选出的 MVP 用户故事进行估算，旨在建立估算意识，而非追求绝对精确。

步骤：

- 产品负责人讲解一个用户故事。
- 每位团队成员私下选择一张代表工作量的卡片（通常用 S、M、L 代表 1、3、5 个故事点）。
- 所有人同时亮出卡片。
- 估算值差异最大的成员简要说明理由。
- 团队讨论后，进行下一轮估算，直至达成共识。
- 记录最终的故事点。

避坑指南：

- 避免范围过大：首次迭代切忌贪多求全。MVP 的核心是“最小可行”，功能必须完整可用，但范围要小。
- 技巧：故事点估算的意义在于促进团队沟通和对工作量的共同理解。如果团队是全新的，前几个故事的估算可能不准，这是正常现象，重在启动这个过程。

验收标准：

团队就本次迭代要完成的 MVP 用户故事清单达成一致，并为每个故事赋予了经过讨论的故事点估值。

2. 确定冲刺目标与拆分任务

操作指南：

1. 定义冲刺目标：用一句简洁、明确、非技术性的语言描述本次迭代要达成的业务成果。
例如：“完成生产任务从创建到报工的核心流程演示，实现 MVP 价值交付。”
2. 拆分冲刺任务：将每个 MVP 用户故事拆分为具体、可执行、可分配的技术任务（Task）。
每个任务应能在 1-3 天内完成。

示例（用户故事：作为车间主任，我希望查看生产任务列表）：

- T001：设计数据库表结构（任务负责人：张三）。
- T002：开发后端查询 API 接口（任务负责人：李四）。

- T003: 开发前端任务列表页面（任务负责人：王五）。
- T004: 前后端接口联调（任务负责人：李四、王五）。

明确依赖关系：识别任务之间的先后依赖关系（如必须先完成数据库设计，才能开发后端接口），并在计划中明确标注。

验收标准：

形成一份明确的冲刺目标陈述，以及一份已将用户故事拆解为具体技术任务的冲刺待办事项清单，并明确了各项任务的负责人和初步的依赖关系。

3. 制定冲刺待办列表与跟踪表

操作指南：

1. 创建冲刺待办列表（基线）：将会议确定的 MVP 用户故事清单、故事点、冲刺目标整理成一份正式的文档（DOCX 格式）。此文档作为本次迭代的范围基线，原则上不应变更。
2. 创建冲刺待办列表跟踪表（动态）：使用在线协作文档（如腾讯文档、金山文档）创建动态跟踪表。包含以下核心字段：
 - 任务 ID、任务标题、所属用户故事、负责人、计划开始日期、计划完成日期、实际开始日期、实际完成日期、任务状态（待办/进行中/已完成/阻塞）、备注/障碍。
3. 初始化跟踪表：将步骤 2 中拆分的所有技术任务录入此跟踪表，并设定计划日期。

避坑指南：

- 任务粒度：拆分的任务要足够小，以便于跟踪。如果一个任务需要一周才能完成，说明拆得不够细。
- 工具使用：使用在线文档是为了实现信息的透明和实时同步。务必确保所有团队成员都有编辑权限，并约定每日更新进度。

验收标准：

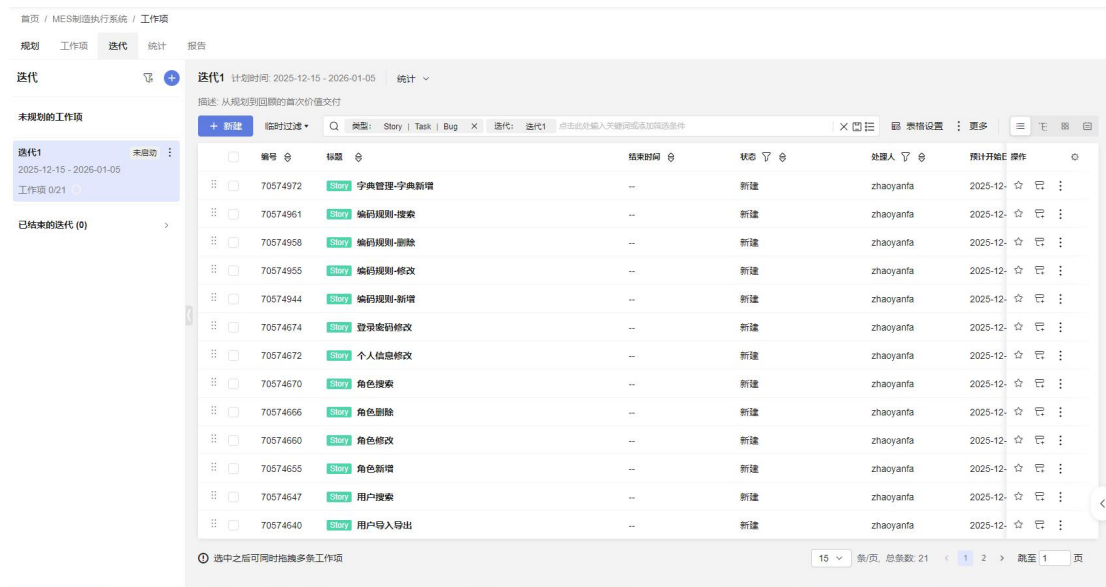
产出两份清晰的文档：一份静态的冲刺计划基线文档，和一份在线的、已初始化完毕的冲刺任务动态跟踪表。

4. 在 CodeArts 中规划迭代

操作指南：

1. 登录华为 CodeArts 平台，进入项目。
2. 规划迭代：在“项目计划”或“迭代计划”中，创建一个新的迭代周期（如“Iteration 1”），并设置好开始和结束日期。
3. 关联工作项：将已在 CodeArts 中创建的、属于本次 MVP 的 Story 级工作项，拖拽到这个新建的迭代中。

4. 生成迭代视图：切换到看板视图，此时可以看到本迭代的所有 Story。团队可以在此基础上，为每个 Story 创建子任务（Task），并分配给具体成员。



验收标准：

在 CodeArts 平台上成功创建了迭代，并将 MVP 包含的所有用户故事（Story）分配到了该迭代中，团队可以通过看板清晰查看迭代任务。

阶段 2：迭代冲刺

本阶段目标是执行迭代计划，通过每日站会、代码开发、测试验证等敏捷仪式，完成 MVP 功能的开发与集成。核心目标是“跑通流程”，实现首次价值交付，并建立团队协作节奏。

1. 召开每日站会

操作指南：

- 会议纪律：每天在固定时间（如上午 9:00）召开站会，时长严格控制在 15 分钟以内。建议设一名计时员。
- 发言规则：每位团队成员按顺序回答三个核心问题，每人发言时间不超过 1 分钟：
 - 昨日完成：我昨天完成了什么？（对应冲刺任务跟踪表中的具体任务项）
 - 今日计划：我今天计划完成什么？（明确具体的任务 ID 或描述）
 - 遇到障碍：我遇到了什么阻碍，需要什么帮助？（清晰描述问题，但不展开讨论）
- 问题处理：站会只识别和暴露问题，不深入讨论解决方案。记录员将障碍记录到“障碍清单”中，明确责任人和解决时限。相关人员在站会后立即组织小范围会议专门解决。
- 进度同步：站会结束后，每位成员立即更新 CodeArts 看板上的任务状态（如“待处理”→“进行中”→“已完成”），确保信息透明。

避坑指南：

- 避免超时：严格遵循时间盒，防止站会变成技术讨论会。技术细节问题留到会后解决。
- 技巧：站会前，成员可花 1-2 分钟快速梳理发言要点，确保表达简洁、聚焦。

验收标准：

每日站会记录完整、规范，障碍清单得到及时跟踪和处理，CodeArts 看板能真实反映当前迭代进度。

2. 任务冲刺与代码开发**操作指南：**

1. 拉取最新代码：每日开始工作前，首先执行 `git pull origin dev` 命令，将开发主分支的最新代码同步到本地，避免后续合并冲突。
2. 创建功能分支：严格遵循既定的分支管理规范。从 `dev` 分支创建新的功能分支进行开发。
 - 分支命名示例：feature/US001-user-login
3. 小步提交：遵循“少量多次”的提交原则。完成一个小功能点或修复一个缺陷后，就执行一次提交。提交信息必须清晰规范。
 - 提交信息格式示例：[T001] feat: 完成用户登录前端页面开发或 [T005] fix: 修复登录接口空指针异常
4. 发起合并请求（Pull Request）：当一个功能开发并自测完成后，将代码推送到远程仓库，并在 CodeArts 上针对该功能分支发起一个合并请求（PR），请求将代码合并到 `dev` 分支。
5. 代码审查：邀请至少一位团队成员进行代码审查。审查者需检查：
 - 功能正确性：代码是否实现了需求？
 - 代码规范：命名、注释、结构是否符合团队约定？
 - 潜在缺陷：是否有明显的逻辑错误或安全隐患？
 - 审查意见需具体，并可直接在 PR 页面上对指定代码行进行评论。
6. 合并代码：审查通过后，由代码提交者或项目负责人将代码合并到 `dev` 分支，并关闭该功能分支。

避坑指南：

- 避免长期不合并：功能分支的生命周期应尽可能短（建议 1-3 天），以减少合并冲突的风险。
- 技巧：代码审查是提升代码质量的关键环节，态度应友好、专业，以改进代码为目标。

验收标准：

所有功能开发均通过功能分支完成，代码提交记录规范，合并请求流程执行严格，代码审查意见得到落实。

3. 同步开展测试

操作指南：

1. 编写轻量测试用例：测试人员与开发同步，针对 MVP 的核心功能点编写测试用例。重点覆盖“快乐路径”（正常流程）和关键异常场景。
 - 测试用例要素：用例 ID、测试模块、操作步骤、输入数据、预期结果、实际结果。
2. 执行测试与缺陷跟踪：
 - 开发人员通知测试人员某个功能已完成开发并可测试。
 - 测试人员执行测试用例，记录实际结果。
 - 当实际结果与预期不符时，在“缺陷跟踪记录表”中新建一个缺陷，清晰描述问题现象、复现步骤、预期结果、实际结果，并标注严重程度（如：致命、严重、一般、提示）。
 - 将缺陷指派给对应的开发人员。
3. 修复与回归：开发人员修复缺陷后，将缺陷状态改为“已修复”，并通知测试人员。测试人员对缺陷进行回归测试，通过后关闭缺陷。

避坑指南：

- 缺陷描述不清：避免使用“功能不好用”等模糊描述，必须提供可复现的具体步骤。
- 技巧：缺陷的严重程度有助于确定修复的优先级，应客观评估。

验收标准：

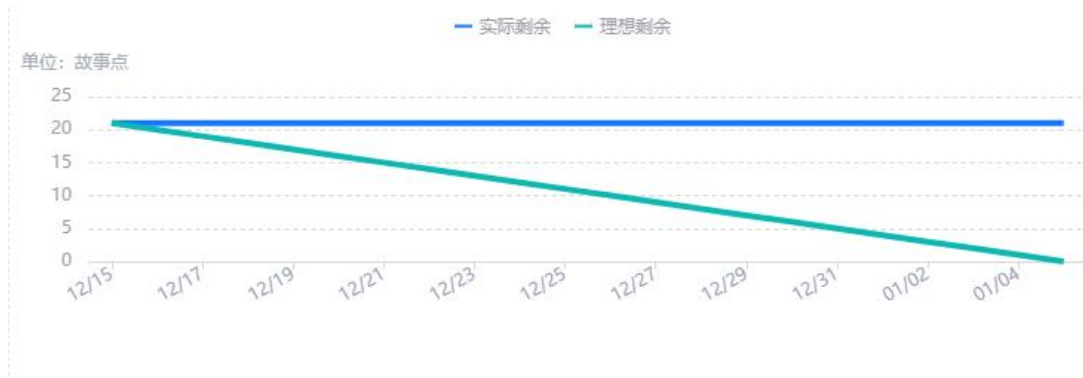
核心功能有对应的测试用例，发现的缺陷被及时记录、跟踪并关闭，缺陷跟踪记录表完整。

4. 进度跟踪与可视化

操作指南：

1. 更新跟踪表：每位成员每天工作结束时，更新冲刺任务动态跟踪表中的任务状态（进度百分比、实际开始/完成时间）。
2. 同步看板状态：根据实际进度，在 CodeArts 看板上拖动任务卡片到对应的列（如“待处理”、“进行中”、“待测试”、“已完成”）。
3. 查看燃尽图：团队每日关注由 CodeArts 自动生成的迭代燃尽图。该图通过对比“剩余工作量”和“理想剩余工作量”来直观反映项目进度是否健康。
 - 理想情况：实际线紧贴理想线下降。
 - 实际线高于理想线：表示进度滞后，团队需分析原因并采取措施（如重新评估任务、调

整分工）。



避坑指南:

- 进度更新不及时: 滞后的进度更新会导致燃尽图失真, 失去指导意义。必须养成每日更新的习惯。

验收标准:

冲刺任务跟踪表和 CodeArts 看板状态每日保持同步, 燃尽图能有效指导团队识别进度风险。

阶段 3: 迭代冲刺评审

本阶段目标是向“客户”(模拟角色)展示并验证本次迭代所交付的成果, 收集反馈以确认产品方向是否正确、价值是否得以实现。评审的核心是“检验成果”和“收集信息”, 而非解决问题。

1. 评审准备

操作指南:

1. 确定评审团队角色: 团队内部提前分配好角色, 每位成员需扮演一个特定的“评审代表”视角, 例如:
 - 最终用户代表: 重点关注功能是否符合实际工作习惯、操作是否便捷。
 - 业务方代表: 重点关注功能是否解决了核心业务问题、带来了何种价值。
 - 运维代表: 重点关注系统的部署、稳定性、日志等情况。
 - 技术负责人代表: 重点关注代码结构、技术实现合理性。
2. 准备演示环境与数据: 确保用于演示的系统版本是稳定、最新的。准备一套完整的、真实的测试数据, 确保演示流程能够顺畅跑通, 避免使用无意义的“aaa”、“123”等假数据。
3. 制定演示脚本: 编写详细的演示脚本, 明确演示的先后顺序、每个功能点要展示的核心价值、以及关键的讲解词。脚本应围绕“我们为用户解决了什么问题”来展开。

- 示例脚本片段：“大家好，现在演示的是‘任务报工’功能。作为操作工，之前他需要手工填写纸质单据，现在他可以在终端上直接扫描工单二维码，点击‘开始’和‘完成’，系统会自动记录时间，大大提升了效率和准确性。”

避坑指南：

- 避免临时准备：演示数据和脚本必须提前准备并反复演练，避免评审时现场出现卡顿、报错或逻辑混乱。
- 技巧：演示数据最好能讲述一个完整的“用户故事”，让评审者能代入场景。

验收标准：

评审角色分配明确，演示环境稳定，测试数据真实有效，演示脚本逻辑清晰且经过演练。

2. 召开冲刺评审会议

操作指南：

1. 成果演示（占总时长 2/3）：由 1-2 名主讲人按脚本进行演示。
 - 价值先行：开场先简要重申本次迭代的目标（即要解决的核心问题）。
 - 流程闭环：按实际业务顺序演示，展示一个完整的、端到端的业务流程，而不是孤立的功能点。
 - 突出亮点：重点展示相比迭代前（或传统方式）的改进和优势。
2. 收集反馈（占总时长 1/3）：演示结束后，引导各位“评审代表”从不同维度提出反馈。记录员需详细记录每条反馈。
 - 功能与价值维度：“这个功能是否解决了你的痛点？还有什么地方觉得不方便？”
 - 用户体验维度：“操作步骤是否清晰？界面提示是否明确？”
 - 潜在问题维度：“在实际使用中，你觉得可能会遇到什么问题？”
3. 处理反馈与需求变更：
 - 对反馈进行分类：缺陷（功能错误）、优化（体验提升）、新需求（超出当前范围）。
 - 原则上，本次迭代不接受新增需求。所有新需求应记录到产品待办列表，纳入后续迭代考量。
 - 对于缺陷和优化建议，讨论其紧急程度和影响范围，决定是否在本迭代内紧急修复，还是纳入下个迭代。
4. 形成评审结论：团队基于反馈讨论后，由产品负责人宣布评审结论：通过、有条件通过（需在约定时间内修复少量关键问题）或不通过（需重新迭代）。

避坑指南：

- 避免辩解：在评审过程中，开发团队的主要任务是“展示”和“倾听”，而不是解释技

术难点或辩解设计理由。重点理解反馈背后的真实需求。

- 技巧：采用“反馈-澄清-确认”的模式。例如，当评审者说“这个按钮不明显”，可以澄清“您的意思是希望它更醒目一些吗？”，确认理解无误后再记录。

验收标准：

会议按计划完成，演示过程流畅，反馈被全面、客观地记录，并形成了明确的评审结论和处理意见。

3. 输出评审成果

操作指南：

1. 整理评审纪要：会后立即整理会议纪要，内容应包括：会议时间、参与人、演示内容摘要、收集到的所有反馈（需分类）、评审结论以及后续行动项。
2. 更新产品待办列表：根据评审结论，更新产品待办列表：
 - 将确认需要在本迭代内修复的缺陷移至迭代待办列表顶部，并设置高优先级。
 - 将优化建议和新需求记录到产品待办列表（动态部分），并为它们评定优先级。
3. 生成反馈表：将反馈整理成结构化的表格，明确每条反馈的提出人、类型、描述、处理建议（立即修复/下期优化/纳入需求池）、责任人，方便跟踪。

验收标准：

产出完整的冲刺评审会议纪要、更新后的产品待办列表以及结构化的评审反馈表，为后续工作提供清晰依据。

阶段 4：冲刺演示与交付

本阶段目标是将迭代成果进行标准化封装，形成可交付、可演示、可追溯的正式版本。核心任务是制作高质量的演示材料，完成系统部署，并实现首次价值交付的闭环。

1. 录制冲刺成果演示录屏

操作指南：

1. 编写录屏脚本：基于冲刺评审的演示脚本，进一步提炼和优化，形成专门的录屏脚本。脚本应逻辑清晰，重点突出，时长严格控制在 3-5 分钟。
 - 开场（30 秒）：简要介绍项目名称、团队、本次迭代的核心目标。
 - 核心演示（2.5-4 分钟）：按实际业务流展示 MVP 功能。操作精准，语言简洁，避免冗余步骤和口头禅。
 - 结尾（30 秒）：总结本次迭代实现的核心价值，并简要展望。
2. 准备演示环境：确保演示环境纯净、稳定。关闭不必要的应用程序通知，清理浏览器缓

存，使用预设的、有代表性的测试数据。

3. 进行录制：

- 工具：使用腾讯会议、OBS 或其他录屏软件，确保分辨率不低于 1920x1080，帧率流畅。
 - 画面：聚焦应用程序界面，鼠标移动轨迹清晰，关键操作点可适当放缓或使用标注工具高亮。
 - 配音：使用外接麦克风，确保语音清晰、无杂音。语速平稳，与操作同步。
4. 后期剪辑与审核：录制完成后，进行简单剪辑，剪掉口误和长时间等待。将初版视频在团队内部小范围播放审核，根据反馈进行修改，形成最终版本。

避坑指南：

- 避免超时与随意：时长是硬性要求，需通过反复演练来控制。内容必须严谨，体现专业性。
- 技巧：配音可先录制音轨，再根据音轨匹配操作画面，以确保音画同步和表述流畅。

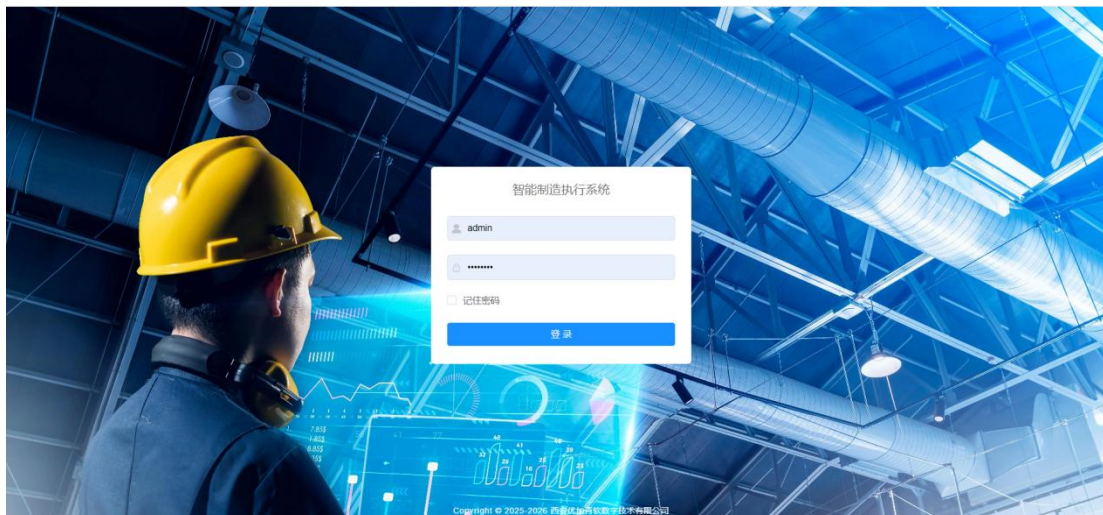
验收标准：

产出时长 3-5 分钟、画质清晰、配音专业、内容紧扣 MVP 核心价值的演示录屏文件（MP4 格式）。

2. 编写轻量化用户手册

操作指南：

1. 确定手册结构：手册应简洁，聚焦于“如何快速上手”，建议包含：系统简介、环境要求、核心功能操作指南、常见问题排查。
2. 采用“步骤+截图”范式：针对每个核心功能，按真实操作顺序，一步一步地说明，并配以当前系统最新版本的界面截图。截图需清晰，关键区域可用红框标注。示例：
 - 步骤 1：打开浏览器，输入系统网址。
 - 步骤 2：在登录页面，输入用户名（test01）和密码（123456），点击【登录】。



3. 语言通俗易懂：避免使用技术术语，从最终用户的角度描述操作，让一个新手能按图索骥完成操作。
4. 生成 PDF 版本：最终定稿后，将文档转换为 PDF 格式，便于分发和阅读，防止格式错乱。

验收标准：

产出一份结构清晰、图文并茂的轻量化用户手册（PDF/DOCX 格式），用户能依据手册独立完成系统核心功能的使用。

3. 程序源码整理与部署

操作指南：

1. 代码整理：

- 确保所有本次迭代的开发代码都已合并到 dev 分支。
- 检查代码注释是否完整，特别是复杂业务逻辑处需有简要说明。
- 确保根目录下的 README.md 文件内容详实，包含：项目简介、技术栈、本地开发环境配置步骤、部署说明。
- 清理临时代码、无用注释和调试语句。

2. 系统部署：

- 本地部署（首选）：确保项目可以在团队成员的本地开发环境（如 IntelliJ IDEA）中一键启动运行。提供详细的依赖安装和数据库初始化脚本。
 - 云服务器部署（可选）：若条件允许，可在阿里云/腾讯云等平台申请一台按量付费的云服务器（ECS），完成 JDK、MySQL、Redis 等环境的配置，并将项目部署至云端，确保可通过公网 IP 访问。此方式演示效果更佳。
3. 部署验证：部署完成后，必须执行一轮完整的核心业务流程测试，确保部署后的系统功能正常。

避坑指南：

- 忽略文档：README.md 是项目的门面，也是可交付性的重要体现，必须认真编写。避免出现“在本地运行一下就知道”等不专业表述。

验收标准：

源码仓库的 dev 分支代码整洁、注释完整，README.md 文件包含完整的运行指导，系统在目标环境（本地或云服务器）上可稳定运行。

4. 完成交付确认**操作指南：**

1. 整理交付物清单：汇总本次迭代的所有交付物，形成清单，包括：演示录屏、程序源码（仓库地址）、轻量化用户手册、相关文档（会议纪要、评审记录等）。
2. 签订交付确认单：制作一份正式的交付确认单，内容应包括：项目名称、迭代周期、交付物清单、验收标准（如“MVP 核心流程运行通畅”）、交付日期。由团队负责人和指导教师（模拟客户）签字确认。
3. 归档：将所有最终版的交付物打包，按规定路径提交至教学平台，完成本次迭代的正式交付。

验收标准：

交付物齐全并已归档，交付确认单签订完毕，标志着本次迭代价值交付流程正式闭环。

阶段 5：冲刺回顾

本阶段目标是团队对第一次迭代的全过程进行复盘，聚焦于“协作流程”和“团队效能”，而非“功能结果”。核心任务是坦诚地梳理成功经验和待改进问题，并制定具体、可落地的改进措施，为后续迭代奠定更高效的协作基础。

1. 召开迭代回顾会议**操作指南：**

1. 营造安全氛围：会议主持人（通常为敏捷教练）在会议开始时明确回顾会议的原则：对事不对人、聚焦改进而非指责、鼓励坦诚发言。目标是让团队变得更好。
2. 采用结构化复盘模板：建议使用“做得好的/待改进的/可尝试的”三栏法，引导讨论聚焦。

个人反思（10 分钟）：每位成员独立思考，并在便签上写下：

- 做得好的（绿色便签）：本次迭代中团队或自己在流程、协作、工具使用等方面做得好的地方。
- 待改进的（黄色便签）：遇到的挫折、低效环节或可以做得更好的地方。

- 可尝试的（蓝色便签）：想到的可以尝试的新方法、工具或技巧。

集体分享与聚类（20 分钟）：

- 每位成员轮流将自己的便签贴到共享白板（或在线文档）的对应栏目下，并简要说明。
- 所有便签贴完后，团队一起将类似内容的便签进行归类，形成几个主要的主题（如“每日站会”、“代码审查”、“沟通效率”）。

深度分析根源（10 分钟）：针对“待改进的”中投票出的最突出的 2-3 个问题，使用“五个为什么（5 Why）”方法追问根源，避免停留在表面现象。

- 示例：问题“代码合并冲突多” -> 为什么？因为多人同时修改同一文件 -> 为什么？因为开发前没有明确接口和分工 -> 根源：任务拆分不够细，沟通不足。

制定改进措施（15 分钟）：针对根源，讨论具体、可执行的改进措施，并明确责任人和下次回顾会议时的验收标准。

避坑指南：

- 流于形式：避免将回顾会开成“表彰会”或“批斗会”。要引导深入分析问题根源，否则改进措施会无效。
- 技巧：主持人要引导讨论，确保每位成员都有发言机会，对事不对人。

验收标准：

回顾会议按流程有效进行，所有成员参与其中，并产出了归类后的意见和初步的改进方向。

2. 输出回顾会议成果

操作指南：

1. 编写回顾会议纪要：纪要需清晰记录：会议时间、参与人、讨论出的主要成功经验、待改进问题清单、对关键问题的根源分析、以及最终确定的改进措施。

2. 制定改进计划表：将改进措施转化为正式的改进计划表，包含以下字段：

改进项、现状描述、改进目标、具体措施、责任人、计划完成时间、验证标准。示例：

- 改进项：提高每日站会效率。
 - 现状描述：站会常超时，易陷入技术细节讨论。
 - 改进目标：站会严格控制在 15 分钟内。
 - 具体措施：①指定计时员；②会前收集障碍；③技术问题会后再议。
 - 责任人：敏捷教练。
 - 计划完成时间：从下次迭代第一天开始执行。
 - 验证标准：接下来一周的站会平均时长低于 15 分钟。
3. 更新团队章程：若改进措施涉及团队协作规则的根本性变化（如站会规则、代码提交规范），则需要更新《团队章程》，并将新版章程同步给全体成员。

验收标准：

产出内容详实的迭代回顾会议纪要和一份具体、可跟踪的改进计划表。若有必要，产出更新版的《团队章程》。

3. 同步与承诺**操作指南：**

1. 公示结果：将回顾会议纪要和改进计划表在团队共享空间（如团队群、在线文档）公示，确保所有成员知悉。
2. 责任到人：改进措施的负责人需明确自己的任务，并在后续工作中主动推进。
3. 融入下一步计划：将改进计划表中的事项，作为“流程改进任务”纳入下一次迭代（迭代2）的冲刺待办列表，确保改进措施能真正落地，而非停留在纸面。

验收标准：

所有团队成员对回顾结论和改进计划达成共识，改进措施已纳入迭代2的规划中。

【实验报告】

按附录1《实验报告参考模版》要求编写，需包含以下核心内容：

实验环境：详细记录硬件配置、软件版本、远程代码仓库地址。

实验过程：按“迭代规划-迭代冲刺-冲刺评审-演示交付-迭代回顾”五个阶段，详细描述操作步骤、关键决策、工具使用方法，附核心截图。

实验结果及分析：

成果总结：列出本次迭代的所有交付物，说明核心功能的实现情况。

数据统计：统计故事点完成情况、缺陷数量、站会平均时长等数据。

问题分析：分析本次迭代中存在的核心问题，结合数据说明。

问题与解决：详细记录实验过程中遇到的具体问题、排查过程、解决方案及效果，每个问题需对应实际操作场景。

实验总结：总结本次迭代的收获，反思改进措施的可行性，展望迭代2的优化方向。

【实验中的常见问题与解决方案】**1. 问题：每日站会超时，沦为技术讨论会**

- 解决方案：严格执行15分钟时间盒。每人发言不超过1分钟，只答三件事：昨天完成、今天计划、遇到障碍。技术细节问题记入“停车场”，会后由相关人员专题讨论。

2. 问题：首次迭代贪多求全，MVP 目标过大

- 解决方案：采用 MoSCoW 法则 严格筛选需求。首次迭代只做“必须有”的功能，确保能交付一个可运行、可演示的最小功能闭环。宁可做少，不可做不全。

3. 问题：迭代末期代码合并冲突频繁

- 解决方案：推行小批量、频繁提交。鼓励每天完成一个小功能点就合并到开发分支，避免长期不合并。开发前简单沟通接口设计，减少协作冲突。

【实验或拓展思考题】

1. **MVP 的“谎言”** 首次迭代交付的 MVP，有时为了追求“可演示性”而简化了异常流程和数据一致性，这可能导致它并非一个真正“可行”的产品。你们是如何处理这种“真实可行性”与“演示速度”之间的矛盾？你认为一个合格的 MVP 应该坚守的底线是什么？
2. **站会的“叛逆者”** 经典的站会三问（昨天、今天、障碍）是否可能扼杀创新和自发协作？如果一位团队成员每天站会都说“我继续研究 XX 技术难题，无明确产出，无障碍”，你认为这是否合理？你会如何引导他以获得对团队更有价值的信息？
3. **代码的“道德”** 在快速交付的压力下，可能会写出一些“能工作但很丑”的代码（即欠下技术债务）。请探讨：在首次迭代中，我们应如何判断哪些技术债务是“可接受的短期贷款”，哪些是“绝不能欠的高利贷”？你的判断标准是什么？

【评分标准】

评分焦点

跑通敏捷基础流程，完成团队磨合与规范建立

是否按要求完成所有协作仪式（站会、评审、回顾）、代码提交是否规范、团队分工是否清晰

评分项	权重	评分标准
功能完整性	25%	MVP 核心功能实现，满足基础业务流程；无致命缺陷，可正常演示；交付物齐全且符合规范。
代码规范性	25%	Git 分支管理符合约定；代码命名、注释规范，无明显语法错误；文档命名统一、格式正确。
实验报告质量	20%	实验过程描述详细，清晰记录团队组建、协作磨合过程；问题与解决记录具体，体现首次协作的真实情况。
创新性	10%	在团队协作流程或基础功能实现上有合理优化，体现初步思考。
团队协作	20%	每日站会记录完整，障碍跟踪及时；Git 提交记录反映有效分工；回顾会议能准确梳理首次协作的核心问题。

【参考教材与实验拓展】

参考教材：

《Scrum 精髓：敏捷转型指南》（Kenneth Rubin 著）、

《持续集成：软件质量保障实践》（Paul Duvall 著）、

《Pro Git》（Scott Chacon 著）、

《持续交付》（Jez Humble 著）。

在线资源：华为 CodeArts CI 官方文档

（<https://support.huaweicloud.com/codearts-pipeline/>）、

GitHub Actions 官方指南（<https://docs.github.com/zh/actions>）、

Scrum 官方指南（<https://www.scrumguides.org/>）。

实验拓展：

学习复杂 CI/CD 流水线配置（如集成自动化测试、自动部署到测试环境）。

调研主流敏捷项目管理工具（如 Jira、Trello）的迭代规划功能，对比 CodeArts 的差异。

学习缺陷管理工具（如 Jira、禅道）的使用，替代手动编写缺陷跟踪记录表。

【注意事项】

学术诚信：引用 AI 生成的代码或他人开源代码时，需在代码注释和 README.md 中注明来源；演示录屏需为团队原创，禁止剪辑他人成果。

工具使用规范：远程代码仓库需设置合理权限，避免代码泄露；录屏内容需聚焦实验成果，禁止包含无关信息。

协作规范：严格遵守每日站会、评审会议的时间约定，缺席需提前请假；任务负责人需对分配的任务全程负责，确保按时完成；反馈他人工作时需客观公正，以解决问题为核心。

交付物质量：演示录屏需保证音视频清晰、无杂音，内容完整；文档命名需规范，避免混乱；源码需确保可编译、可运行，避免提交未完成或报错的代码。

实验三：迭代 2（基于回顾的增量交付与流程优化）

【实验目标】

（一）知识目标

掌握基于迭代回顾的改进措施落地方法，理解“持续改进”的敏捷核心思想。

熟悉代码质量基础评估标准，掌握基础的代码优化与轻量测试强化技巧。

深化对敏捷协作仪式的理解，掌握更高效的站会组织、评审反馈转化方法。

（二）能力目标

能够基于迭代 1 的回顾改进项，完成迭代 2 规划，将改进措施转化为具体开发与协作行动。

能够优化协作流程，提升每日站会、代码提交、评审反馈的效率，减少首次迭代中出现的问题。

能够强化代码质量与测试覆盖，减少缺陷数量，实现增量功能的稳定交付。

能够组织针对性的迭代回顾会议，聚焦改进措施的落地效果，形成更具体的后续优化方案。

（三）素质目标

培养“质量内建”的工程思维，将代码规范、测试验证融入开发全流程，而非事后补救。

强化“持续改进”的职业素养，能够从实践中总结问题、落地措施、验证效果。

提升团队协作的默契度，形成稳定的沟通机制与问题解决模式。

【实验内容】

任务 1：完成第二次迭代规划

任务描述

基于迭代 1 的回顾改进项与剩余需求，优先推进模块化设计的核心业务功能增量开发，将代码规范、协作流程等改进措施转化为具体任务，完成迭代 2 的冲刺计划。

任务知识

迭代规划优化方法、改进项落地路径、模块化设计基本原则。

任务成果

冲刺计划会议纪要、冲刺待办列表（待办基线）、冲刺待办列表跟踪表（在线文档）、冲刺目标文档。

思路简述

以“落地改进措施、夯实设计基础”为核心，将迭代 1 关于代码规范、沟通效率的改进项拆解为具体任务；结合剩余业务需求，优先开发能体现模块化设计的核心功能；功能设计避免过度复杂，聚焦“规范落地+可用稳定”。

💡 **教学提示：** 本迭代规划的核心是将回顾会议的“改进措施”转化为可执行的“具体任务”。例如，若迭代 1 的问题是“代码不规范”，则本迭代待办列表中必须有“落实代码规范检查”的具体任务，确保改进落地。

任务 2：完成第二次迭代冲刺

任务描述

落地迭代 1 的流程改进措施，采用规范的代码审查机制开发核心业务模块，进行模块化设计并针对关键函数编写基础单元测试，同步开展轻量测试与缺陷跟踪，确保冲刺任务高效推进。

任务知识

代码审查流程、模块化设计方法、基础单元测试编写、轻量测试用例设计、缺陷跟踪方法。

任务成果

每日站会记录、障碍清单、燃尽图/看板进度视图、测试报告、缺陷跟踪记录表、代码审查记录、核心模块设计图

思路简述

以“流程优化+设计改进”为核心，严格落实迭代 1 确定的改进措施；开发过程中注重模块化拆分，降低模块耦合度；针对核心函数编写基础单元测试，不追求覆盖率，重点培养测试意识；代码审查与开发同步进行，聚焦代码规范落地与功能正确性。

💡 **教学提示：** 本迭代是“质量内建”的关键阶段。重点从“实现功能”转向“写好代码”，代码审查和单元测试是提升质量的核心手段。审查时需关注设计合理性（如

模块耦合度），而不仅是语法正确性。

任务 3：完成第二次冲刺评审

任务描述

组织高标准的冲刺评审会议，演示增量功能与模块化设计成果，说明代码规范与协作流程较迭代 1 的改进点，从功能完整性、代码规范度、设计合理性、流程优化效果四个维度收集反馈，形成评审结论。

任务知识

多维度评审流程、代码规范评估方法、设计合理性讨论技巧、反馈分类与优先级排序。

任务成果

冲刺评审会议纪要、用户变更记录表、评审反馈表（含功能/代码/设计/流程四类反馈）。

思路简述

评审标准较迭代 1 显著提升，不仅关注功能可用，更重视代码规范与设计改进；演示时需对比迭代 1 代码，说明改进细节（如命名规范、重复代码优化）；对反馈按优先级排序，核心缺陷需在本次迭代内修复，优化类反馈纳入迭代 3。

💡 **教学提示：**评审标准较迭代 1 应显著提高，需关注代码结构、设计模式、数据接口设计等技术决策的合理性。演示时应准备前后对比（如迭代 1 与迭代 2 的代码对比），体现改进效果，而不仅是新功能展示。

任务 4：完成第二次冲刺演示与交付

任务描述

录制 5-10 分钟的冲刺成果演示录屏，详细展示增量功能、模块化设计亮点及改进落地效果；完善轻量化用户手册；完成系统平滑升级，签订交付确认单。

任务知识

复杂功能演示技巧、用户手册优化方法。

任务成果

冲刺成果演示记录、冲刺成果演示录屏（5-10 分钟）、程序源码、轻量化用户手册（V2.0）、交付确认单。

思路简述

演示录屏重点突出模块化设计优势与改进落地效果，通过对比迭代 1 场景体现进步；用户手册补充新增功能操作指南与设计规范相关说明；系统升级采用“平滑过渡”方式，确保迭代 1 功能不受影响，记录升级过程与注意事项。

💡 **教学提示：**本环节演示需突出“演进”二字。除了新功能，更要展示架构优化、性能提升、用户体验改善等隐性价值。用户手册应开始沉淀设计决策和接口说明，为后续开发和维护提供支持。

任务 5：完成第二次迭代回顾

任务描述

聚焦改进措施落地效果与模块化设计实践，梳理技术与流程层面的核心问题，总结成功经验，制定长效改进方案，为迭代 3 奠定基础。

任务知识

技术复盘方法、改进措施优化策略、设计经验沉淀技巧。

任务成果

冲刺回顾会议纪要、改进计划表。

思路简述

回顾会议采用“问题-根源-措施”结构化模板，重点复盘改进项落地不彻底的原因；针对模块化设计中遇到的问题，提炼可复用的设计经验；改进措施需具体可落地，明确责任人与验证标准，必要时更新团队章程。

💡 **教学提示：**回顾会议应深入技术层面，复盘架构设计、技术债务、测试策略等核心问题。采用“5Why”等根因分析法，避免停留在表面现象。改进措施应具体到代码/设计层面，并为迭代 3 的技术改进提供明确输入。

【实验要求】

（一）交付物

任务一：冲刺计划会议纪要、冲刺待办列表（待办基线）、冲刺待办列表跟踪表（必要，在线文档或管理平台）、冲刺目标文档。

任务二：每日站会记录（必要）、障碍清单、燃尽图/看板进度视图（截图）、测试报告、缺陷跟踪记录表、代码审查记录、架构评审记录。

任务三：冲刺评审会议纪要、用户变更记录表、评审反馈表。

任务四：冲刺成果演示记录、演示录屏（必要，MP4 格式，5-10 分钟）、程序源码（必要，压缩包）、轻量化用户手册、交付确认单。

任务五：冲刺回顾会议纪要、改进计划表。

（二）质量标准

代码质量达标：代码命名、注释规范，无冗余代码，代码重复率<10%，无高危漏洞，架构一致性良好。

流程改进落地：迭代 1 的改进项 100%落地。

交付物规范：所有文档格式统一，源码可编译运行，演示录屏清晰完整，符合时长要求。

【实验环境】

（一）硬件环境

处理器：Intel Core i5 及以上。

内存：8GB 及以上。

硬盘：剩余空间 50GB 及以上。

网络：稳定的互联网连接（用于访问远程代码仓库、项目管理平台、CI 工具）。

（二）软件环境

操作系统：Windows 10/11 64 位或 macOS 12 及以上。

开发工具：IntelliJ IDEA 2022 及以上、Visual Studio Code（可选）。

版本控制工具：Git、Git GUI（如 Tortoise，可选）。

项目管理平台：华为 CodeArts（含需求管理、甘特图、看板）。

持续集成工具：华为 CodeArts CI。

数据库：MySQL 8.0、Navicat 15 及以上。

录屏：使用腾讯会议或者其他会议软件进行录制。

文档工具：Microsoft Office 2019 及以上、WPS 2022 及以上。

【实验步骤】

阶段 1：迭代规划

本阶段目标是基于迭代 2 的回顾改进项与剩余高优先级需求，制定本次迭代的冲刺计划。核心任务是落实“持续改进”，将流程优化措施转化为具体任务，并筛选能体现架构演进和业务价值的增量功能。

1. 召开冲刺计划会议

操作指南：

1. 复盘改进项（首要任务）：会议开始时，团队首先逐项回顾迭代 2 回顾会议中制定的“改进计划表”。确认每一项改进措施的完成情况，对于未彻底落地的措施，分析原因并决定是否纳入本次迭代继续落实。
2. 梳理架构与业务需求：结合迭代 2 已实现的业务功能，明确本次迭代需要开发的增量功能。重点考虑功能的模块化设计，确保新功能与现有代码的融合性，并能为系统带来可衡量的价值提升。
3. 筛选高价值需求：对剩余的产品待办列表进行优先级排序。优先选择那些既能交付用户价值，又能为团队提供技术改进实践（如应用设计模式、编写单元测试）的需求。
4. 确定冲刺目标：基于以上分析，明确本次迭代的核心目标。目标应具体，例如：“完成 XX 模块开发，落地代码审查检查清单，将核心函数单元测试覆盖率提升至 30%”。

避坑指南：

- 避免改进与开发脱节：改进措施必须是具体的、可纳入待办列表的任务。例如，“提高代码质量”是空话，“为 XX 模块编写 10 个核心单元测试用例”才是可执行的任务。
- 技巧：可以将重要的改进措施作为独立的“技术故事”加入待办列表，并赋予其与业务功能同等的优先级，确保资源投入。

验收标准：

团队对迭代 2 的改进项落地情况清晰，并共同确定了本次迭代要完成的增量功能列表和明确的冲刺目标。

2. 拆分冲刺待办事项

操作指南：

1. 任务拆解：将本次迭代计划开发的每个用户故事/增量功能，拆分为具体的开发任务。此次拆分的粒度应比迭代 1 更细，需明确包含设计、开发、测试、审查等环节。示例（用户故事：优化订单查询性能）：

- T001: 后端 API 性能分析, 识别瓶颈 (负责人: 张三)。
 - T002: 设计缓存方案, 绘制流程图 (负责人: 李四)。
 - T003: 实现缓存逻辑, 编写核心代码 (负责人: 李四)。
 - T004: 为缓存组件编写单元测试 (负责人: 王五)。
 - T005: 前端配合改造, 集成新接口 (负责人: 赵六)。
 - T006: 性能测试与对比报告 (负责人: 测试员)。
2. 融入改进任务: 将需要落地的改进措施同样拆解为具体任务。示例 (改进项: 落实代码审查):
- T007: 制定代码审查检查清单 (负责人: 技术负责人)。
 - T008: 组织团队进行代码审查规范培训 (负责人: 敏捷教练)。
3. 明确依赖与验收标准: 识别任务间的依赖关系 (如 T001 必须在 T002 之前完成), 并为每个任务定义清晰的完成标准 (即“完成定义”)。

验收标准:

产出详细的冲刺待办事项列表, 其中既包含业务功能开发任务, 也包含了明确的流程改进任务, 且每个任务都有负责人和验收标准。

3. 制定冲刺待办列表与跟踪表

操作指南:

1. 创建冲刺待办列表 (基线): 将会议确定的迭代目标、用户故事清单、故事点估算、以及分解后的所有任务整理成一份正式的冲刺计划文档 (DOCX 格式)。此文档作为本次迭代的范围基线。
2. 更新冲刺待办列表跟踪表: 在迭代 2 使用的在线跟踪表基础上, 创建本次迭代的跟踪表。本次迭代的跟踪表字段应更完善, 可增加“关联的改进项 ID”、“代码审查结果”、“单元测试通过率”等字段, 以强化质量跟踪。
3. 初始化 CodeArts 看板: 在 CodeArts 平台中规划新的迭代 (如 Iteration 3), 将本次迭代的所有 Story 和 Task 级工作项拖入该迭代, 生成新的看板视图。

避坑指南:

- 忽略改进项跟踪: 改进任务和开发任务必须使用同一套跟踪机制, 同等重要, 避免改进项因“不重要”而被无限期推迟。

验收标准:

冲刺计划基线文档、在线跟踪表、CodeArts 看板三者内容同步且初始化完毕, 团队对本次迭代的工作量、目标和重点改进方向达成共识。

阶段 2：迭代冲刺

本阶段目标是执行迭代 2 的冲刺计划，核心聚焦于落地迭代 1 的改进措施并完成增量功能的稳定交付。重点是将以上一迭代回顾中确定的流程优化（如代码规范、站会效率）转化为团队的实际行动，并在开发中体现初步的质量意识。

1. 迭代冲刺执行与每日站会

操作指南：

1. 应用改进后的站会流程：每日站会严格遵循迭代 1 回顾中优化的新规则（如严格 15 分钟时间盒、指定计时员、会前收集障碍等）。
2. 重点关注改进项落地：每位成员发言时，需额外说明所负责的“改进措施”的进展情况（如“昨日已按新规范完成了代码提交”、“今日计划应用代码审查检查清单”）。
3. 障碍升级机制：对阻碍改进措施落地的障碍（如新代码规范理解不清、审查流程不熟悉）视为高优先级问题，立即记录并指定敏捷教练或技术负责人当日协助解决。

避坑指南：

- 退回旧习惯：迭代初期团队容易不自觉地退回旧有工作模式。主持人需主动引导，确保新的站会规则被严格遵守。
- 技巧：可将改进措施（如“应用代码审查清单”）作为一项每日站会必须汇报的“任务”，强化其重要性。

验收标准：

每日站会记录体现新流程的执行情况，改进措施相关的障碍得到及时跟踪和解决。

2. 代码开发与初步质量内建

操作指南：

1. 践行代码规范：开发人员在编写代码时，需有意识地遵循迭代 1 制定的或本迭代优化后的代码规范（如命名、注释）。可在 IDE 中配置代码模板辅助检查。
2. 引入代码审查（关键改进）：功能开发完成后，必须发起合并请求（Pull Request）。审查者依据迭代 2 规划中制定的简易代码审查清单（可包含：命名规范、简单逻辑错误、是否包含基础注释、是否引入明显安全风险等要点）进行审查。
3. 编写基础单元测试（关键改进）：针对本次迭代开发的最核心、最关键的几个函数/方法，尝试编写简单的单元测试。目标是建立测试意识，不追求高覆盖率，重点验证核心逻辑正确即可。
4. 规范的代码提交：每次提交信息需关联任务 ID，并可备注改进点，如 [T002] feat: 实现订单查询功能 [遵循新命名规范]。

避坑指南：

- 审查走过场：避免审查意见空泛。要求审查者至少提出一条具体建议或疑问，即使很小，旨在培养审查习惯。
- 技巧：单元测试可从工具自动生成的基础测试用例开始，重点在于理解测试的价值和基本写法，降低初始门槛。

验收标准：

所有新增代码都经过代码审查流程（有记录），核心功能配备了基础单元测试，代码提交信息规范。

3. 功能开发与集成测试**操作指南：**

1. 增量开发：基于迭代 1 的 MVP 成果，开发新增功能。开发时注意与现有功能的衔接，确保数据一致性。
2. 集成测试：测试人员重点关注新功能与迭代 1 已有功能的集成联调测试，确保系统整体流程依然通畅。
3. 缺陷修复：对测试发现的缺陷，严格记录并关联到对应任务，修复后需经过代码审查和回归测试才能关闭。
4. 进度跟踪：每日更新 CodeArts 看板，燃尽图应能反映改进措施任务（如代码审查、单元测试）的工作量消耗。

避坑指南：

- 忽略回归测试：新增功能可能对旧功能产生意外影响。必须进行基本的回归测试，防止“修复一个 bug，引入两个新 bug”。

验收标准：

新增功能开发完成并与原有系统集成良好，无明显集成缺陷，看板进度反映真实情况。

4. 改进措施落地验证**操作指南：**

1. 中期检查：在迭代中期，可快速检查一次改进措施的落地情况（如：代码审查是否每次都都在做？单元测试写了多少？）。
2. 问题微调：如果发现某项改进措施执行困难，可在站会上讨论，进行小范围调整（如简化审查清单），确保其可行性，但核心目标（落地改进）不变。

验收标准：

迭代结束时，迭代 1 回顾会议计划中针对本迭代的改进措施，基本得到落实，并有过程记录可查。

阶段 3：迭代冲刺评审

本阶段目标是召开一次更深入、更全面的冲刺评审会议。评审重点从迭代 1 的“功能可用”提升到“质量达标”，需从功能完整性、代码规范性、设计合理性和流程优化效果四个维度，验证迭代 2 的增量成果与改进措施是否真正落地。

1. 评审准备

操作指南：

1. 材料整合：整理本次迭代的关键过程材料，包括：冲刺待办列表最终状态、缺陷跟踪记录表、代码审查记录摘要、以及迭代 2 改进措施落地情况说明（如：代码规范检查清单使用情况、单元测试编写情况等）。
2. 环境与数据准备：确保演示环境稳定，并准备能充分展示增量功能和改进效果的测试数据。例如，准备两份代码截图，对比迭代 1 和迭代 2 对同一功能的实现，以直观展示代码规范的改进。
3. 明确评审议程：提前将评审议程发给所有参会人员（全体团队成员），明确本次评审将采用分维度评审法，并规定每个维度的演示和反馈时间。

避坑指南：

- 准备不足：避免只准备功能演示。务必将反映代码质量、设计改进的过程性材料（如审查记录、测试报告）准备好，否则评审会退化为功能展示会。

验收标准：

评审材料齐全，演示环境可靠，团队对多维度评审的议程和规则清晰。

2. 召开冲刺评审会议

操作指南：

1. 成果演示（20 分钟）：由开发负责人进行演示，分两部分：
 - 改进效果对比演示（5 分钟）：先快速对比展示迭代 2 在流程、代码质量上较迭代 1 的改进点（如：播放迭代 1 站会超时的片段 vs 迭代 2 站会准时的记录；展示迭代 1 某段代码与迭代 2 重构后的代码对比）。
 - 增量功能演示（15 分钟）：按业务流程演示本次迭代开发的增量功能，重点展示新功能与迭代 1 已有功能的衔接顺畅度和数据一致性。
2. 多维度反馈收集（25 分钟）：引导团队成员（模拟用户、测试、运维等角色）从以下四个维度提出反馈，记录员按维度分类记录：

- 功能与流程维度：增量功能是否满足需求？与现有功能整合是否顺畅？有无逻辑断点？
 - 代码与规范维度：新增代码是否遵循了既定规范？（可现场抽查少量代码）代码审查是否发现了实质性问题？
 - 设计与技术维度：模块拆分是否更清晰？是否存在重复代码？关键函数是否有单元测试？
 - 流程与协作维度：新的代码审查流程、站会规则是否有效？协作效率是否较迭代 1 有提升？
3. 反馈处理与结论：对收集的反馈进行分类排序。致命问题（如功能不可用、数据错误）必须在本迭代内修复；严重问题（如主要功能体验缺陷）需在项目收尾前修复；优化类建议纳入产品待办列表。最终形成评审结论（通过/有条件通过/不通过）。

避坑指南：

- 维度混淆：主持人需引导评审者按维度发表意见，避免不同维度的反馈混杂在一起，导致问题失焦。例如，当有人提到“这个功能不好用”时，应追问“是操作流程（功能/体验维度）的问题，还是界面设计（设计维度）的问题？”

验收标准：

评审会议按议程顺利进行，反馈按维度被清晰记录，并形成了明确的处理意见和评审结论。

3. 输出评审成果

操作指南：

1. 编写评审会议纪要：纪要需详细记录各维度的反馈内容、处理意见（修复、优化、纳入后续），并明确评审结论。
2. 更新用户变更记录表：如评审中产生了新的、被认可的需求变更，需正式记录到用户变更记录表中，并评估对后续迭代的影响。
3. 生成评审反馈表：将反馈内容整理成结构化表格，包含“反馈维度、具体描述、提出人、优先级、处理方式、责任人”等字段，便于跟踪。

验收标准：

产出内容详实的多维度评审会议纪要、更新后的用户变更记录表（如有）和结构化的评审反馈表。

阶段 4：冲刺演示与交付

本阶段目标是对迭代 2 的成果进行标准化封装与交付，核心任务是全面展示增量功能、突出改进效果、实现平滑升级。交付物需体现迭代 2 在功能完备性、代码质量和团队协作上的进阶。

1. 录制冲刺成果演示录屏

操作指南：

1. 策划录屏脚本：录屏内容需超越迭代 1 的 MVP 演示，重点展示增量功能与改进效果。脚本结构建议：
 - 开场（1 分钟）：简要回顾迭代 1 成果，明确本次迭代的核心目标（如：实现 XX 模块、落地代码审查）。
 - 改进效果展示（2-3 分钟）：对比展示迭代 2 的改进点，例如：快速对比迭代 1 与迭代 2 的代码片段（命名规范、注释改进）、展示代码审查记录、演示新增的单元测试运行情况。
 - 增量功能演示（3-4 分钟）：完整演示本次迭代开发的增量功能，重点展示新功能与迭代 1 已有功能的无缝衔接和业务流程闭环。
 - 总结（1 分钟）：总结迭代 2 在功能、质量、流程上的全面提升。
2. 录制与后期：确保画面清晰、语音清楚。可在演示改进点时使用标注工具高亮关键变化。严格控制总时长在 5-8 分钟内。

避坑指南：

- 沦为功能罗列：避免简单演示新功能。必须突出“对比”和“演进”，清晰传达迭代 2 的进步所在。

验收标准：

产出时长 5-8 分钟的高清录屏，内容必须包含改进效果对比和增量功能演示，清晰体现迭代 2 的价值。

2. 完善轻量化用户手册

操作指南：

1. 更新手册结构：在迭代 1 的用户手册 V1.0 基础上，升级至 V2.0。新增章节应包括：
 - 增量功能详解：详细说明本次迭代新增功能的操作流程。
 - 系统优化说明：简要说明本次迭代在性能、体验上的优化点（若有）。
2. 内容整合：将新增内容与原有内容整合，确保整个核心业务流的操作指南是连贯、完整的，而不是割裂的。

验收标准：

产出轻量化用户手册 V2.0，内容覆盖迭代 1 和迭代 2 的全部核心功能，逻辑连贯，图文并茂。

3. 程序源码整理与系统升级部署

操作指南：

1. 源码整理：

- 确保迭代 2 的所有代码均已合并至 dev 分支，且通过了代码审查。
- 更新 README.md 文件，补充迭代 2 的模块说明、部署升级步骤以及关键的改进点（如新增的代码规范）。

2. 系统升级部署：

- 部署策略采用平滑升级。确保在部署迭代 2 新功能时，迭代 1 的已有功能不受影响。
- 部署前需备份迭代 1 的系统数据与代码。
- 部署后，需立即验证迭代 1 功能的可用性，再验证迭代 2 的新功能。

3. 制定回滚预案：明确部署失败的应急回滚步骤，并确保预案可执行。

避坑指南：

- 忽视回归测试：升级部署后，必须对系统进行全面回归测试，确保旧功能完好，避免“升级即故障”。

验收标准：

源码仓库整洁，README.md 更新完整；系统升级成功，新旧功能均正常运行；回滚预案已制定并验证。

4. 完成交付确认

操作指南：

1. 整理交付物清单：汇总迭代 2 的所有交付物，包括：演示录屏、程序源码、用户手册 V2.0、各类过程文档（评审纪要、审查记录等）。
2. 签订交付确认单：交付确认单需明确本次迭代的交付范围（增量功能清单）、验收标准（功能、质量、改进措施落地情况）和版本号（V2.0）。

验收标准：

交付物齐全，交付确认单签订完毕，标志迭代 2 价值交付正式完成。

阶段 5：冲刺回顾

本阶段目标是团队对第二次迭代的全过程进行深入复盘，核心聚焦于评估改进措施落地效果、分析增量开发中的问题、总结协作模式优化经验，并为后续项目工作提供可复用的实践指导。

1. 召开迭代回顾会议

操作指南：

1. 数据准备：会议前收集并公示本次迭代的关键数据，为复盘提供事实依据：
 - 改进措施落地数据：代码审查执行次数、单元测试覆盖率变化、站会平均时长、缺陷数量趋势等。
 - 过程数据：任务按时完成率、燃尽图与实际进度偏差、代码提交频率与质量。
 - 团队反馈：会前匿名收集成员对本次迭代协作流程的简短反馈。
2. 结构化复盘（采用“改进项复盘”模板）：
 - 改进措施效果评估（20 分钟）：逐项核对迭代 1 回顾中制定的“改进计划表”。用数据说话，评估每项措施是否真正落地、效果如何、为何未能完全落地。
 - 增量开发问题分析（15 分钟）：梳理在本次增量功能开发、集成测试中出现的新问题，如模块接口不清晰、数据流转错误、环境配置差异等。
 - 协作流程优化点总结（10 分钟）：总结本次迭代中运行良好的新协作模式（如更高效的代码审查流程），以及仍存在的协作瓶颈。
 - 根源分析与改进建议（15 分钟）：针对核心问题，深入分析根源，并制定 1-2 项最优先、可执行的改进建议，明确纳入后续工作。

避坑指南：

- 避免空泛讨论：主持人需引导讨论基于具体事例和数据，避免“感觉效率不高”等模糊表述，而要问“哪个环节、什么任务、数据如何体现”。
- 技巧：对未达标的改进项，重点分析是“措施不可行”、“执行不到位”还是“资源不支持”，从而制定更切实的下一步行动。

验收标准：

回顾会议有数据支撑，讨论深入具体，对改进项落地情况有清晰评估，并产出了新的优先级改进项。

2. 输出回顾会议成果

操作指南：

1. 编写回顾会议纪要：纪要需结构化呈现：
 - 改进措施落地情况统计（含数据支撑）。
 - 本次迭代的核心问题汇总（按技术、协作、流程分类）。
 - 根源分析结论。
 - 新的改进计划表（针对本次迭代发现的问题）。
2. 更新团队知识库：将本次迭代中形成的有效实践（如高效的代码审查清单、好用的测试方法）和遇到的典型问题及其解决方案进行沉淀，整理成简明的文档，纳入团队知识库。

验收标准：

产出内容详实、有数据支撑的迭代回顾会议纪要和一份新的、具体的改进计划表。团队知识库得到更新。

3. 经验沉淀与收尾**操作指南：**

1. 经验分享：在团队内部简要分享本次迭代的最大收获和教训，确保经验在团队内部分享。
2. 确认收尾：团队负责人确认本次迭代的所有工作（包括回顾会议产生的行动项）均已分配并明确责任人，迭代正式结束。

验收标准：

团队对本次迭代的得失形成共识，新的改进措施已纳入规划，迭代闭环。

【实验报告】

按附录 1《实验报告参考模版》要求编写，需包含以下核心内容：

实验环境：详细记录硬件配置、软件版本、工具使用情况。

实验过程：按“迭代规划-迭代冲刺-冲刺评审-演示交付-迭代回顾”五个阶段，详细描述改进项落地过程、模块化设计思路、代码审查流程，附核心截图。

实验结果及分析：

成果总结：列出迭代 2 交付物清单，说明增量功能与模块化设计实现情况。

数据对比：对比迭代 1 与迭代 2 的关键数据（缺陷数量、站会时长、代码审查通过率、改进项落地率）。

问题分析：分析改进措施落地不彻底、模块化设计不合理、协作流程仍存在的核心问题。。

问题与解决：详细记录实验过程中的技术问题与流程问题，说明排查过程、解决方案及效果。

实验总结：总结迭代 2 的核心收获（改进落地能力、模块化设计意识、代码质量意识），反思质量内建与持续改进的关键点，展望迭代 3 的优化方向。

【实验中的常见问题与解决方案】**1. 问题：代码审查流于形式，评论空洞**

- 解决方案：提供代码审查清单（如命名、注释、逻辑、安全）。要求审查意见必须具体到代码行，并提出改进建议。禁止使用“Looks good to me”等模糊评语。

2. 问题：回顾会议的改进措施在本迭代未能落地

- 解决方案：将改进项转化为具体的冲刺任务。例如，“提高代码质量”转化为“为 XX 模块编写 10 个单元测试用例”，并像开发任务一样分配故事点和责任人，纳入看板跟踪。

3. 问题：新增功能与旧模块集成时出错

- 解决方案：坚持“契约先行”。开发前明确接口规范（请求/响应格式），并编写轻量级的集成测试用例。在 CI 流水线中自动运行，确保集成稳定性。

【实验或拓展思考题】

1. **效率的悖论** 代码审查、单元测试等质量保障活动无疑会降低当前的开发速度。请论证：在什么情况下，这些“慢功夫”反而会成为项目整体效率的“加速器”？能否从长远视角构建一个“效率模型”来说服追求速度的团队成员？
2. **“最佳实践”的陷阱** 业界推崇的“最佳实践”（如微服务、DDD）在什么情况下可能不适合你们的项目？当团队内部对是否引入一项复杂的最佳实践产生分歧时，你更倾向于“先简后繁”还是“一步到位”？请阐述你的哲学。
3. **反思的极限** 迭代回顾旨在持续改进，但过度反思也可能导致“分析瘫痪”或团队士气低落。你认为一个回顾会议的“健康度”指标应该有哪些？如何避免回顾会变成单纯的“抱怨大会”或“追责大会”？

【评分标准】

落地改进措施，提升代码质量与协作效率

迭代 1 的改进措施是否在本次迭代落地、缺陷数量是否下降、代码规范是否更完善

评分项	权重	评分标准
功能完整性	25%	增量功能符合需求规划；核心业务流程更顺畅，迭代 1 遗留问题全部修复；交付物规范齐全。
代码规范性与质量	30%	代码命名、注释、结构更规范，无冗余逻辑；核心模块缺陷率较迭代 1 下降 $\geq 30\%$ ；Git 分支管理与提交记录更规范。
实验报告质量	20%	详细对比迭代 1 与迭代 2 的改进点（协作流程、代码质量、缺陷数量）；问题与解决记录体现针对性，明确改进措施的落地效果。
创新性	10%	在改进措施落地或代码质量优化上有创新方法，且效果显著。
团队协作	15%	迭代 1 回顾的改进措施 100%落地；协作仪式（站会、评审）效率提升；代码审查记录完整，体现质量管控意识。

【参考教材与实验拓展】

参考教材：

《代码整洁之道》（Robert C. Martin 著）、

《敏捷软件测试实践》（Lisa Crispin 著）、

《持续交付》（Jez Humble 著）。

实验拓展：调研接口自动化测试工具，尝试将核心接口测试用例固化；学习代码重构基础技巧，优化现有模块耦合问题；探索 AI 辅助代码规范检查工具的使用，提升审查效率。

【注意事项】

学术诚信：代码与设计图需为团队原创，引用第三方框架或开源代码需注明来源；代码审查需客观公正，禁止走过场。

质量管控：严格执行代码审查与测试流程，不跳过任何质量环节；禁止为追求进度降低代码规范与设计标准。

团队协作：代码审查、任务跟踪等流程需按约定执行，避免推诿扯皮；技术负责人需主动协调设计分歧，确保团队技术认知一致。

数据记录：详细记录改进项落地情况与质量相关数据，为迭代回顾与评分提供依据。

实验四：迭代 3（成熟实践下的高效价值交付）

【实验目标】

（一）知识目标

掌握“业务价值闭环”的核心逻辑，理解从 MVP 到完整业务流程的演进方法。

熟悉项目全流程沉淀的核心要点，掌握跨迭代复盘与经验提炼的方法。

深化对敏捷成熟实践的整合应用，实现高效协作与高质量交付的有机结合。

（二）能力目标

能够整合前两次迭代成果，完成核心业务场景的全流程闭环开发，实现高价值功能交付。

能够优化迭代效率，整合成熟协作方法，减少冗余环节，实现高效交付。

能够组织全面的冲刺评审，从业务价值角度验证产品效果，收集针对性反馈。

能够开展跨迭代复盘，总结三次迭代的核心经验与教训，形成可复用的实践沉淀。

（三）素质目标

树立“用户价值导向”的产品思维，聚焦业务场景的完整性与实用性，而非单纯功能堆砌。

强化“成果沉淀”的职业习惯，重视项目过程中的经验积累与知识传承。

提升团队成熟协作的素养，形成高效、稳定、可复制的协作模式与开发流程。

【实验内容】

任务 1：完成第三次迭代规划

任务描述

基于迭代 2 的回顾改进项与剩余高优先级需求，聚焦核心业务流程闭环，明确用户体验优化方向，完成迭代 3 的冲刺计划，确保交付高价值、易用性强的完整功能。

任务知识

高价值需求筛选方法（MoSCoW 法）、业务流程闭环设计逻辑、用户体验轻量优化原则。

任务成果

冲刺计划会议纪要、冲刺待办列表（待办基线）、冲刺待办列表跟踪表（在线文档）、冲刺目标文档、用户体验优化清单。

思路简述

以“价值最大化、体验最优化”为核心，采用 MoSCoW 方法筛选剩余需求，优先交付“必须有”和“应该有”的高价值功能，确保核心业务流程闭环；用户体验优化聚焦高频操作流程，列出具体优化点，避免过度设计。

💡 **教学提示：**最终迭代的规划应聚焦于“交付价值最大化”和“产品体验闭环”。优先处理前序迭代积累的高优先级优化项和关键缺陷，确保核心业务流程顺畅、稳定，达到可交付状态。

任务 2：完成第三次迭代冲刺

任务描述

执行冲刺计划，整合前两次迭代的成熟协作实践，开发高价值功能实现业务闭环；基于用户反馈开展轻量 UX 优化；强化核心流程测试覆盖，确保功能、体验双达标。

任务知识

业务流程整合方法、用户体验轻量优化技巧、核心流程测试设计、高效协作实践整合。

任务成果

每日站会记录、障碍清单、燃尽图/看板进度视图、测试报告、缺陷跟踪记录表。

思路简述

以“高效交付+体验优化”为核心，复用前两次迭代的成熟协作流程；开发过程中重点关注业务流程的连贯性，确保各模块衔接顺畅；UX 优化采用“小步快跑”模式，针对高频操作场景进行局部调整，通过团队内部测试验证优化效果；核心流程测试覆盖全链路，降低业务风险。

💡 **教学提示：**本阶段是“收口”阶段，重在集成、测试和优化。开发应侧重于系统整体稳定性、性能和数据一致性，而非大量新功能。所有代码提交需更加谨慎，充分测试，避免引入新的风险。

任务 3：完成第三次冲刺评审

任务描述

组织全面的冲刺评审会议，邀请“模拟客户”（其他团队成员）参与，从功能完整性、业务闭环性、用户体验、协作效率四个维度进行全方位评估；收集多视角反馈，验证产品是否达到交付标准。

任务知识

多维度评审流程设计、用户体验评估方法、客户反馈收集与转化技巧。

任务成果

冲刺评审会议纪要、用户变更记录表、评审反馈表。

思路简述

评审标准达到“交付级”要求，重点验证核心业务流程是否闭环、用户操作是否便捷、功能是否满足实际业务需求；通过简单的可用性测试（如邀请其他同学完成指定任务）收集体验反馈；对反馈分类处理，致命问题必须在本次迭代内修复，优化类反馈视情况纳入项目收尾阶段。

 **教学提示：**本次评审应模拟“交付验收”。评审者需从最终用户视角，全面检验功能完整性、流程顺畅性、界面友好度和系统稳定性。应将评审标准提升到“可上线”水平，为最终答辩做准备。

任务 4：完成第三次冲刺演示与交付

任务描述

录制 10-15 分钟的冲刺成果演示录屏，全面展示核心业务闭环流程、高价值功能、UX 优化亮点；完善轻量化用户手册（V3.0），补充高级操作指南与体验优化说明；完成系统最终部署（稳定演示环境），签订交付确认单，实现项目核心价值的完整交付。

任务知识

交付级演示技巧、用户手册编写规范（含体验优化说明）、稳定环境部署规范。

任务成果

冲刺成果演示记录、冲刺成果演示录屏（10-15 分钟）、程序源码、轻量化用户手册（V3.0）、交付确认单、部署说明文档。

思路简述

演示录屏按“业务全流程”逻辑展开，清晰展示从功能入口到最终结果的完整链路，突出 UX 优化前后的差异；用户手册补充高价值功能操作指南与体验优化说明，提升实用性；系统部署选择稳定的演示环境，确保演示效果流畅，记录部署步骤与注意事项。

💡 **教学提示：**最终演示是对项目成果的全面展示。录屏脚本应精心设计，清晰阐述项目价值、演示核心业务闭环、突出技术亮点和团队协作成果。部署文档的准确性和完整性是本环节交付的关键。

任务 5：完成第三次迭代回顾

任务描述

召开项目级迭代回顾会议，全面总结三次迭代的敏捷实践成果、技术沉淀、团队协作经验与待改进问题；聚焦“项目收尾准备”，制定收尾阶段的工作规划与风险应对预案，为实验五的项目复盘与归档奠定基础。

任务知识

项目级复盘方法、敏捷实践沉淀技巧、收尾阶段风险管控策略。

任务成果

冲刺回顾会议纪要、改进计划表。

思路简述

回顾会议采用“成果沉淀+问题复盘+收尾规划”三维模板，总结三次迭代中成熟的实践方法、技术难点的解决经验、团队协作的高效模式；梳理项目收尾阶段可能面临的风险，制定应对措施；项目收尾工作规划表明确复盘、答辩、归档的时间节点与责任人；必要时编写《敏捷实践总结文档》，固化团队在项目中的有效经验。

💡 **教学提示：**本次回顾是项目级的复盘。需从技术、流程、协作三个维度，全面总结从启动到收尾的全过程经验教训。产出物（如技术规范、协作流程）应力求可复用，为团队未来项目积累资产。

【实验要求】

（一）交付物

任务一：冲刺计划会议纪要、冲刺待办列表、冲刺待办列表跟踪表（必要）、冲刺目标文档。

任务二：每日站会记录（必要）、障碍清单、燃尽图/看板进度视图（截图）、测试报告。

任务三：冲刺评审会议纪要、用户变更记录表、评审反馈表。

任务四：冲刺成果演示记录、演示录屏（必要，MP4 格式，10-15 分钟）、程序源码（必要，压缩包，含完整项目代码、自动化测试代码、README.md 详细说明）、轻量化用户手册、交付确认单。

任务五：冲刺回顾会议纪要、改进计划表、项目收尾工作规划表、敏捷实践总结文档（非必要）。

（二）质量标准

功能完整性：高价值功能全部实现，核心业务流程全闭环，无功能缺失或逻辑断点。

用户体验：操作流程简洁（高频任务操作步骤 ≤ 3 步），界面布局合理，操作反馈及时（ $\leq 1s$ ），无明显易用性缺陷。

协作高效：复用前两次迭代的成熟协作流程，每日站会高效，代码审查及时，任务状态更新同步。

交付规范：所有交付物格式统一、内容详实，源码可编译运行，部署文档清晰可复现，符合企业级基础交付标准。

【实验环境】

（一）硬件环境

处理器：Intel Core i5 及以上。

内存：8GB 及以上。

硬盘：剩余空间 50GB 及以上。

网络：稳定的互联网连接（用于访问远程代码仓库、项目管理平台、CI 工具）。

（二）软件环境

操作系统：Windows 10/11 64 位或 macOS 12 及以上。

开发工具：IntelliJ IDEA 2022 及以上、Visual Studio Code（可选）。

版本控制工具：Git、Git GUI（如 Tortoise，可选）。

项目管理平台：华为 CodeArts（含需求管理、甘特图、看板）。

持续集成工具：华为 CodeArts CI。

数据库：MySQL 8.0、Navicat 15 及以上。

录屏：使用腾讯会议或者其他会议软件进行录制。

文档工具：Microsoft Office 2019 及以上、WPS 2022 及以上。

【实验步骤】

阶段 1：迭代规划

本阶段目标是为最终迭代制定一份聚焦于交付最大价值和最优体验的冲刺计划。核心任务是筛选出能实现核心业务闭环的“必须有”和“应该有”的高优先级需求，并规划轻量的用户体验优化，确保项目完美收尾。

1. 召开冲刺计划会议

操作指南：

- 需求价值评估：采用 MoSCoW 方法 对剩余的产品待办列表中的所有需求进行快速分类：
 - 必须有：没有这些功能，核心业务闭环无法实现，系统达不到交付标准。
 - 应该有：对核心业务流程有重要增强，能显著提升产品价值和用户体验，应尽力实现。
 - 可以有：锦上添花的功能，如果时间允许则实现。
 - 这次不会有：明确本次迭代不做，纳入未来可能的功能清单。
- 确定业务闭环范围：团队需明确回答：“经过本次迭代后，系统的哪一个或哪几个核心业务流程可以实现从开始到结束的完整闭环？” 以此作为本次迭代的核心交付目标。
- 规划用户体验优化：梳理高频操作场景，列出 1-3 个最影响用户操作效率或满意度的具体优化点（如：简化某个关键操作的点击步骤、优化界面加载速度、增加成功操作提示等），形成 用户体验优化清单。优化应遵循“最小化修改”原则，确保可控。

避坑指南：

- 范围蔓延：最终迭代极易产生“再加一个小功能”的想法。必须严格遵循 MoSCoW 分类结果，优先保障“必须有”的功能，严格控制范围。
- 技巧：对于有争议的需求，可进行快速的价值与实现成本评估，优先选择价值高、成本低的需求。

验收标准：

团队使用 MoSCoW 方法对剩余需求完成分类，明确了本次迭代要实现的核心业务闭环，并制定了具体的用户体验优化清单。

2. 制定冲刺目标与待办列表

操作指南：

1. 定义冲刺目标：冲刺目标应清晰反映本次迭代的价值。格式为：“完成【核心业务 A】和【核心业务 B】的全流程闭环，实现高价值交付，并通过【具体优化点】提升关键操作体验。”
2. 任务拆分与估算：将筛选出的高优先级用户故事拆分为开发任务。本次迭代的任务拆分应更为精细，需明确包含：
 - 业务功能开发任务：实现新功能的代码开发。
 - 集成与测试任务：确保新功能与已有功能无缝衔接的联调测试、业务流程贯通测试。
 - 用户体验优化任务：针对优化清单的具体修改任务。
 - 技术债务清理任务（可选）：如果时间允许，可纳入 1-2 项最关键的技术债务清理任务。
3. 故事点估算：对任务进行故事点估算。由于是最后一次迭代，估算应更为谨慎，为集成测试和意外问题预留充足的缓冲时间。

验收标准：

产出明确的冲刺目标文档和一份详细的、包含业务功能、集成测试和体验优化任务的冲刺待办列表。

3. 风险评估与预案制定

操作指南：

1. 识别收尾风险：重点识别最终迭代的特有风险：
 - 集成风险：新功能与大量已有功能集成时出现不可预见的冲突。
 - 时间风险：对项目收尾工作（如系统部署、文档整理）的工作量预估不足。
 - 质量风险：为赶进度而降低测试标准，导致交付版本不稳定。
2. 制定应对预案：针对上述风险，制定具体预案：
 - 为集成测试预留专门的时间盒。
 - 明确收尾阶段各项工作的具体时间节点和负责人。
 - 约定绝不降低质量门槛的原则，若功能无法按时达标，则将其降级为“可以有”或“这次不会有”。

验收标准：

识别出最终迭代的关键风险，并制定了可行的应对预案，更新至风险管控文档。

4. 输出规划成果

操作指南：

生成规划文档：整理输出冲刺计划会议纪要、冲刺待办列表（基线）、冲刺目标文档。

更新在线工具：在 CodeArts 等项目管理平台中，将本次迭代的计划（用户故事、任务）录入，并生成新的迭代看板和甘特图。

验收标准：

所有规划文档齐全，项目管理平台中的迭代计划已设置完毕，团队对最终迭代的目标、范围和计划清晰无误。

阶段 2：迭代冲刺

本阶段目标是执行最终迭代的冲刺计划，核心聚焦于实现核心业务闭环、确保系统稳定性、优化用户体验。本次冲刺强调“质量优先”，通过严格的代码审查、全面的集成测试和用户体经验证，确保交付成果达到可发布标准。

1. 迭代冲刺执行与每日站会

操作指南：

1. 冲刺启动：在 CodeArts 看板中启动迭代 3，所有任务状态初始化为“待处理”，团队按照计划开始冲刺。
2. 高效率站会：每日站会保持高效（≤15 分钟），重点关注：
 - 集成风险：重点关注新功能与现有功能的集成进度和暴露的问题。
 - 质量指标：同步代码审查通过率、测试用例通过率、缺陷修复进度等质量指标。
 - 体验优化进度：跟踪用户体验优化任务的完成情况。
3. 障碍升级：任何阻碍业务闭环或影响系统稳定性的问题立即升级，确保当天解决。

避坑指南：

- 忽视集成问题：最终迭代最容易出现集成问题，必须每天同步集成状态，避免后期集中爆发。
- 技巧：站会中使用“红/黄/绿”灯标识任务风险，快速识别需要关注的任务。

验收标准：

每日站会高效聚焦集成和质量风险，障碍及时解决，看板状态实时更新。

2. 代码开发与质量保障

操作指南：

1. 业务闭环开发：按照业务流程顺序开发新功能，确保每个环节的数据连贯性和逻辑一致性。

2. 严格代码审查：所有代码必须通过双重审查：

- 架构审查：关注代码是否符合设计规范，是否引入不必要的依赖。
- 业务逻辑审查：确保业务规则实现正确，边界情况处理完善。

3. 用户体验优化实施：

- 对每个优化点进行 A/B 测试或团队内部评审，确保优化确实提升体验。
 - 优化后立即更新用户手册相应章节。
- ## 4. 自动化测试覆盖：为核心业务流程编写端到端的自动化测试脚本，确保业务闭环的稳定性。

避坑指南：

- 降低质量标准：临近截止日期时最容易忽视质量，必须坚持代码审查和测试标准。
- 技巧：采用“小批量提交、频繁集成”的策略，及早发现集成问题。

验收标准：

所有新代码通过严格审查，用户体验优化得到验证，核心业务流程有自动化测试覆盖。

3. 系统集成与全面测试

操作指南：

1. 持续集成：每次代码提交后自动运行：
 - 单元测试套件（确保基础质量）
 - 集成测试套件（验证模块间协作）
 - 端到端测试（验证完整业务流程）
2. 兼容性测试：验证新功能与所有现有功能的兼容性，特别关注数据流转和界面交互。
3. 性能测试：对核心业务链路进行压力测试，确保系统在预期负载下稳定运行。
4. 用户体验测试：邀请其他团队成员（非本项目开发者）进行可用性测试，收集真实反馈。

验收标准：

所有测试用例通过，系统集成稳定，核心业务链路性能达标，用户体验获得积极反馈。

4. 缺陷管理与修复

操作指南：

1. 缺陷分级处理：按照严重程度分类处理缺陷：
 - 致命/严重缺陷：立即修复，阻塞版本发布

- 一般缺陷：本迭代内修复
 - 轻微缺陷：评估后决定是否修复
2. 回归测试：每个缺陷修复后必须执行完整的回归测试，确保不引入新问题。
 3. 缺陷分析：对重复出现的缺陷类型进行根因分析，总结经验教训。

验收标准：

所有致命和严重缺陷已修复，一般缺陷修复率达到 90%以上，有完整的缺陷跟踪记录。

5. 进度跟踪与质量监控**操作指南：**

1. 燃尽图监控：每日更新燃尽图，重点关注剩余工作量与理想曲线的偏差。
2. 质量看板：建立可视化质量看板，实时展示：
 - 代码审查通过率
 - 测试覆盖率趋势
 - 缺陷数量趋势
 - 用户体验优化完成度
3. 风险预警：当质量指标出现异常趋势时立即预警，团队集中讨论解决方案。

验收标准：

燃尽图反映真实进度，质量看板数据完整准确，风险预警机制有效运行。

阶段 3：迭代冲刺评审

本阶段目标是对最终迭代成果进行全面验收评审，核心任务是验证核心业务闭环完整性、评估系统整体质量、确认产品达到交付标准。评审重点从功能实现转向价值交付和产品成熟度。

1. 评审准备**操作指南：**

1. 准备评审材料：
 - 演示数据：准备覆盖核心业务全流程的完整测试数据，确保演示过程流畅自然。
 - 质量报告：整理代码质量报告、测试覆盖率报告、性能测试结果等客观质量数据。
 - 需求跟踪矩阵：建立需求与实现功能的对应关系，确保每个需求都有对应实现。
2. 制定评审议程：
 - 业务价值演示（40%）：展示核心业务全流程实现效果

- 质量评估报告（30%）：呈现各项质量指标数据
 - 用户体验验证（20%）：演示关键体验优化点
 - 交付确认（10%）：形成最终评审结论
3. 邀请评审人员：邀请项目相关方参与评审，确保多维度验证产品价值。

避坑指南：

- 准备不充分：避免仅准备功能演示，要同时准备客观质量数据支撑评审结论。
- 技巧：提前进行内部预演，确保演示流程顺畅，数据准备充分。

验收标准：

评审材料齐全客观，演示流程经过演练，评审议程合理明确。

2. 召开冲刺评审会议**操作指南：****1. 业务价值演示：**

- 按真实业务场景完整演示核心业务流程，重点展示业务闭环的实现效果
- 演示过程中突出展示本次迭代解决的业务痛点
- 展示新功能与已有功能的无缝集成效果

2. 质量评估汇报：

- 呈现代码质量指标（复杂度、重复率等）
- 展示测试覆盖率和通过率数据
- 汇报性能测试结果和系统稳定性数据

3. 用户体验验证：

- 对比展示关键体验优化点的改进效果
- 可用性测试结果汇报
- 用户反馈收集和分析

4. 多维度评审：

- 业务维度：功能完整性、流程顺畅度、价值实现度
- 技术维度：系统稳定性、性能表现、代码质量
- 用户体验维度：操作便捷性、界面友好度、使用效率
- 可维护性维度：文档完整性、系统可扩展性

避坑指南：

- 忽视非功能需求：避免只关注功能实现，要同等重视性能、安全等非功能需求。
- 技巧：采用“情景-任务-结果”的演示模式，让评审者更好地理解产品价值。

验收标准：

评审覆盖多个维度，获得全面反馈，形成客观的评审结论。

3. 问题处理与验收确认

操作指南：

1. 问题分类处理：

- 阻碍性问题：影响系统正常使用的严重问题，必须修复
- 改进性问题：影响用户体验但不影响使用的问题，评估修复优先级
- 优化建议：提升性建议，纳入后续版本考量

2. 验收标准确认：

- 明确系统交付的准出标准
- 确认各项质量指标的达标要求
- 确定问题修复的时间要求

3. 形成评审结论：

- 通过：达到交付标准，可以发布
- 有条件通过：需在约定时间内完成问题修复
- 不通过：需要重新迭代开发

验收标准：

问题处理方案明确，验收标准达成共识，评审结论清晰可执行。

4. 输出评审成果

操作指南：

1. 编写评审报告：

- 记录评审过程和结果
- 附上各项评估数据
- 明确后续行动计划

2. 更新产品状态：

- 根据评审结论更新产品版本状态
- 更新产品待办列表优先级

3. 知识沉淀：

- 总结评审过程中的最佳实践
- 记录遇到的问题和解决方案

验收标准：

评审报告内容完整准确，产品状态更新及时，知识得到有效沉淀。

阶段 4：冲刺演示与交付

本阶段目标是对最终迭代成果进行产品化封装和标准化交付，核心任务是**制作高质量交付物、完成系统部署、形成可复用的项目资产**。交付成果应达到可实际使用的成熟水平。

1. 产品化演示录屏制作

操作指南：

1. 专业脚本设计：

- 采用“问题-解决方案-价值”的叙事结构
- 开场明确产品定位和目标用户（30 秒）
- 核心演示完整业务闭环，突出关键价值点（4-5 分钟）
- 结尾总结产品优势和特色功能（1 分钟）

2. 高标准制作要求：

- 使用专业录屏工具，确保 1080P 画质
- 配备降噪麦克风，保证音频清晰
- 添加字幕和关键操作特写标注
- 总时长严格控制在 6 分钟以内

3. 多版本输出：

- 完整版：面向专业评审的详细演示
- 精简版：面向用户的快速导览

验收标准：

产出画面精美、逻辑清晰的专业演示视频，能充分展现产品价值。

2. 交付物标准化整理

操作指南：

1. 源码交付包：

- 包含完整源代码、依赖库说明
- 提供自动化构建脚本
- 完善的 API 接口文档
- 数据库初始化脚本

2. 技术文档：

- 系统架构设计文档
- 部署运维手册
- 二次开发指南

3. 用户文档：

- 交互式产品使用手册
- 常见问题排查指南
- 版本更新说明

验收标准：

交付物完整规范，达到企业级交付标准。

3. 生产级部署验证

操作指南：

1. 多环境部署：

- 开发环境：最新功能验证
- 测试环境：完整流程验证
- 演示环境：稳定版本部署

2. 部署检查清单：

- 环境依赖检查
- 配置文件校验
- 服务启动验证
- 基础功能冒烟测试

3. 性能验证：

- 并发用户测试
- 数据压力测试
- 长时间运行稳定性测试

验收标准：

系统在多环境中稳定运行，性能指标达标。

4. 交付确认与知识传承**操作指南：**

1. 交付确认流程：

- 制定详细的验收检查清单
- 组织正式的产品演示验收会
- 获取客户/指导教师签字确认

2. 知识传承材料：

- 项目总结报告
- 技术决策文档
- 经验教训总结
- 团队能力矩阵图

3. 资产归档：

- 使用标准化命名规范
- 建立清晰的目录结构
- 设置合理的访问权限

验收标准：

交付流程规范完整，知识资产可传承复用。

5. 最终交付物清单**交付内容：**

- 产品演示视频（完整版+精简版）
- 可运行系统（多环境部署验证）
- 完整源代码库（含提交历史）
- 技术文档套装（架构/部署/API）
- 用户文档套装（手册/FAQ/更迭说明）
- 项目过程资产（会议纪要/评审记录）
- 知识传承材料（总结报告/经验文档）

交付形式：

- 标准化交付包（ZIP 压缩）
- 在线文档库（GitHub Wiki）
- 演示环境访问权限

阶段 5：冲刺回顾

本阶段目标是对整个项目周期（三次迭代）进行全面复盘，核心任务是系统总结项目成果、深入分析经验教训、形成可复用的团队知识资产。通过结构化复盘，为团队持续改进和未来项目提供实践指导。

1. 项目级全面复盘

操作指南：

1. 数据化复盘准备：

- 收集三次迭代的关键数据：需求完成率、缺陷密度、代码质量趋势、团队速率变化
- 整理各次迭代的回顾会议纪要和改进措施落实情况
- 准备项目燃尽图、累积流图等可视化数据

2. 多维度复盘分析：

- 技术维度：架构设计合理性、技术债务积累情况、代码质量演进
- 流程维度：敏捷实践效果、协作效率变化、工具链成熟度
- 团队维度：角色分工合理性、沟通协作效果、个人能力成长
- 管理维度：风险管控效果、需求变更管理、质量保障体系

3. 根因分析：

- 对重复出现的问题进行 5Why 分析
- 识别影响团队效能的根本因素
- 分析成功实践的可复制性

验收标准：

完成数据驱动的全面复盘，识别出关键成功因素和待改进领域。

2. 经验教训提炼

操作指南：

1. 成功经验固化：

- 将验证有效的实践整理成团队规范
- 制作技术决策记录和设计模式文档

- 形成团队专属的敏捷实践指南
- 2. 失败教训转化：
 - 将重大问题和应对措施案例化
 - 制定风险预案库和应对策略
 - 建立团队知识库的问题解决方案索引
- 3. 能力成长评估：
 - 个人技能矩阵更新
 - 团队能力短板识别
 - 制定个人发展计划

验收标准：

形成可操作的经验教训文档，团队能力图谱清晰。

3. 改进措施制定**操作指南：**

1. 短期改进计划：
 - 针对当前项目收尾阶段的具体改进项
 - 明确改进措施、责任人、时间节点
 - 制定验证标准和验收方法
2. 长期改进方向：
 - 团队协作模式的优化方向
 - 技术栈的演进规划
 - 工程能力提升路径
3. 知识传承机制：
 - 建立团队知识库维护规范
 - 制定新成员入职培训材料
 - 设计项目复盘模板和指南

验收标准：

制定具体可行的改进计划，建立知识传承机制。

4. 项目总结与成果固化

操作指南：

1. 项目总结报告：
 - 项目概况与目标达成情况
 - 关键成果与价值分析
 - 经验教训总结
 - 对未来项目的建议
2. 团队知识库建设：
 - 技术文档归档
 - 过程资产整理
 - 最佳实践沉淀
3. 成果展示材料：
 - 项目演示材料
 - 技术分享 PPT
 - 成果总结视频

验收标准：

完成完整的项目总结材料，知识库初具规模。

5. 回顾会议输出物

1. 必需交付物：
 - 项目复盘报告（含数据分析）
 - 经验教训文档
 - 改进行动计划
 - 团队知识库
 - 项目总结报告
2. 可选交付物：
 - 技术分享材料
 - 团队能力评估报告
 - 过程改进建议书

【实验报告】

按附录 1《实验报告参考模版》要求编写，需包含以下核心内容：

实验环境：详细记录硬件配置（开发/测试/演示环境）、软件版本（含开发工具、UX 设计工具、数据库版本）、演示环境部署配置、核心技术栈版本。

实验过程：按“迭代规划-迭代冲刺-冲刺评审-演示交付-迭代回顾”五个阶段，详细描述核心业务闭环开发细节、UX 优化的设计与测试过程、成熟协作实践的整合应用，附核心截图。

实验结果及分析：

成果总结：列出迭代 3 的交付物清单，说明核心业务闭环、高价值功能、UX 优化的具体成果，对比三次迭代的进阶效果。

数据统计：核心业务流程完成时间（优化前后对比）、UX 可用性测试数据（操作时间、错误率、满意度）、缺陷数量（按严重程度分布）、协作效率数据（站会时长、代码审查通过率）。

问题分析：分析业务流程整合、UX 设计、协作实践中存在的核心问题，结合数据说明改进效果与不足。

问题与解决：详细记录实验过程中遇到的复杂技术问题、排查过程、解决方案及效果，每个问题对应具体技术场景与代码片段。

实验总结：总结迭代 3 的核心收获（业务闭环设计能力、UX 优化思维、成熟敏捷实践应用），反思项目全程的经验与教训，展望项目收尾阶段的重点工作。

【实验中的常见问题与解决方案】

1. 问题：系统可维护性下降，“技术债务”累积

- 解决方案：将小规模重构常态化。在开发新功能时，如果接触到的旧代码有优化空间，应适时进行重构。遵守童子军军规：让代码比你来时更整洁。

2. 问题：为了赶进度，降低测试标准

- 解决方案：质量门槛绝不能降低。约定：任何功能必须通过所有关联的单元测试和集成测试才能合并。如果时间紧张，应缩减功能范围，而非牺牲质量。

3. 问题：演示录屏重点不突出，价值表达不清

- 解决方案：采用“问题-解决方案-价值”的叙事结构。开场先点明用户痛点，再演示你的功能如何解决它，最后总结带来的核心价值。严格控制时长在 5 分钟内。

【实验或拓展思考题】

1. **“完成”的幻觉** 你认为一个软件项目在什么情况下才算真正“完成”？是功能全部实现？是测试通过？是用户满意？还是代码完美？请尝试为你自己的项目定义一个多维度的、有层次的“完成”标准。
2. **技术的“保质期”** 假设这个项目需要持续维护和演进三年。以你当前的技术架构和代码质量来看，你认为它在半年、一年、三年后可能会遇到哪些可预见的挑战？你会从现在开始做哪些“防腐”措施？
3. **价值的重估** 回顾整个开发过程，如果现在有机会重来一次，在需求、技术或流程上，你会做哪一项最大胆的改变？为什么？这个“后悔药”揭示了你现在对软件工程怎样的新理解？

【评分标准】

实现业务价值闭环，沉淀项目经验

核心业务流程是否完整闭环、实验报告是否有深度复盘、是否形成可复用的实践经验

评分项	权重	评分标准
功能完整性与价值	30%	核心业务流程全闭环，高价值功能全部实现；系统能够解决实际业务场景问题，体现实用价值；无核心功能缺失
代码规范性与质量	20%	代码质量稳定，无高危漏洞与冗余逻辑；测试覆盖全面，缺陷率维持在低水平；版本管理规范有序。
实验报告质量	25%	包含跨迭代深度复盘，分析三次迭代的演进过程、团队成长、核心经验与教训；数据支撑充分（如缺陷率、协作效率对比）；总结反思有深度（≥400字）。
创新性	10%	在业务价值实现或项目沉淀上有创新，如形成独特的协作模式、功能优化方案或经验总结文档。
团队协作	15%	团队协作高效默契，协作仪式简洁有效；Git提交记录与分工合理，体现成熟团队的协作水平；成果沉淀完整，可为后续项目提供参考。

【参考教材与实验拓展】

参考教材：

《用户体验要素：以用户为中心的产品设计》（Jesse James Garrett 著）、

《业务流程设计与优化》（王玉荣 著）。

实验拓展：

学习业务流程自动化工具（如Flowable），尝试将MES系统中的核心流程（如工单审批）实现自动化流转。

调研用户体验评估工具（如UserTesting），学习专业的可用性测试方法，提升UX优

化的科学性。

学习容器化部署（Docker），将 MES 系统打包为 Docker 镜像，实现跨环境快速部署与一致性保障。

【注意事项】

学术诚信：业务流程设计、UX 优化方案、代码实现需为团队原创，禁止抄袭他人成果；引用第三方技术框架、开源工具需在文档与代码中注明来源，查重率超过 20% 视为抄袭。

环境安全：演示环境部署时，禁止使用真实生产服务器的敏感信息；部署完成后及时清理测试数据，关闭不必要的服务端口，避免安全风险。

团队协作规范：项目收尾阶段需重点做好成果沉淀与文档归档，每位成员需按分工完成相关文档的编写与整理；回顾会议需坦诚交流，聚焦经验沉淀与问题解决，为项目闭环画上圆满句号。

交付物质量：演示录屏需保证音视频清晰、逻辑连贯，完整展示业务闭环与优化亮点；用户手册需内容详实、截图清晰，便于使用者快速上手；源码需确保可编译、可运行，README.md 文档需包含完整的环境配置与部署说明。

实验五：敏捷项目收尾（复盘沉淀与团队收尾）

【实验目标】

（一）知识目标

掌握敏捷项目复盘的核心方法，理解项目收尾的完整流程与关键节点。

熟悉项目演示与答辩的技巧，掌握答辩 PPT 的结构化设计方法。

了解项目归档的规范要求，明确过程性材料与成果性材料的分类管理逻辑。

理解团队解散的正式流程，掌握项目收尾阶段的风险管控与知识沉淀方法。

（二）能力目标

能够组织高效的项目复盘会议，全面梳理项目成果、问题与经验，输出高质量的《项目总结报告》。

能够编制结构清晰、重点突出的答辩 PPT，完成 10-20 分钟的项目完整演示与答辩，清晰回应评审提问。

能够按规范对项目全生命周期的材料进行分类、整理与归档，确保材料完整可追溯。

能够撰写正式的团队解散说明，完成项目收尾的各项闭环工作。

（三）素质目标

培养“复盘沉淀”的职业习惯，能够从项目实践中提炼可复用的经验与改进方向。

强化“责任闭环”意识，确保项目收尾阶段的各项工作（复盘、答辩、归档）规范落地。

提升公开表达与逻辑呈现能力，能够清晰、专业地展示项目成果。

树立“知识管理”理念，重视项目过程性材料的积累与传承，为后续项目提供参考。

【实验内容】

任务 1：完成项目复盘与收尾

任务描述

组织项目复盘会议，全面回顾项目从启动到迭代 3 的全流程，梳理成果、分析问题根源、总结成功经验与失败教训；基于复盘结果编写《项目总结报告》，完成项目收尾阶段的风险排查与闭环。

任务知识

项目复盘方法（4F 法、5Why 分析法）、项目总结报告编写规范、收尾阶段风险排查要点。

任务成果

项目复盘会议纪要、《项目总结报告》（含成功经验、失败教训、改进建议）、收尾阶段风险排查表。

思路简述

以“全面复盘+闭环收尾”为核心，复盘会议聚焦“成果回顾-问题分析-经验提炼-改进落地”四个维度；采用结构化复盘方法深挖问题根源；《项目总结报告》需数据支撑充分，改进建议具体可落地；同步排查收尾阶段风险，确保项目顺利闭环。

💡教学提示：项目复盘是“把经验转化为能力”的关键一步。需基于整个项目周期的数据（如燃尽图、缺陷率、代码记录）进行客观分析，避免凭感觉总结。复盘报告的价值在于提炼出可复用的“模式”和需规避的“陷阱”。

任务 2：完成项目演示与答辩

任务描述

编制项目答辩 PPT，内容涵盖项目背景、需求分析、技术选型、迭代成果、性能优化、团队协作、复盘总结等核心模块；调试演示环境，完成 10-20 分钟的项目完整演示与答辩，回应评审老师与同学的提问。

任务知识

答辩 PPT 结构化设计方法、项目演示技巧（重点突出、逻辑连贯）、答辩问答应对策略。

任务成果

项目答辩 PPT、项目演示录屏（10-20 分钟）、答辩问答记录（非必要）。

思路简述

以“清晰呈现+重点突出”为核心，答辩 PPT 遵循“背景-目标-过程-成果-总结”的结构化逻辑，多用数据、截图、图表替代大段文字；演示录屏完整展示系统核心功能、技术亮点与性能优势，时长严格控制在 10-20 分钟；提前预判答辩可能涉及的问题，做好应对准备。

💡 **教学提示：**答辩 PPT 的逻辑重于形式。建议采用“背景-问题-解决方案-过程-成果-价值-反思”的故事线，清晰呈现项目全貌。答辩时，对技术选型、架构决策的深入理解，比功能演示更能体现工程能力。

任务 3：项目归档与团队解散

任务描述

按规范对项目全生命周期的材料进行分类整理（成果性材料、过程性材料），统一打包提交至教学平台；撰写正式的团队解散说明，明确团队解散的时间、后续材料对接人及知识传承安排，完成团队协作的最终闭环。

任务知识

项目材料分类标准、归档文件命名规范、团队解散说明编写要求。

任务成果

项目归档材料包、团队解散说明。

思路简述

以“分类清晰+完整可追溯”为核心，项目材料按“成果性材料（源码、用户手册、演示录屏）”“过程性材料（会议纪要、测试报告、复盘文档）”“管理性材料（团队章程、开发计划、风险文档）”三类分类；归档文件命名遵循“模块-文件名称-版本号.格式”规范；团队解散说明需正式、规范，明确项目收尾后的责任对接与知识沉淀安排，确保项目无遗留问题。

💡 **教学提示：**归档质量直接决定项目知识的可复用性。所有文档、代码、数据应分类清晰、命名规范、版本明确。团队解散说明是一次正式的流程闭环，标志着项目职责的终结，是职业素养的体现。

【实验要求】

（一）交付物

任务一：项目复盘会议纪要（含会议时间、参与人、复盘流程、核心结论）、《项目总结报告》（必要，需包含项目概况、成果统计、问题分析、经验教训、改进建议 5 个核心部分）、收尾阶段风险排查表（非必要）。

任务二：项目答辩 PPT（必要，PPTX 格式，页数控制在 15-20 页）、项目演示录屏（必

要，MP4 格式，10-20 分钟，画面清晰、逻辑连贯）、答辩问答记录（非必要，记录关键提问与回应）。

任务三：项目归档材料包（必要，压缩包格式，含分类目录文档）、团队解散说明（非必要）、材料提交确认单（需团队负责人与指导教师签字确认，非必要）。

（二）质量标准

复盘质量：复盘会议纪要完整，问题分析深入（需用 5Why 等方法追溯根源），《项目总结报告》数据支撑充分，改进建议具体可落地。

答辩要求：PPT 结构清晰、重点突出，演示录屏完整展示系统核心功能与技术亮点，答辩逻辑连贯，能够准确回应评审提问。

归档规范：材料分类合理、命名统一，包含项目全生命周期的关键材料（无遗漏核心文档、源码、视频），压缩包无损坏，可直接解压使用。

解散说明：格式正式、内容完整，明确项目收尾后的各项对接安排，无遗留责任问题。

【实验环境】

（一）硬件环境

处理器：Intel Core i5 及以上。

内存：8GB 及以上。

硬盘：剩余空间 50GB 及以上。

网络：稳定的互联网连接（用于访问远程代码仓库、项目管理平台、CI 工具）。

（二）软件环境

操作系统：Windows 10/11 64 位或 macOS 12 及以上。

开发工具：IntelliJ IDEA 2022 及以上、Visual Studio Code（可选）。

版本控制工具：Git、Git GUI（如 Tortoise，可选）。

项目管理平台：华为 CodeArts（含需求管理、甘特图、看板）。

持续集成工具：华为 CodeArts CI。

数据库：MySQL 8.0、Navicat 15 及以上。

录屏：使用腾讯会议或者其他会议软件进行录制。

文档工具：Microsoft Office 2019 及以上、WPS 2022 及以上。

【实验步骤】

阶段 1：项目复盘与收尾

本阶段目标是对整个项目生命周期进行全面系统的复盘，核心任务是总结经验教训、固化最佳实践、完成项目收尾的风险管控，形成可传承的项目资产。

1. 项目复盘准备

操作指南：

1. 材料系统化整理：

- 收集项目全周期的文档资料：各次迭代计划、评审记录、回顾纪要、代码提交记录、测试报告等
- 整理关键数据：需求完成率、缺陷趋势、团队速率、代码质量指标等量化数据
- 准备可视化图表：燃尽图、累积流图、代码质量趋势图等

2. 确定复盘方法论：

- 采用 4F 复盘法（事实-感受-发现-未来）
- 结合 5Why 分析法深挖问题根源
- 制定详细的复盘会议议程和时间安排

3. 参会人员准备：

- 要求全体项目成员提前准备个人复盘意见
- 邀请指导教师或领域专家参与提供外部视角
- 明确各角色在复盘会中的职责和发言重点

验收标准：

完成项目资料的系统化整理，形成完整的复盘数据包，制定出详细的复盘会议计划。

2. 召开项目复盘会议

操作指南：

1. 事实回顾（30 分钟）：

- 按时间线回顾项目关键里程碑和决策点
- 用数据展示项目目标的达成情况
- 客观呈现项目过程中的重大事件和转折点

2. 感受分享（20 分钟）：

- 团队成员轮流分享项目过程中的个人感受
- 采用“高峰体验”和“低谷体验”的引导方式
- 营造安全、开放的发言氛围，鼓励坦诚交流

3. 发现提炼（40 分钟）：

- 分析项目成功的关键因素和失败的根本原因
- 识别项目中重复出现的问题模式
- 提炼有效的实践方法和需要改进的环节

4. 未来规划（30 分钟）：

- 制定个人和团队的改进计划
- 建立经验传承机制和知识管理流程
- 明确后续项目的优化方向

验收标准：

完成结构化复盘会议，产出全面的复盘记录，团队成员充分参与并达成共识。

3. 编写项目总结报告

操作指南：

1. 报告结构设计：

- 项目概况与目标达成情况
- 关键成果与价值分析
- 过程数据与指标分析
- 经验教训总结
- 改进建议与未来展望

2. 内容撰写要求：

- 数据支撑：用量化数据说明项目成效
- 案例佐证：用具体案例说明经验教训
- 图表结合：用可视化方式呈现复杂信息
- 客观公正：既展示成绩也不回避问题

3. 报告评审与定稿：

- 团队内部评审确保内容准确完整
- 邀请指导教师审阅提供修改建议
- 根据反馈意见完善报告内容

验收标准：

完成内容详实、结构完整的项目总结报告，获得团队和指导教师的认可。

4. 收尾阶段风险排查**操作指南：****1. 风险识别：**

- 检查项目交付物的完整性和质量
- 验证项目目标的达成情况
- 评估知识传承的充分性
- 排查合同和行政流程的闭环情况

2. 风险应对：

- 制定风险解决的时间计划和责任人
- 建立风险跟踪机制直至完全关闭
- 准备应急预案应对可能出现的问题

3. 验收确认：

- 与相关方确认项目验收标准
- 完成项目成果的正式交付和签收
- 确保所有合同和义务履行完毕

验收标准：

完成全面的风险排查，制定有效的应对措施，确保项目顺利收尾。

5. 输出成果**必需交付物：**

- 项目复盘会议纪要
- 项目总结报告（详细版）
- 收尾阶段风险排查表
- 改进措施行动计划

质量要求：

- 内容完整准确，数据真实可靠
- 分析深入透彻，建议切实可行
- 格式规范统一，表达清晰易懂

阶段 2：项目演示与答辩

本阶段目标是通过专业的演示和答辩，全面展示项目成果和价值，核心任务是制作高质量答辩材料、完成流畅的项目演示、有效回应评审提问，体现团队的专业能力和项目价值。

1. 答辩材料准备

操作指南：

1. 答辩 PPT 设计：

- 采用“问题-解决方案-价值-成果”的叙事逻辑
- 封面包含项目名称、团队信息、答辩日期等基本信息
- 目录页清晰展示汇报结构（建议按背景、方案、实现、成果、总结划分）
- 每页内容精炼，多用图表、架构图、数据图表替代大段文字
- 设计统一的视觉风格，保持版面整洁专业

2. 演示脚本编写：

- 制定详细的演示时间分配计划（总时长控制在 15-20 分钟）
- 为每位演讲者分配明确的演讲段落和时间要求
- 编写演讲词要点，确保关键信息不遗漏
- 标注 PPT 翻页点和演示操作节点

3. 问答准备：

- 预测可能的问题类型：技术实现、设计决策、项目管理、业务价值等
- 准备标准答案要点，确保团队回答口径一致
- 分配答题职责，确保各类问题都有对应负责人回答

验收标准：

完成专业、简洁的答辩 PPT 和详细的演示脚本，团队完成问答预演。

2. 演示环境调试

操作指南：

1. 演示系统部署：

- 准备干净的演示环境，确保系统稳定运行
- 部署最新稳定版本的代码
- 准备完整的测试数据集，覆盖核心业务场景
- 验证所有演示功能正常运行

2. 技术设备检查：

- 测试投影设备、麦克风、翻页笔等演示工具
- 准备备用设备以防突发故障
- 确认网络连接稳定，在线演示内容可正常访问

3. 应急预案准备：

- 准备录屏备份，防止现场演示出现问题
- 关键功能点准备截图备用
- 制定设备故障时的应急演示方案

验收标准：

演示环境稳定可靠，设备运行正常，应急预案完备。

3. 答辩演练与优化

操作指南：

1. 内部演练：

- 组织至少 3 次完整的全流程演练
- 严格计时，确保在规定时间内完成演示
- 模拟评审提问环节，锻炼临场应答能力
- 录制演练过程，用于后续分析和改进

2. 外部评审：

- 邀请其他团队或指导教师进行模拟评审
- 收集改进建议，优化演示内容和方式
- 重点关注演示逻辑的清晰度和技术深度的把握

3. 持续优化：

- 根据演练反馈调整 PPT 内容和演讲节奏
- 优化演示用例，突出项目亮点和价值
- 完善问答准备，提高应答的专业性和准确性

验收标准：

完成多次演练，团队配合默契，演示时间控制精准，问答准备充分。

4. 正式演示与答辩

操作指南：

1. 演示执行：

- 按预定流程进行演示，重点突出项目创新点和价值
- 控制演讲节奏，确保关键信息有效传递
- 演示操作准确流畅，避免技术故障
- 团队配合有序，衔接自然

2. 问答环节：

- 认真聆听问题，确保理解问题要点
- 按预定分工由对应成员回答，其他成员补充
- 回答简洁扼要，重点突出，避免冗长
- 遇到不确定问题坦诚说明，承诺后续补充

3. 过程记录：

- 指定专人记录评审问题和团队回答
- 录制答辩全过程，用于后续复盘
- 收集评审的书面评价和建议

验收标准：

演示流程顺畅，问答环节应对得体，完整展现项目价值和团队能力。

5. 答辩总结与材料归档**操作指南：**

1. 材料整理：

- 整理最终的答辩 PPT 和演示视频
- 汇总评审问题和团队回答记录
- 收集评审的反馈意见和评分结果

2. 经验总结：

- 分析答辩过程中的亮点和不足
- 总结有效的演示和应答策略
- 形成团队答辩经验文档

3. 成果归档：

- 将答辩材料纳入项目交付物
- 更新项目总结报告中的答辩情况

- 将评审反馈纳入团队知识库

验收标准：

完成所有答辩材料的整理和归档，形成完整的答辩总结报告。

阶段 3：项目归档与团队解散

本阶段目标是完成项目的规范化归档和团队的有序解散，核心任务是建立完整的项目资产库、形成可传承的知识体系、实现团队的正式闭环。

1. 项目材料系统化归档

操作指南：

1. 制定归档规范：

- 建立统一的文件命名规范：项目阶段-文件类型-版本号.扩展名
- 设计清晰的目录结构，按项目生命周期阶段分类
- 确定归档材料的质量标准和验收要求

2. 材料分类整理：

- 成果性材料：源代码、可执行程序、用户手册、技术文档等
- 过程性材料：需求文档、设计文档、测试案例、会议纪要等
- 管理性材料：项目计划、周报、评审记录、总结报告等
- 多媒体材料：演示视频、截图、录音等

3. 质量检查：

- 检查材料的完整性和准确性
- 验证文档版本的一致性
- 确保敏感信息已进行脱敏处理

验收标准：

完成所有项目材料的规范化整理，建立完整的归档目录，材料质量符合标准。

2. 知识资产沉淀与传承

操作指南：

1. 经验文档化：

- 编写技术决策记录，记录重要技术选型的决策过程
- 整理典型问题解决方案，形成故障排查指南
- 总结最佳实践，包括代码规范、协作流程等

2. 模板标准化:

- 将验证有效的文档模板进行标准化
- 制作项目各阶段的检查清单
- 建立常见任务的标准化操作流程

3. 知识库建设:

- 使用 Wiki 或文档管理系统建立团队知识库
- 设计合理的知识分类和检索机制
- 制定知识库的维护和更新机制

验收标准:

形成完整的知识资产包，建立可长期使用的知识管理体系。

3. 团队解散与交接

操作指南:

1. 责任交接:

- 明确各项待办事项的后续负责人
- 完成技术文档和代码的交接说明
- 建立问题咨询和支援机制

2. 经验分享会:

- 组织最终的经验分享会
- 每位成员分享最大的收获和体会
- 总结团队协作的成功经验和改进方向

3. 团队解散仪式:

- 召开正式的团队解散会议
- 感谢团队成员的付出和贡献
- 宣布项目团队正式解散

验收标准:

完成所有交接工作，团队解散流程规范有序。

4. 项目收尾确认

操作指南:

1. 收尾检查清单：

- 验证所有交付物已完成并验收通过
- 确认所有合同和义务已履行完毕
- 检查所有行政流程已闭环

2. 最终报告：

- 编写项目收尾总结报告
- 记录项目的关键指标和成果
- 提出对未来项目的建议

3. 材料提交：

- 按要求提交所有归档材料
- 获取项目结项确认书
- 完成项目相关账户的关闭或移交

验收标准：

完成所有收尾工作，获得项目结项正式确认。

5. 成果输出清单

最终交付物：

- 完整的项目归档包（按要求格式）
- 团队知识库访问信息
- 项目收尾总结报告
- 团队解散会议纪要
- 项目结项确认书

质量要求：

- 所有材料完整、准确、规范
- 知识体系结构清晰、易于使用
- 交接流程完整、无遗留问题

【实验报告】

按附录 1《实验报告参考模版》要求编写，需包含以下核心内容：

实验环境：详细记录硬件配置、软件版本（含 PPT 工具、录屏工具、压缩工具）、演示与答辩环境配置、教学平台信息。

实验过程：按“项目复盘-演示答辩-项目归档-团队解散”四个阶段，详细描述操作步骤、关键决策、工具使用方法，附核心截图。

实验结果及分析：

成果总结：列出项目收尾阶段的所有交付物，说明复盘、答辩、归档的完成情况，总结项目全生命周期的核心成果与价值。

数据统计：《项目总结报告》字数、答辩演示时长、归档材料总数、材料完整率等数据。

问题分析：分析项目收尾过程中存在的核心问题，结合实践说明改进效果与不足。

问题与解决：详细记录实验过程中遇到的具体问题、排查过程、解决方案及效果，每个问题对应实际操作场景。

实验总结：总结项目收尾阶段的收获，反思整个项目的经验与教训，展望未来在类似项目中的改进方向，分析深度 ≥ 400 字。

【实验中的常见问题与解决方案】

1. 问题：项目复盘会陷入相互指责或空泛表扬

- 解决方案：主持人需引导对事不对人的讨论。使用“保持/停止/开始”框架：哪些做法应保持？哪些应停止？哪些可以开始尝试？聚焦流程改进，而非追究责任。

2. 问题：项目材料归档混乱，知识无法传承

- 解决方案：项目启动即定好归档规范（统一的命名、目录结构）。收尾时指定专人按规范整理，并生成一份《项目资产索引》，清晰说明所有交付物的位置和用途。

3. 问题：团队解散潦草，项目收尾不正式

- 解决方案：召开正式的团队解散会议。感谢成员贡献，汇总项目成果，明确后续材料对接人。签署项目结项确认书，形成闭环，培养职业素养。

【实验或拓展思考题】

1. 知识的“蒸发” 项目结束了，但团队积累的知识（尤其是那些“只可意会”的隐性知识）很可能随之流失。除了文档，你还能设计出哪些更生动、更有效的机制，让这些知识能传承给未来接手项目或新加入团队的成员？
2. 成功的“多重定义” 这个项目在你的心目中算成功吗？请从以下几个维度至少选择两个，来评估其成功度，并说明理由：技术成功（架构优雅、代码整洁）、过程成功（协作愉快、流程高效）、业务成功（实现核心价值）、个人成功（能力成长、获得成就感）。你认为哪个维度对你而言最重要？

3. **敏捷的“下一站”** 经过三个迭代的实战，你对“敏捷”的理解是否发生了变化？如果你为团队下一个项目提出一项最大胆的改进建议（例如：尝试一种新的敏捷框架如 Kanban，引入结对编程，甚至试行一周四天工作制），你会提出什么？为什么它值得尝试？

【评分标准】

评分项	权重	评分标准
复盘质量	25%	《项目总结报告》结构完整、数据支撑充分；复盘会议纪要详实，问题根源分析深入，改进建议具体可落地。
答辩效果	30%	答辩 PPT 结构清晰、重点突出；演示录屏完整规范（10-15 分钟），核心成果展示充分；答辩时表达流畅、逻辑清晰，能准确回应提问。
归档完整性与规范性	25%	归档材料分类清晰、命名规范，包含项目全生命周期的核心材料（完整率 $\geq 95\%$ ）；归档目录清晰，压缩包无损坏，提交及时。
团队协作与收尾闭环	10%	复盘、答辩、归档过程中团队分工明确、协作高效；团队解散说明完整规范，项目收尾各项工作闭环无遗留问题。
创新性	10%	复盘总结有独特见解，提炼的经验可复用性强；答辩展示方式有创新（如互动演示、可视化成果展示）。

【参考教材与实验拓展】

参考教材：

《项目复盘：让经验成为资产》（邱昭良 著）、

《演讲的力量》（克里斯·安德森 著）、

《敏捷项目管理指南》（PMI 著）。

实验拓展：

学习项目管理工具（如 Jira、Confluence）的知识管理功能，尝试将本次项目的复盘经验与归档材料录入工具，形成可复用的项目模板。

调研企业级项目收尾的规范流程，对比本次教学实验的收尾流程，分析差异与改进方向。

尝试将项目成果转化为技术博客或开源项目，进一步提升知识传承的范围与价值。

【注意事项】

学术诚信：项目总结报告、答辩 PPT、归档材料需为团队原创，禁止抄袭他人成果；引用外部资料或开源代码需注明来源。

归档安全：归档材料中若包含敏感信息，需提前脱敏处理。

答辩规范：遵守答辩时间要求，不超时、不提前结束；演示时尊重评审人员，保持礼貌用语；团队成员需全程参与答辩，共同回应提问。

责任闭环：团队负责人需统筹协调项目收尾的各项工作，确保复盘、答辩、归档、解散等环节规范落地；每位成员需对自己负责的材料质量负责，确保无遗漏、无错误。

附录 1 实验报告模版



西北工业大学
NORTHWESTERN POLYTECHNICAL UNIVERSITY

课程实验报告

实验名称：_____制造执行系统_____

专业班级：_____

学 号：_____

姓 名：_____

任课教师：_____

XXX 学院

202X 年 X 月 X 日

实验项目编号	001	实验项目名称	敏捷项目启动	
实验学时	X	实验日期	202X.XX.XX	
实验地点	XXX		成绩	

一、实验目的

1. 掌握敏捷开发的核心原理与价值观，理解自组织团队的运作机制。
2. 能够独立完成自组织团队的组建与分工，制定规范的团队章程。
3.

二、实验任务

1. 自主组建 7-9 人项目团队，明确团队成员分工，制定团队协作规则，形成正式的团队相关文档。
2. 明确你团队项目相关的需求目标与范围，通过用户故事梳理核心需求，绘制用户故事地图，形成产品待办列表，完成需求评审与规划。
3.

三、实验环境与工具

（示例，这部分内容需要同学们自己完成）

（本次实验用到的环境与工具以及用途）

- OS: Windows 11
- 工具: MySQL Workbench 8.0

四、实验过程及成果

（示例，这部分内容需要同学们自己完成）

（附实验过程截图与关键步骤说明）

1. 确认项目目标和范围
2. 组建项目团队
3. 编制《项目开发计划》
4.

五、实验问题与解决

（示例，这部分内容需要同学们自己完成）

（此部分内容可选）

- 问题 1: [描述]

解决：修改[配置文件路径]。

– 问题 2：[描述]

解决：修改[配置文件路径]。

六、实验收获与总结

（示例，这部分内容需要同学们自己完成）

在本次实验中，我的主要目标是 XXX。

实验过程与遇到的问题：XXX。

通过本次实验，我掌握了 XXX。

在下次实验中，我应该 XXX。

（备注：总结的格式只作为参考，不强制要求，只要能说明实验收获与反思即可）